Dokey

Engineering a Vietnamese Input Method

Dokey

Engineering a Vietnamese Input Method

Orthography, Telex and VNI, a Pure-Rebuild Engine, and Keystroke Delivery on macOS and Windows

Do Huy Toan

FIRST EDITION, NOVEMBER 2026

Dokey: Engineering a Vietnamese Input Method
Orthography, Telex and VNI, a Pure-Rebuild Engine, and Keystroke Delivery on macOS and Windows

© 2026 Do Huy Toan. All rights reserved.

Only the author, Do Huy Toan, may publish, sell or distribute this book. No other person or organisation may publish, sell, resell, print or reprint it, in whole or in part, in any form. Copyright belongs solely to Do Huy Toan. No part of this book may be reproduced, stored in a retrieval system, or transmitted in any form or by any means, electronic, mechanical, photocopying, recording or otherwise, without the prior written permission of the author, except for brief quotations in reviews and scholarly works with full attribution.

Dokey is proprietary software, © 2026 Do Huy Toan, all rights reserved. Source code excerpts are reproduced to explain the design and grant no licence to the code. Product and company names mentioned in this book may be trademarks of their respective owners; they are used only to identify the products discussed, and no endorsement is implied.

The author has made every effort to ensure that the information in this book is accurate as of its publication. The software and the measurements described relate to Dokey version 0.5.4 (repository commit 675038e). The information is provided without warranty of any kind.

The part-opening illustrations and the cover art were generated with an AI image model under the author's art direction. The technical diagrams were drawn for this book. Claude, an AI assistant made by Anthropic, assisted with drafting and checking the text; the author reviewed all content.

Related work: Do Huy Toan, *Dokey: A Vietnamese Input Method That Rebuilds the Whole Word on Every Keystroke*, working paper, Zenodo, 2026, https://doi.org/10.5281/zenodo.23888629.

Set in Source Serif 4, Inter and JetBrains Mono (SIL Open Font License).

First edition, November 2026.
DOI: https://doi.org/10.5281/zenodo.24015181
https://dhtoan.github.io/ · ORCID 0009-0007-5803-4405

Contents

PART I · THE PROBLEM

1 Typing Vietnamese on a Standard Keyboard . . . 3
- 1.1 What an input method does . . . 3
- 1.2 Where things go wrong . . . 5
- 1.3 What Dokey does differently . . . 6
- 1.4 Scope of the system . . . 7
- 1.5 Privacy . . . 8
- 1.6 How the rest of the book is organised . . . 8

2 Vietnamese Orthography for Engineers . . . 9
- 2.1 The alphabet . . . 9
- 2.2 Diacritics and tones are different things . . . 10
- 2.3 Unicode: precomposed and combining forms . . . 11
- 2.4 The structure of a syllable . . . 12
- 2.5 Spelling constraints that matter to software . . . 14
- 2.6 The size of the inventory . . . 15
- 2.7 Where the tone mark goes . . . 16
- 2.8 Upper case . . . 17
- 2.9 Summary . . . 17

3 Telex and VNI as Formal Rules . . . 19
- 3.1 The key tables . . . 19
- 3.2 The parser as a left fold . . . 20
- 3.3 The step function . . . 22
- 3.4 Worked examples . . . 25
- 3.5 Canonical keys . . . 26
- 3.6 What the rules do not decide . . . 26
- 3.7 Quick Telex . . . 27
- 3.8 Summary . . . 28

PART II · THE ENGINE

4 The Pure-Rebuild Engine . . . 31
- 4.1 Two ways to keep a word . . . 31
- 4.2 The engine's state . . . 32
- 4.3 The path of a key . . . 33
- 4.4 Rendering . . . 34
- 4.5 Committing: the minimal edit . . . 35

4.6 A full trace . . . 36
4.7 Word breaks . . . 37
4.8 Resetting . . . 38
4.9 Properties of the design . . . 39
4.10 Limits . . . 39

5 Syllable Validation as a Gate . . . 41
5.1 Why a gate . . . 41
5.2 The dictionary . . . 42
5.3 Plausibility . . . 43
5.4 Corrections at the end of a word . . . 45
5.5 What the gate cannot do . . . 47
5.6 Spelling checks off . . . 48
5.7 Summary . . . 48

6 Tone Placement and Output Encoding . . . 51
6.1 Why placement is computed, not stored . . . 51
6.2 The placement rules . . . 52
6.3 Testing placement independently . . . 54
6.4 From letters to code points . . . 55
6.5 The combining encoding . . . 56
6.6 Upper case and tones . . . 57
6.7 Summary . . . 57

7 Editing: Backspace, Reopening, Undo and Shortcuts . . . 59
7.1 Backspace inside a word . . . 59
7.2 Reopening the previous word . . . 61
7.3 Undo with Escape . . . 62
7.4 Quick Telex . . . 63
7.5 Text shortcuts . . . 63
7.6 How the features interact . . . 64
7.7 Summary . . . 65

PART III · THE PLATFORMS

8 Minimal Edits and Per-Application Delivery . . . 69
8.1 The edit as a contract . . . 69
8.2 Four ways delivery fails . . . 70
8.3 Delivery strategies . . . 73
8.4 Classifying applications . . . 73
8.5 Why delivery is not in the engine . . . 74
8.6 Summary . . . 75

9 The macOS Platform Layer . . . 77
9.1 Capturing keys with an event tap . . . 77
9.2 Which keys reach the engine . . . 79
9.3 Classifying the front application . . . 80

9.4 Delivery profiles . . . 81
9.5 Delivering an edit . . . 82
9.6 Secure input . . . 84
9.7 Per-application memory and exclusions . . . 85
9.8 Diagnostics . . . 85
9.9 Summary . . . 85

10 The Windows Platform Layer . . . 87
10.1 Capturing keys with a low-level hook . . . 87
10.2 Which keys reach the engine . . . 89
10.3 Keeping corrections in order . . . 90
10.4 Building the input for an edit . . . 92
10.5 Other input methods . . . 93
10.6 Limits of the platform . . . 94
10.7 Settings and diagnostics . . . 94
10.8 Summary . . . 94

11 The C API . . . 95
11.1 Design principles . . . 95
11.2 Types . . . 96
11.3 Functions . . . 97
11.4 Threads and memory . . . 99
11.5 A complete example in C . . . 100
11.6 Binding from Swift . . . 101
11.7 Command-line tool . . . 101
11.8 Stability . . . 101

PART IV · THE EVIDENCE

12 Testing Strategy . . . 105
12.1 Principles . . . 106
12.2 The engine test program . . . 107
12.3 The Korean composer's tests . . . 110
12.4 Platform tests . . . 110
12.5 Manual typing tests . . . 112
12.6 Continuous integration . . . 111
12.7 Summary . . . 113

13 Evaluation and What Is Not Tested . . . 115
13.1 Automated results . . . 115
13.2 Manual typing results . . . 117
13.3 What is not tested . . . 119
13.4 Summary . . . 121

14 Performance . . . 123
14.1 What to measure . . . 123
14.2 Method . . . 124

14.3 Results 124
14.4 Scaling with word length 126
14.5 Where the time goes 128
14.6 Why the cost does not matter 128
14.7 Summary 129

15 Building and Packaging 131
15.1 Repository layout 131
15.2 Building the core 132
15.3 Building the macOS application 132
15.4 Building the Windows application 134
15.5 Continuous integration and release 135
15.6 Self-update 136
15.7 Versioning 136
15.8 Summary 137

16 Lessons and Future Work 139
16.1 Lessons 139
16.2 How the project was built 141
16.3 Future work 142
16.4 Closing 143

APPENDICES

A Rule Tables 147
B Unicode Code Points for Vietnamese Letters 153
C Initials and Rhymes 157
D The Complete Syllable Inventory 159
E Regression Data 177
F Measurement Programs 187
G The Korean Prototype 195
Glossary 199
References 203
Index 205
About the Author 209

Preface

This book describes one piece of software in depth: Dokey, an input method for typing Vietnamese on macOS and Windows. It explains how Vietnamese words are built, how the two common typing conventions (Telex and VNI) can be written down as exact rules, and how an engine can turn a stream of key presses into correct Vietnamese text without ever disagreeing with what is on the screen. It then follows that text out of the engine and into real applications, where most of the practical difficulty lies.

I wrote Dokey because typing Vietnamese on a computer still went wrong in ordinary use. Letters were jumbled or dropped in web browsers and in desktop applications built on Chromium. English words picked up Vietnamese tone marks in the middle of a sentence. The text on the screen and the text that the input method believed it had typed could drift apart, after which every correction made things worse. None of these problems is new, and none of them is caused by Vietnamese itself. They come from the way an input method keeps state and from the way it sends its corrections to other programs.

Dokey takes three positions on those two questions. First, the engine keeps only the raw keys of the word being typed and recomputes the whole word from them on every keystroke, with a pure function. Second, a mark or tone key is applied only when the result can still be a Vietnamese syllable; otherwise the keys are shown as typed. Third, every correction leaves the engine as one minimal edit, a number of characters to delete and a short text to insert, and the platform layer chooses how to deliver that edit according to the kind of application in front of the user. The rest of the book is the detail behind those three sentences.

Who this book is for

The book is written for software engineers who build or maintain text input software: input methods, keyboard layouts, accessibility tools, remote-desktop clients, automation tools and editors that must cooperate with them. It is also written for engineers who work with Vietnamese text in any form and want an exact account of its orthography, and for readers who are interested in how a small, carefully tested system is designed from a written specification.

The reader is assumed to read C and C++ comfortably and to know the basics of Unicode. No knowledge of Vietnamese is assumed. Chapter 2 teaches as much of the language's writing system as an engineer needs, and every Vietnamese example in the book is shown with the keys that produce it.

What this book covers

Part I sets out the problem. Chapter 1 describes what an input method does and where the common failures come from. Chapter 2 is a compact description of Vietnamese orthography for engineers: letters, diacritics, tones, syllable structure and the spelling rules that matter to software. Chapter 3 writes Telex and VNI down as formal rules over key sequences.

Part II is the engine. Chapter 4 presents the pure-rebuild design and walks through the code that implements it. Chapter 5 explains syllable validation and how it is used as a gate, which is the mechanism that keeps English words intact. Chapter 6 covers tone placement and output encodings. Chapter 7 covers the editing features: Backspace, reopening the previous word, undo, Quick Telex and text shortcuts.

Part III is the platform layer. Chapter 8 discusses the minimal edit as a contract between engine and platform, and the delivery strategies that make that edit arrive intact. Chapter 9 describes the macOS layer, built on a session-level event tap, and Chapter 10 describes the Windows layer, built on a low-level keyboard hook and

`SendInput`. Chapter 11 is a reference for the C API that joins the two halves.

Part IV is about evidence. Chapter 12 describes the testing strategy, Chapter 13 reports the evaluation and lists plainly what has not been tested, Chapter 14 measures performance, and Chapter 15 explains how Dokey is built, packaged and released. Chapter 16 closes with the lessons of the project and the work that remains.

The appendices contain the complete rule tables, the Unicode code points for every Vietnamese letter, the initial and rhyme inventories, the full list of 3,957 syllables that the engine accepts, the regression data and the programs used for every measurement in the book.

How the numbers in this book were obtained

Every number in this book was either taken from the Dokey source repository and its documentation, taken from the published working paper on Dokey (DOI 10.5281/zenodo.23888629), or measured by me for this book. Where I measured something, the text says how, on which machine and at which commit, and Appendix F contains the source of the measurement programs. Nothing has been estimated or rounded upward. Where something has not been tested, the book says so; Chapter 13 is devoted to that list.

The book describes Dokey as of version 0.5.4, repository commit `675038e`, dated 3 November 2026. The engine described here is the same engine that version 0.4.0 shipped, with later additions noted where they occur.

Conventions

Keys typed by the user are set in a monospaced font: `tieengs`. The text that appears on screen is set in italics: *tiếng*. An arrow joins the two: `tieengs` → *tiếng*. Special keys are written with symbols: ␣ for Space, ⌫ for Backspace and ⎋ for Escape. Unicode code points are written in the usual form, U+1EBF. File paths are relative to the root of the Dokey repository, for example `src/engine.cpp`. Code list-

ings are excerpts from the repository; where an excerpt has been shortened, the omission is marked.

A note on the software

Dokey is proprietary software. Its source code is published at https://github.com/dhtoan/Dokey for reading and review, under a licence that reserves all rights. The excerpts in this book are reproduced by its author and are © 2026 Do Huy Toan. They are included to explain the design; they do not grant any licence to copy or reuse the code.

Acknowledgements and a statement on AI assistance

Claude, an AI assistant made by Anthropic, assisted with the code of Dokey, with debugging, and with drafting and checking this book. I reviewed every chapter. The tests and measurements reported here were run on my own machine for this book, and I am solely responsible for the content. Anthropic did not sponsor, review or endorse this work.

Do Huy Toan Delaware, November 2026



PART I

The Problem

Chapters 1–3

THE PROBLEM

1

Typing Vietnamese on a Standard Keyboard

Vietnamese is written in the Latin alphabet, but with more letters than a standard keyboard has keys. Seven letters carry a diacritic that changes the sound (ă, â, đ, ê, ô, ơ, ư), and every syllable carries one of six tones, five of which are written as a mark above or below a vowel. A word such as *người* (person) therefore contains a letter with a horn and a tone mark on top of it; *tiếng* (language, sound) contains a letter with both a circumflex and an acute accent.

No common keyboard has keys for these letters. Vietnamese is typed instead through an *input method*: a program that watches the keys the user presses and turns sequences of ordinary keys into the letters that are wanted. This chapter describes what an input method has to do, why the job is harder than it looks, and what Dokey sets out to do differently.

1.1 What an input method does

An input method sits between the keyboard and the application that receives text. In the two conventions used by almost every Vietnamese typist, ordinary keys play a second role:

- In **Telex**, letters double as marks. Typing `a` twice gives *â*; `dd` gives *đ*; `w` after a vowel adds a horn or a breve. A final `s`, `f`, `r`, `x` or `j` adds one of the five written tones. The word *tiếng* is typed `tieengs`.

- In **VNI**, the digit keys are the marks. `6` adds a circumflex, `7` a horn, `8` a breve and `9` the stroke of *đ*; `1` to `5` add the tones. The word *tiếng* is typed `tie61ng` or `tieng61`.

In both conventions the mark keys can come later than the letter they modify. The user types `t`, `i`, `e`, `e`, and the input method must replace the *e* already on screen with *ê*. When the user then types `n`, `g` and `s`, the input method must replace *ê* with *ế*, three characters back from the caret. The input method is therefore not a simple key-to-character map. It is an editor that keeps rewriting the last word on the screen.

That rewriting is the heart of the problem. Applications do not offer input methods a general "replace these characters" operation that works everywhere. On macOS and Windows, an input method that wants to work in every application, including web browsers and terminal emulators, usually does what a person would do: it sends Backspace key presses to delete the old letters and then sends the new letters as key presses. Those key presses are *synthetic*: they are created by software, not by the keyboard, but the application sees them as ordinary keystrokes.

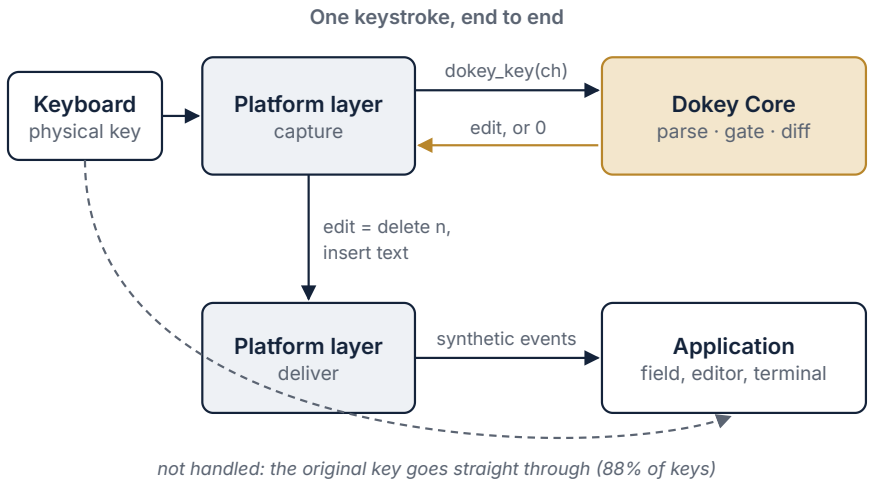


Figure 1.1 The parts of an input method. Keys are captured by the platform layer, interpreted by the engine and written back into the application as synthetic events.

1.2 Where things go wrong

Two kinds of failure are common enough that most Vietnamese computer users will recognise them at once.

Jumbled and lost letters

The first kind is a delivery failure. The input method computes the right word but the application ends up showing something else: letters out of order, letters doubled, letters missing. The Dokey project's own test records contain several real examples, each with the keys typed and the text that appeared:

- In a rich-text editor of the ProseMirror family running in a Chromium-based browser, `Haxy` should give *Hãy*. An early version of Dokey produced *Hayãy*: the editor ignored the synthetic Backspace presses, so the old letters stayed and the corrected word was added after them.
- In a browser address bar, the first Backspace was spent on the browser's own inline suggestion instead of the user's letter, and `ddoox` (*đỗ*) came out as *đđỗ*.
- When a user typed fast, the vowel key was often still held down when the tone key arrived. A synthetic event that carried the code of that held key was treated by Chromium as a key repeat and dropped, so `Haxy` gave *Hy* and `maf` (*mà*) gave *m*.

None of these was a mistake in Vietnamese spelling. Each one came from the path between the input method and the application: an application that processes keys in a separate process, an editor that handles text input events in its own way, a text field with a selection that the input method could not see. Chapter 8 discusses each failure and the delivery strategy that fixed it.

Marks on words that are not Vietnamese

The second kind of failure is a recognition failure. Vietnamese users type English all the time: product names, technical words, commands, URLs. In Telex, the letters `s`, `f`, `r`, `x`, `j` and `w` are mark keys, and they appear in ordinary English words. An input method that applies every mark it can will turn `windows` into *wíndows* and

`master` into *máter*. Users then switch the input method off and on many times a day, or they correct the damage by hand.

The remedy is not obvious. Some English words are also valid Vietnamese syllables when typed in Telex: `test` really does mean *tét* to a Vietnamese typist, and `this` is how one types *thí*. An input method cannot know which language the user meant in those cases. What it can know is whether a sequence of keys *could* be Vietnamese at all. `windows` cannot: there is no Vietnamese syllable that starts with *w*, and none that ends in *ws*. Chapter 5 shows how Dokey uses that knowledge.

Drift between screen and state

There is a third failure that users rarely name but often experience. Most input methods keep an internal model of the word being typed and update it incrementally as each key arrives: add a letter, add a mark, move a tone. If that model ever disagrees with the screen, for example because the user clicked somewhere else, or an application auto-corrected a word, or one synthetic event was lost, then every later correction is computed from the wrong starting point and applied to the wrong letters. The visible symptom is a word that gets steadily worse as the user tries to fix it.

1.3 What Dokey does differently

Dokey is an input method written from scratch, starting on 29 October 2026 (Indochina Time). Version 0.4.0, released on 1 November 2026, is the version described in the published working paper; version 0.5.4, released on 3 November 2026, is the version described in this book. It runs on macOS from 10.13 (one universal binary for Intel and Apple Silicon) and on Windows 10 and 11 (builds for x86, x64 and ARM64). Its design rests on three decisions.

Rebuild the whole word on every key. The engine keeps the raw keys of the current word, exactly as the parser reads them. After every key it recomputes the displayed word from the complete key sequence, with a pure function: the same keys always produce the same word. There is no incremental model that can drift. The output to the platform is a single edit, computed by comparing the

word currently shown with the new word. Chapter 4 describes the design and Chapter 14 shows that its cost, under one microsecond per key on a 2014 desktop processor, is negligible.

Use spelling as a gate. A mark or tone key is applied only if the result is still the beginning of a valid Vietnamese syllable. Otherwise the key is printed as a letter. When spelling checks are on and the word on screen stops being a valid start of a syllable, the engine shows exactly the keys that were typed. That single rule keeps `windows`, `chrome`, `github` and more than a thousand other English and developer words intact, as the regression tests in Chapter 12 show.

Send each correction as one minimal edit, chosen per application. Because the engine compares the old and new words, it can report the smallest change: delete *n* characters, insert this text. When the change is just "insert the key that was typed", the engine reports that it did nothing, and the original key goes through untouched. In the regression corpus, 88 per cent of keys pass through this way. For the rest, the platform layer chooses a delivery strategy suited to the application: Backspace presses for ordinary applications, selection and overtyping for browsers and Electron applications, a guard character for text fields with inline suggestions. Chapters 8 to 10 describe those strategies.

1.4 Scope of the system

Dokey consists of three parts, kept deliberately separate.

Dokey Core is the engine: a C++17 static library with no external dependencies. It contains the Telex and VNI parser, the syllable tables, tone placement, the editing features and a small C API. At the commit described in this book, the library's source files and its two main public headers total 1,584 lines. Of those, 252 are a prototype Korean composer that shares the same API (Appendix G); the remaining 1,332 lines are the Vietnamese engine, its tables and the C API. The core also builds a command-line tool, `dokey`, that converts typed text and is useful for testing.

The macOS application is written in Swift with AppKit. It captures keys with a session-level `CGEventTap`, passes them to the engine

through the C API, and delivers the resulting edits as synthetic keyboard events.

The Windows application is written in C++ with Win32 and GDI+. It captures keys with a low-level keyboard hook, `WH_KEYBOARD_LL`, and delivers edits with `SendInput`.

Both applications include a control panel, per-application memory of the Vietnamese or English mode, a list of excluded applications, text shortcuts and an update checker. This book concentrates on the parts that concern typing. The control panels and installers are described only as far as building and packaging require.

1.5 Privacy

An input method sees everything the user types, including passwords where the operating system allows it. Dokey's position is simple: it makes no network connections except a check for new releases against GitHub's public release interface, which the user can turn off, and it never stores keystrokes. Both applications have an optional debug log, off by default. On macOS the log records the edits that Dokey sends and so contains the words typed; it stays on the user's machine. On Windows the log records events and counts only, never which key was pressed. Chapter 9 and Chapter 10 give the details.

1.6 How the rest of the book is organised

The next two chapters describe the inputs to the engine: the writing system it must produce and the typing conventions it must understand. Readers who already know Vietnamese well can skim Chapter 2, but should read its sections on syllable structure and spelling constraints, because those constraints become the gate of Chapter 5. Readers who only want the engineering of event delivery can start at Chapter 8 and return to the engine later; every chapter in Part III states which engine behaviour it depends on.

THE PROBLEM

2

Vietnamese Orthography for Engineers

An input method cannot be better than its model of the language it types. This chapter describes the Vietnamese writing system from the point of view of software that must produce it: which letters exist, how diacritics and tones combine, how a syllable is built, which combinations are allowed, and where the tone mark goes. It does not attempt a full account of Vietnamese phonology. Everything here is public knowledge, found in any grammar of the language, and every rule stated in this chapter corresponds to a table or a function in Dokey Core.

2.1 The alphabet

Modern Vietnamese writing, *chữ Quốc ngữ*, uses 29 letters. Twenty-two are the familiar Latin letters without *f*, *j*, *w* and *z*; the other seven carry a diacritic:

- **ă** (a with breve), **â** (a with circumflex),
- **ê** (e with circumflex),
- **ô** (o with circumflex), **ơ** (o with horn),
- **ư** (u with horn),
- **đ** (d with stroke).

The letters *f*, *j*, *w* and *z* do not occur in native Vietnamese words, which is why Telex is free to use them as mark keys. They do occur

in loanwords, names and foreign text, which is one reason the engine must be able to leave a word alone.

For software the important division is between **vowel letters** and everything else. There are twelve vowel letters: *a, ă, â, e, ê, i, o, ô, ơ, u, ư, y*. Each of them can carry a tone mark, so there are 12 × 6 = 72 lower-case forms of vowels with and without tones, and 72 upper-case forms. Together with *đ* and *Đ*, those 146 characters are all that Vietnamese adds to basic Latin. Dokey stores them as two 12 × 6 tables of UTF-16 code units, reproduced in Appendix B.

2.2 Diacritics and tones are different things

A Vietnamese letter may carry two independent marks, and it is essential not to confuse them.

A **diacritic** in this book's sense (the Dokey source calls it a *mark*) changes the letter. *a*, *ă* and *â* are three different vowels, as different to a Vietnamese reader as *a* and *o* are to an English one. The four diacritics are the **circumflex** (â, ê, ô), the **breve** (ă), the **horn** (ơ, ư) and the **stroke** (đ). A letter has at most one diacritic.

A **tone** changes the pitch of the whole syllable. Vietnamese has six tones. One, *ngang* (level), has no mark. The other five are written with a mark on one vowel of the syllable:

Table 2.1 The six tones.

Tone	Name	Mark	Example	Combining code point
1	ngang (level)	none	*ma*	—
2	sắc (rising)	acute	*má*	U+0301
3	huyền (falling)	grave	*mà*	U+0300
4	hỏi (dipping)	hook above	*mả*	U+0309
5	ngã (broken rising)	tilde	*mã*	U+0303
6	nặng (heavy)	dot below	*mạ*	U+0323

A syllable has exactly one tone, and the tone mark sits on exactly one vowel. A vowel can carry a diacritic and a tone at the same time, as in *ế* (e + circumflex + acute) or *ợ* (o + horn + dot below).

In Dokey's code, the two marks are separate enumerations:

```
enum class Mark : unsigned char { None, Circumflex, Breve, Horn, Stroke
};
enum class Tone : unsigned char { None = 0, Acute, Grave, Hook, Tilde,
Dot };
```

A letter in the engine is a base letter (`'a'` to `'z'`), a `Mark`, and an upper-case flag. The tone is not stored on any letter at all. It belongs to the word, and its position is computed only when the word is drawn (Chapter 6). That choice removes a whole class of bugs in which a tone ends up on the wrong vowel after an edit.

2.3 Unicode: precomposed and combining forms

Unicode can represent *ế* in more than one way. The **precomposed** form is a single code point, U+1EBF LATIN SMALL LETTER E WITH CIRCUMFLEX AND ACUTE. The **decomposed** forms build it from a base letter and combining marks: *e* + U+0302 COMBINING CIRCUMFLEX ACCENT + U+0301 COMBINING ACUTE ACCENT, or *ê* (U+00EA) + U+0301. Unicode Normalization Form C (NFC) prefers the precomposed form when one exists, and every Vietnamese letter with every tone has a precomposed form in the Latin Extended Additional block (U+1EA0 to U+1EF9) or earlier blocks.

Dokey offers two encodings:

- **Unicode (precomposed, NFC)**, the default. Every letter is one UTF-16 code unit. A word of *n* letters is *n* code units long, which makes edits easy to count.
- **Unicode with combining tone marks.** Letters with a diacritic stay precomposed (*ê*, *ơ*), but the tone is written as a separate combining character after the vowel. *tiếng* becomes *t, i, ê, U+0301, n, g*: six code units for five visible letters. Some older software and some fonts handle this form better.

Several legacy eight-bit encodings (TCVN3, VNI Windows, CP1258) are still found in old documents. Dokey does not produce them; they are listed as future work in Chapter 16.

2.4 The structure of a syllable

Vietnamese is written one syllable at a time, with spaces between syllables. A word such as *Việt Nam* is two written syllables. For an input method this is a gift: the unit that the engine must rebuild is never longer than one syllable, and a syllable is short. In the inventory used by Dokey, the longest syllable has seven letters (*nghiêng*), and most have three to five.

A written syllable can be analysed into an **initial** consonant and a **rhyme**, and the rhyme into an optional **medial** (a glide, written *o* or *u*), a **nucleus** (the main vowel), and an optional **final** (a consonant or a semivowel). The tone belongs to the syllable as a whole.

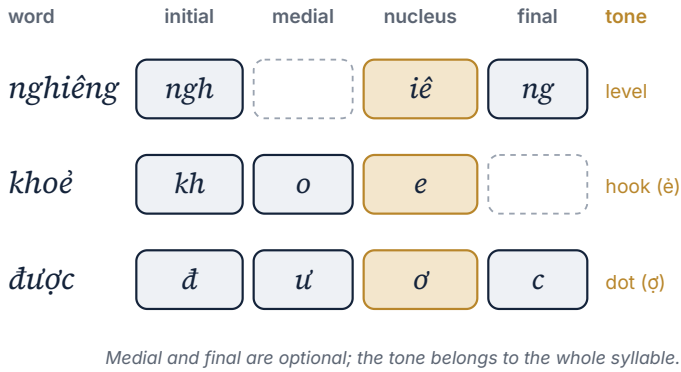


Figure 2.1 The parts of a written Vietnamese syllable, shown for *nghiêng*, *khoẻ* and *được*. The medial and final are optional; every syllable has a nucleus and a tone.

Dokey does not split rhymes into medial, nucleus and final. It keeps two lists, one of written initials and one of written rhymes, and builds syllables by combining them under a few constraints. That is simpler than a phonological model and exactly sufficient for spelling. The lists are shown in full in Appendix C.

Initials

Dokey's table lists 28 initials, counting the empty initial of syllables that begin with a vowel:

> (none), b, c, ch, d, đ, g, gh, gi, h, k, kh, l, m, n, ng, ngh, nh, p, ph, qu, r, s, t, th, tr, v, x

and one more, *kr*, used in place names of the Central Highlands such as *Krông Búk* and *Krông Pắc*. These names come from minority languages of the region and are written with spellings outside the standard inventory. Dokey admits *kr* and the rhymes *ăk* and *uk* so that the provinces and districts of the region can be typed as they appear on official maps (*Đắk Lắk*, *Krông Búk*).

Several initials are pairs of letters that sound the same but are written differently according to the following vowel. Those pairs are the source of the main spelling constraints.

Rhymes

The rhyme table in `src/vietnamese.cpp` lists 156 written rhymes, from *a* to *y*, as they are spelled after an ordinary initial consonant. A further seven rhymes occur only after *qu* (*quy*, *quyên*, *quyết*, *quynh*, *quých*, *quýt*, *quýp*), because in those syllables the *u* belongs to the initial and the rhyme starts with *y*. The tables are linguistic data, written out by hand, and they are the only place where the engine encodes which vowel and consonant combinations exist.

Finals and checked syllables

A syllable can end in a vowel, in a semivowel (*i*, *y*, *o*, *u*), in a nasal (*m*, *n*, *ng*, *nh*) or in a stop (*c*, *ch*, *p*, *t*). Syllables ending in a stop are called **checked** syllables. In Vietnamese they can carry only two of the six tones: *sắc* (acute) and *nặng* (dot below). *học*, *sách*, *tập* and *một* are correct; a word such as *hòc* cannot exist.

Dokey treats the Central Highlands final *k* (*Đắk*, *Búk*) like the other stops. The rule appears in the engine as a single function:

```
bool stopFinal(const std::vector<Letter>& w) {
    size_t b, e;
    vowelRun(w, b, e);
    if (e == b) return false;
    std::u16string fin = lowerMarked(std::vector<Letter>(w.begin() + e,
w.end()));
    return fin == u"c" || fin == u"ch" || fin == u"p" || fin == u"t" ||
```

```
fin == u"k";
}
```

`toneAllowed` uses it to refuse the grave, hook and tilde tones on checked syllables. A user who types `hocf` therefore does not get an impossible word; Chapter 5 explains what happens instead.

2.5 Spelling constraints that matter to software

Vietnamese spelling is highly regular, and a handful of constraints decide which combinations of initial and rhyme are written. Dokey enforces these when it builds its syllable inventory.

c, k and q. The sound /k/ is written *k* before the front vowels *e, ê, i* and *y*, and *c* elsewhere: *kê, kia, ký*, but *ca, cô, cũ*. Before the glide /w/ it is written *qu*: *qua, quê, quyết*. In Dokey: *k* combines only with rhymes that start with a front vowel; *c* combines only with rhymes that do not; *q* never appears without *u*.

g and gh, ng and ngh. In the same way, *gh* and *ngh* are written before *e, ê* and *i*, and *g* and *ng* before everything else: *ghế, nghĩ*, but *gà, ngủ*. *gh* and *ngh* never appear before *y*.

gi. The initial *gi* is written with an *i* that is part of the consonant. When the rhyme itself starts with *i*, one *i* is dropped: *gi* + *ì* is written *gì*, not *giì*; *gi* + *in* is *gìn*; *gi* + *iêng* is *giêng*. For tone placement the *i* of *gi* does not count as a vowel when another vowel follows (*giá, giữ*), but it carries the tone when it is the only vowel (*gì*).

qu. Similarly, the *u* of *qu* is part of the initial. In *quá* the tone sits on *a*; in *quý* on *y*. Only some rhymes may follow *qu*. Dokey admits the rhymes beginning with *a, ă, â, e, ê* and *ơ*, the rhymes of up to three letters beginning with *i* (*qui, quit, quinh*), and the seven *qu*-only rhymes listed above.

iê and yê. The diphthong /iə/ is written *iê* after a consonant (*tiếng, biển*) and *yê* when no consonant precedes it (*yêu, yến*). After the glide it is also *yê* (*khuyên, quyết*). Dokey's dictionary builder rewrites *iê* to *yê* when the initial is empty.

ươ and uơ. The diphthong /ɯə/ is written *ươ* when a final follows (*người, được, hương*). In the rare open syllables *thuở* and *huơ*, it is

written *ươ*: the *u* has no horn. This is the only place where the engine changes a letter when the word ends (Chapter 5).

The full constraint set is shown in Table 2.2 together with the code that applies it.

Table 2.2 Spelling constraints and where Dokey Core applies them.

Constraint	Examples	Applied in
k only before e, ê, i, y	*kê*, *kia*, *ký*	`Dictionary()`
c, *g*, *ng* not before e, ê, i (or y)	*ca*, *gà*, *ngủ*	`Dictionary()`
gh, *ngh* only before e, ê, i	*ghế*, *nghĩ*	`Dictionary()`
q only as *qu*, with a restricted set of rhymes	*quá*, *quyết*	`Dictionary()`
gi + rhyme starting with *i* drops one *i*	*gì*, *gìn*, *giêng*	`Dictionary()`
iê written *yê* without an initial	*yêu*, *yến*	`Dictionary()`
checked syllables take only acute or dot below	*học*, *một*, *Đắk*	`toneAllowed()`
ươ at the end of a word is *ươ*	*thuở*, *huơ*	`Engine::render()`
ư + *o* before a final is *ươ*	*ươn*, *được*	`completeUo()`

2.6 The size of the inventory

Combining 29 initials with 156 rhymes under these constraints, plus the *qu*-only rhymes, Dokey's dictionary builder generates 3,964 entries. Seven of them are duplicates, produced when two different initial and rhyme pairs spell the same string; all seven involve *gi* (*gia* is both *gi* + *a* and *gi* + *ia*). The dictionary therefore contains **3,957 distinct syllables without tone**. I counted these figures for this book with a small program that reads the engine's own tables (Appendix F); the complete list is printed in Appendix D.

Of the 3,957 syllables, 1,424 are checked syllables, which can take three tone forms (none, acute, dot below), and 2,533 are open or end in a nasal or semivowel, which can take all six. The number of written syllables with tone that Dokey can produce is therefore $1{,}424 \times 3 + 2{,}533 \times 6 = 19{,}470$.

It is important to say what this inventory is not. It is not a list of attested Vietnamese syllables. It is the set of strings that the spelling rules *permit*, and it contains combinations that no Vietnamese

word uses. For an input method that is the right choice. The purpose of the dictionary is to decide whether a sequence of keys can be Vietnamese, not whether it is a word that someone has used before. A dictionary that is too small refuses names and rare words that users need; a dictionary that follows the spelling rules refuses only what cannot be written.

Table 2.3 The syllable inventory by length (letters, without tone).

Letters	Syllables
1	10
2	223
3	1,276
4	1,559
5	743
6	145
7	1
Total	**3,957**

Earlier Dokey documentation gives a dictionary size of 3,759 syllables. That is the number of generated entries (3,752 of them distinct) in versions 0.1.0 to 0.3.4, before the Central Highlands initial *kr* and the rhymes *ăk* and *uk* were added in version 0.4. I checked this by compiling the tables of the `v0.1.0` and `v0.3.4` tags. The counts in this book are measured at commit `675038e`.

2.7 Where the tone mark goes

Because the tone belongs to the syllable but is written on one vowel, every Vietnamese writer and every input method must decide which vowel gets the mark. For most syllables the answer is fixed by convention; for a few there are two conventions in use.

Dokey's rules, from `SPEC.md`, consider the vowel group of the syllable, not counting the *u* of *qu* or the *i* of *gi* when another vowel follows:

1. If a vowel in the group carries a diacritic (*ă, â, ê, ô, ơ, ư*), that vowel takes the tone. In *ươ*, the tone goes on *ơ*: *người*, *được*.

2. Otherwise, if a final consonant follows the group, the last vowel of the group takes the tone: *toán*, *hoàn*, *huýt*.
3. Otherwise, in a group of three vowels, the middle vowel takes it: *ngoài*, *xoáy*.
4. Otherwise, in a group of two vowels, the first vowel takes it: *mía*, *mùa*, *táy*. The exceptions are *oa*, *oe* and *uy*, which depend on the chosen style.
5. A single vowel takes the tone.

The exceptions in rule 4 are a matter of style, not correctness. In the **modern style** the tone goes on the second vowel: *hoà*, *khoẻ*, *thuý*. In the **traditional style** it goes on the first: *hòa*, *khỏe*, *thúy*. Both are in common use. Dokey defaults to the modern style and offers the traditional one as an option. Chapter 6 shows how the rules are implemented, with a decision diagram.

2.8 Upper case

Case in Vietnamese follows the base letter. *Đường* is *đường* with a capital *Đ*; *TIẾNG* is *tiếng* in capitals. Diacritics and tones are kept on capitals. Dokey stores case per letter, taken from the key that typed the letter, and the mark keys never change it: `Aan` gives *Ân*, `DDuowngf` gives *Đường*, and `TIEENGS` gives *TIẾNG*.

2.9 Summary

For an input method, Vietnamese orthography reduces to a small and exact model: twelve vowel letters and *đ*; four diacritics, at most one per letter; six tones, exactly one per syllable, written on one vowel by fixed rules; and a syllable inventory generated from 29 initials and 156 rhymes (plus seven that follow only *qu*) under a handful of spelling constraints. The next chapter describes how the keys of Telex and VNI map onto that model.

THE PROBLEM

3

Telex and VNI as Formal Rules

Telex and VNI are conventions, not standards. They were spread by teaching and by the input methods that implemented them, and each implementation interprets the edge cases in its own way. A user who types `tieengs` expects *tiếng* in every Vietnamese input method; a user who types `ddd`, `uow` or `tienges` may get different results in different programs. To build an engine that behaves predictably, Dokey's specification writes both conventions down as rules. This chapter presents those rules, first informally as key tables, then as the exact procedure that the engine's parser follows.

3.1 The key tables

Telex

Keys	Effect	Example
`aa ee oo`	circumflex: *â ê ô*	`caan` → *cân*
`aw`	breve: *ă*	`awn` → *ăn*
`ow uw`	horn: *ơ ư*	`mow` → *mơ*
`uow`	horn on both: *ươ*	`huowng` → *hương*
`w` alone	*ư*	`nhw` → *như*
`dd`	stroke: *đ* (first letter only)	`ddi` → *đi*
`s f r x j`	tone: acute, grave, hook, tilde, dot	`maf` → *mà*
`z`	remove the tone	`masz` → *ma*

VNI

Keys	Effect	Example
`6`	circumflex on *a, e* or *o*	`ca6n` → *cân*
`7`	horn: *ơ ư*; `uo7` gives *ươ*	`huo7ng` → *hương*
`8`	breve: *ă*	`a8n` → *ăn*
`9`	stroke: *đ*	`d9i` → *đi*
`1 2 3 4 5`	tone: acute, grave, hook, tilde, dot	`ma2` → *mà*
`0`	remove the tone	`ma10` → *ma*

Rules common to both

- **Free order.** A mark key can follow the letter it modifies at any distance within the word. `tieengs`, `tienges` and `tiengse` all give *tiếng*. In VNI, `tie61ng`, `tieng61` and `tieng16` all give *tiếng*.
- **Repeat to undo.** Pressing the same mark key again, immediately after it was applied, removes the mark and prints the key as a letter: `aaa` → *aa*, `ass` → *as*, `ddd` → *dd*, `a11` → *a1*. After an undo, that key types a plain letter for the rest of the word.
- **Change of tone.** A different tone key replaces the tone: `asf` → *à*.
- **Case.** Each letter keeps the case of the key that typed it. Mark keys do not affect case: `Aan` → *Ân*, `DDuowngf` → *Đường*.
- **Word breaks.** Space, punctuation, Enter and Tab end the current word and pass through unchanged. In Telex, digits also end the word; in VNI they are mark keys inside a word, and a digit at the start of a word begins a number, not a word (`2026` stays *2026*).

These tables are what users learn. They leave many questions open. Which *a* does `aa` modify in a word that has two? What does `w` do after *uo*, after *ua*, after a consonant? Is `dd` a stroke in the middle of a word? What happens when a tone key arrives before any vowel? The rest of this chapter answers those questions by describing the procedure exactly.

3.2 The parser as a left fold

Dokey's parser reads the raw keys of the current word one at a time, from left to right, and builds a value of type `Word`:

```
struct Letter {
    char base;          // 'a'..'z'
    Mark mark;
    bool upper;
    bool standaloneW;   // "ư" created by a lone Telex w
};

struct Word {
    std::vector<Letter> letters;
    Tone tone = Tone::None;
    bool disabled[kClassCount] = {};
    std::u16string typed;   // what to show if the word is not
Vietnamese
    bool undid = false;     // the last key cancelled a modifier
    int applied = -1;       // KeyClass applied by the current key
    int previous = -1;      // KeyClass applied by the previous key
};
```

Formally, if $k_1 \dots k_n$ are the raw keys, the parser computes

> W_0 = empty word, $W_i = \text{feed}(W_{i-1}, k_i)$, result = W_n

where `feed` is a function of the current word and one key. There is no other input. In particular the parser does not look at the screen, at the previous result, or at any state outside the key sequence. This is the property on which the whole design of Chapter 4 rests: given the same keys, the parser always builds the same `Word`.

The `Word` has four parts:

- `letters`: the letters so far, each with its base, diacritic and case;
- `tone`: the single tone of the word, not yet attached to any letter;
- `disabled`: the set of *key classes* that have been undone in this word and now type literally;
- `typed`: the keys exactly as they would appear if the word were not Vietnamese.

The key classes are the groups of mark keys that share an undo state:

```
enum KeyClass { kCircA, kCircE, kCircO, kHorn, kBreve, kStroke, kTone,
kClassCount };
```

Telex has separate classes for `aa`, `ee` and `oo`, because `a`, `e` and `o` are different keys. All five tone keys share one class, `kTone`: after `ass` has undone the acute tone, `f` still cannot add a grave tone to that word. Telex `w` both horns and breves and therefore shares a single undo state across `kHorn` and `kBreve`.

3.3 The step function

Each call to `feed` does the following.

1. Record `previous` = the class applied by the last key, and clear `applied` and `undid`.
2. Map the key to an action according to the input method (Table 3.1). Letters are compared in lower case; the original key is kept for its case.
3. Try the action. An action either succeeds, changing the word and setting `applied` to its class; or performs an undo; or fails.
4. If the action did not use the key, append the key as a plain letter.
5. Unless the key was an undo, append it to `typed`.

Table 3.1 Actions by key.

Telex key	VNI key	Action
`s f r x j`	`1`–`5`	`applyTone(tone)`
`z`	`0`	remove the tone, if there is one and the tone class is not disabled
`a`	`6` (on *a*)	`applyCircumflex(kCircA)`
`e`	`6` (on *e*)	`applyCircumflex(kCircE)`
`o`	`6` (on *o*)	`applyCircumflex(kCircO)`
`w`	`7`	`applyHornBreve(horn)`; Telex also breve and stand-alone *ư*
—	`8`	`applyHornBreve(breve)`
`d`	`9`	`applyStroke()`
any other letter	other letters	none: append as a letter

In VNI, key `6` uses a single class (`kCircA`) and looks for any of *a*, *e* or *o*; in Telex the circumflex key must match the vowel (`a` for *a*, and so

on). That difference is why `ae` in Telex is two letters, while `a6e6` in VNI is never needed: one `6` marks the nearest eligible vowel.

The vowel group

Most actions work on the **vowel group**: the run of vowel letters (*a, e, i, o, u, y*, with any diacritic) that forms the nucleus of the syllable. The function `vowelRun` finds it, skipping the *u* of *qu* and the *i* of *gi* when another vowel follows:

```
void vowelRun(const std::vector<Letter>& w, size_t& begin, size_t& end)
{
    const size_t n = w.size();
    size_t init = 0;
    auto vowelAt = [&](size_t i) { return i < n &&
isVowelBase(w[i].base); };
    if (n > 2 && w[0].base == 'q' && w[1].base == 'u' && vowelAt(2)) {
        init = 2;
    } else if (n > 2 && w[0].base == 'g' && w[1].base == 'i'
               && w[1].mark == Mark::None && vowelAt(2)) {
        init = 2;
    } else {
        while (init < n && !isVowelBase(w[init].base)) init++;
    }
    begin = init;
    end = init;
    while (end < n && isVowelBase(w[end].base)) end++;
}
```

Only the first run of vowels counts. In a word such as `windows`, which is not Vietnamese, the vowel group is the *i* alone; the later *o* is outside it and is never marked.

Tones

`applyTone` succeeds if the tone class is not disabled, the word has a vowel, and, when spelling checks are on, the result is still plausible (Chapter 5). If the word already has the same tone and the previous key applied a tone, the key is an undo: the tone is removed, the class is disabled, the key is appended as a letter, and the key is not recorded in `typed` because the undo replaces the key it cancels. If the word already has the same tone but the previous key was something else, the key fails and becomes a letter.

The tone is stored once for the whole word. Its position is not decided here at all.

Circumflex

`applyCircumflex` scans the vowel group from the end towards the start, looking for a letter whose base matches the key (`a` for Telex `a`; any of *a, e, o* for VNI `6`):

- If that letter already has a circumflex, the key is an undo, but only if the previous key applied the same class. Otherwise the key fails.
- If the letter carries a different diacritic, the scan continues to an earlier vowel.
- If the letter is unmarked, the engine tries the circumflex on it. If the result is plausible, the mark is applied; if not, the key fails. The scan does not try earlier vowels after a refusal.

Scanning from the end means the *last* eligible vowel is marked. In practice a Vietnamese vowel group has at most one vowel that can take a circumflex, so the choice rarely matters; where it could, the last vowel is the one closest to the caret and therefore the one the user most likely meant.

Horn and breve

Telex `w` and VNI `7` and `8` go through `applyHornBreve`, which is the most involved action because *ư* and *ơ* often occur together:

1. Collect the vowels that already carry the requested mark.
2. Look for an adjacent *u, o* pair in the vowel group. If both already have horns, the key is an undo (if the previous key applied the class). If *o* is unmarked and *u* is unmarked or already horned, try putting a horn on both. This is how `uow`, `uwow` and VNI `uo7` all give *ươ*.
3. Otherwise scan the vowel group from the end for the last vowel that accepts the mark (*o* or *u* for horn, *a* for breve), skipping letters that already have it or have another diacritic, and apply it if plausible.

4. If every candidate already had the mark and the previous key applied the class, undo.
5. Telex only: if there is no vowel group at all, `w` itself becomes the letter *ư*, flagged as stand-alone (`nhw` → *như*, `w` → *ư*).

The stand-alone flag matters for undo. Typing `ww` gives *w*: the second `w` turns the stand-alone *ư* back into the letter *w*, instead of leaving *ư* and appending another *w*.

Stroke

`applyStroke` applies only when the **first** letter of the word is *d*. A second `d` (Telex) or `9` (VNI) then turns it into *đ*. A *d* elsewhere in the word is never stroked, so `add` stays *add*. A third `d` undoes the stroke: `ddd` → *dd*.

3.4 Worked examples

The program in Appendix F prints, for any key sequence, what the engine shows after each key. The traces below come from that program run on the engine at commit `675038e`.

`tieengs` → *tiếng*:

Key	Shown	Note
`t i e`	*tie*	letters
`e`	*tiê*	circumflex on the *e* of the vowel group
`n g`	*tiêng*	letters
`s`	*tiếng*	acute tone, placed on *ê* (diacritic rule)

`tiengse` → *tiếng*, the same word with the marks in a different order:

Key	Shown	Note
`t i e n g`	*tieng*	letters; *tieng* is a valid prefix (it can become *tiêng*)
`s`	*tiéng*	acute tone; placed on the last vowel because a final follows
`e`	*tiếng*	circumflex on the *e*; the tone moves to *ê*

`dduowcj` → *đước* and `dduocjw` → *đước*:

Keys	`dduowcj` shows	`dduocjw` shows
`dd`	*đ*	*đ*
`u o`	*đuo*	*đuo*
5th key	`w`: *đươ*	`c`: *đuoc*
6th key	`c`: *đươc*	`j`: *đuọc*
7th key	`j`: *được*	`w`: *được*

`aaa` → *aa*: the first `a` is a letter, the second applies the circumflex (*â*), and the third, repeating the class just applied, undoes it and prints *a*.

3.5 Canonical keys

The parser accepts many key sequences for the same word. For Backspace (Chapter 7) the engine needs the reverse: given a word on screen, a key sequence that produces it. `encodeKeys` writes the **canonical** sequence: each letter followed immediately by its diacritic key, and the tone key once at the end.

Word	Canonical Telex	Canonical VNI
tiếng	`tieengs`	`tie6ng1`
đường	`dduowngf`	`d9u7o7ng2`
Việt	`Vieetj`	`Vie6t5`

In Telex the canonical form of *ươ* is `uwow`; the parser also accepts `uow`, and `uwo` followed by a final consonant. The test suite's round trip (Chapter 12) uses an independent implementation of the same canonical form to type all 3,957 syllables with every allowed tone.

3.6 What the rules do not decide

Two decisions are deliberately left out of the parser.

Where the tone is drawn. The parser records *that* the word has an acute tone, not which letter carries it. Placement is a function of the finished letter sequence, computed by `tonePosition` when the word is rendered (Chapter 6). This is why a tone typed early moves

by itself when a later key changes the vowels: in `tiengse` above, the acute moved from *e* to *ê*.

Whether the word is Vietnamese. The parser applies a mark only if the result is plausible, but it does not decide what to show for a word that is not. That decision is made when the word is rendered, using the `typed` field that the parser keeps alongside the letters. Chapter 5 describes it.

The separation keeps each function small and each rule in one place. It also means that Telex and VNI differ only in the table that maps keys to actions; everything after that point is shared.

3.7 Quick Telex

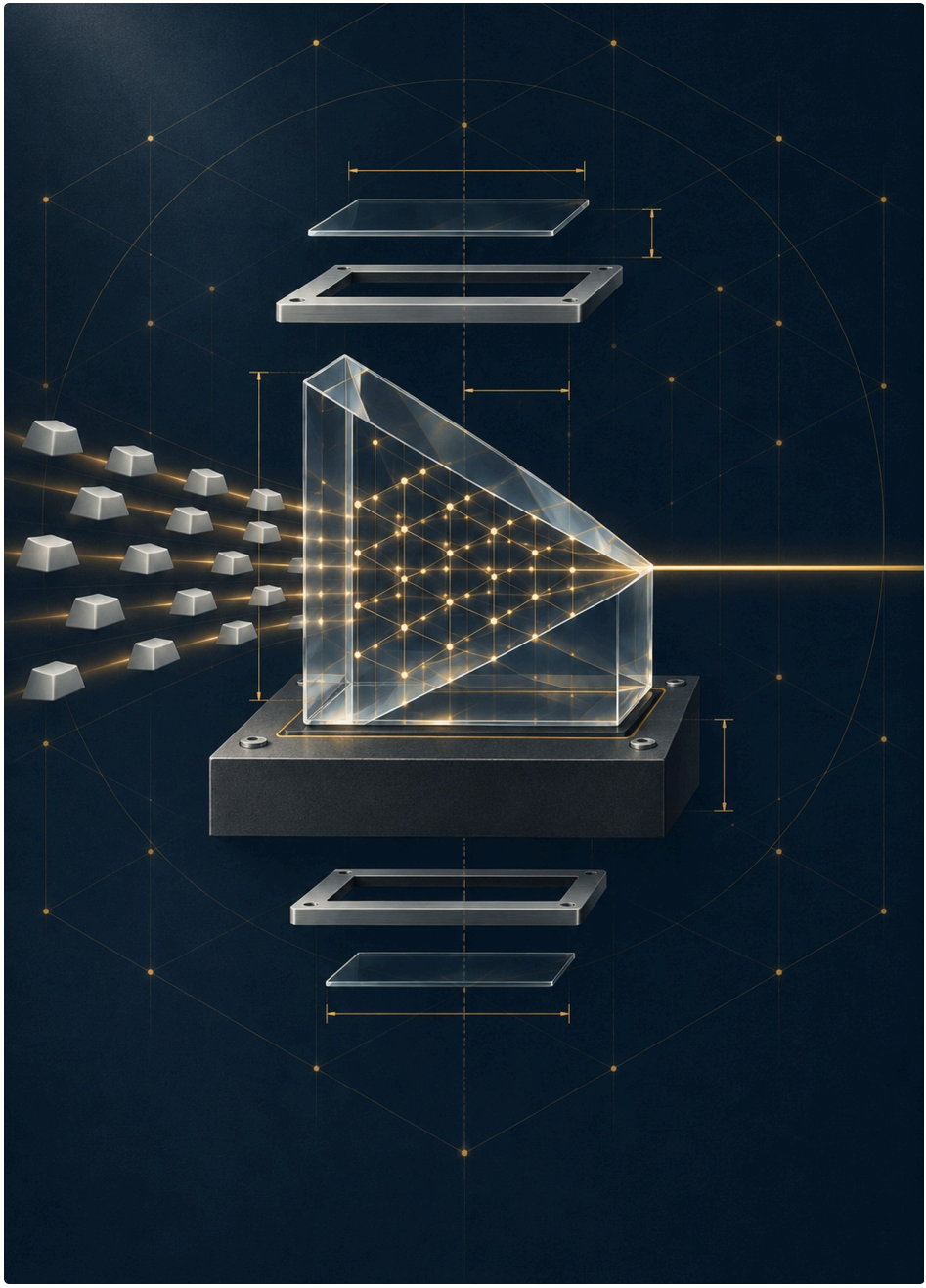
Dokey also implements an optional extension to Telex, called Quick Telex in the user interface. At the start of a word, doubling one of the consonants *c, g, k, n, p, q, t* types a two-letter initial:

Keys	Initial
`cc`	*ch*
`gg`	*gi*
`kk`	*kh*
`nn`	*ng*
`pp`	*ph*
`qq`	*qu*
`tt`	*th*

So `ccaof nnayf` gives *chào ngày*. Quick Telex is off by default, because it changes the meaning of keys that English words use (`app`, `accept`). With the option on, `app` and `accept` still come out unchanged, because the shortcut applies only when the doubled consonant forms the first two keys of the word; in `app` the doubled *p* comes after a vowel. Quick Telex is implemented in `Engine::key` before the parser runs, by appending the second letter of the pair to the raw keys instead of the doubled consonant.

3.8 Summary

Written as rules, Telex and VNI become a left fold over the key sequence with five actions: tone, circumflex, horn or breve, stroke, and plain letter. Each action works on the vowel group, succeeds only if the result can still be Vietnamese, and can be undone by repeating its key. The parser keeps a second, literal view of the keys in `typed`, and it leaves tone placement and the final language decision to later stages. The next chapter shows how the engine runs this parser on every keystroke and turns its result into a single edit.



PART II

The Engine

Chapters 4–7

THE ENGINE

4

The Pure-Rebuild Engine

The engine is the part of Dokey that decides what the current word should look like. It receives one character at a time from the platform layer and answers each one with an edit: how many characters to delete before the caret, and what text to insert. This chapter describes the engine's state, the path a key takes through it, and the reasons for the one design decision that shapes everything else: the word is rebuilt from scratch on every keystroke.

4.1 Two ways to keep a word

An input method can keep the word being typed in one of two ways.

The **incremental** way keeps a model of the word as it appears on screen (its letters, marks and tone) and updates the model in place as each key arrives. A mark key finds its vowel and changes it; a tone key finds the right vowel and adds the tone; Backspace removes the last letter. Each update is cheap, and the model is always "the current word". The weakness is that the model is a summary of history. If the summary ever goes wrong (an edge case in an update rule, an event that never reached the application, an application that changed the text on its own), the model and the screen disagree, and every later update is applied to a word that is not there.

The **rebuild** way keeps only the history itself: the keys of the current word. After every key it throws away the previous result and computes the word again from all the keys. Nothing is carried from one result to the next except the key list. If the rules are right for every key sequence, the result is right for every key sequence, and there is no summary to go stale.

Dokey uses the rebuild way. The cost is that each key re-parses the whole word, which is at most a few dozen keys. Chapter 14 measures that cost; it is under a microsecond per key on a 2014 processor. The benefit is that the engine's behaviour is a function in the mathematical sense, and that makes it testable: for any key sequence there is exactly one expected result, and a test can state it.

4.2 The engine's state

The engine class is small. Its private state, from `include/dokey/engine.hpp`, is:

```
Options options_;
std::vector<Macro> macros_;
std::u16string raw_;      // keys of the current word, as the parser
reads them
std::u16string typed_;    // keys exactly as typed (for undo)
std::u16string display_;  // what is on screen for the current word
bool literal_ = false;    // after undo: the rest of the word is typed
as is
// The previous word, while only one space follows it (for reopening).
bool hasPrevious_ = false;
std::u16string prevRaw_, prevTyped_, prevDisplay_;
```

Three strings describe the current word.

- `raw_` is the input to the parser. It is usually the keys the user typed, but not always: Quick Telex replaces `cc` with `ch` here, and after Backspace the engine rebuilds `raw_` from the text on screen (Chapter 7).
- `typed_` is the keys exactly as typed. It is used by undo (↻) and by text shortcuts.
- `display_` is the engine's belief about what is on screen for the current word. It is the only thing the next edit is computed against.

The remaining fields serve the editing features of Chapter 7: `literal_` is set after an undo, and the `prev` fields remember the previous word while exactly one space follows it.

4.3 The path of a key

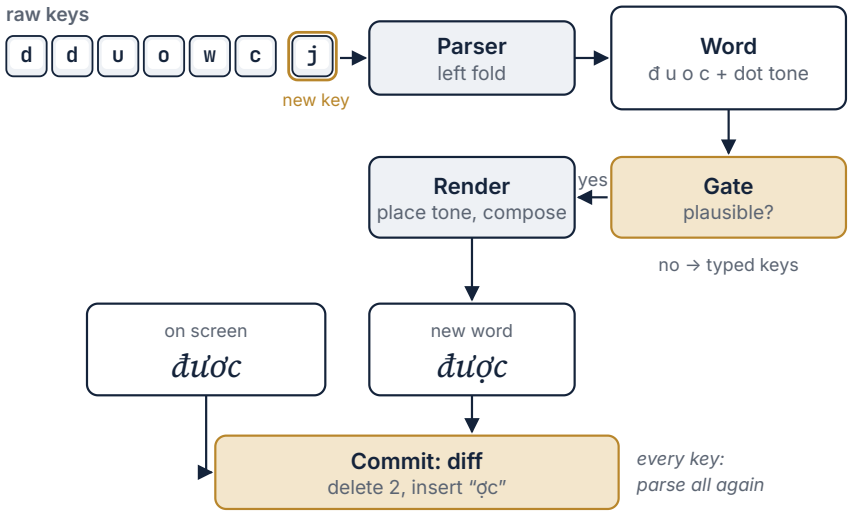


Figure 4.1 The path of one key through the engine. The raw keys are parsed from the start, the result is rendered, and the new word is compared with the word on screen to produce one edit.

`Engine::key(char32_t ch)` first decides whether the character belongs to a word. In Telex, word keys are the letters `a` to `z` and `A` to `Z`. In VNI the digits `0` to `9` are word keys too. Everything else (space, punctuation, Enter, Tab, Telex digits, and any non-ASCII character) is a **word break**, handled separately below.

For a word key, the steps are:

1. If the word is in literal mode (after ↺), append the key to `typed_` and `display_` and return *not handled*. The key goes through as typed.
2. If the word is empty and the key is a digit, return *not handled*. A number is not a word; this only arises in VNI.
3. Append the key to `typed_`.

4. If Quick Telex applies (Chapter 3), append the expanded pair to `raw_`, render and commit.
5. Append the key to `raw_`. If `raw_` is now longer than 32 keys, stop transforming: append the key to `display_` and return *not handled*. Long tokens such as URLs, hashes and identifiers pass through untouched.
6. Otherwise, render `raw_` and commit the result.

The two functions at the heart of this, `render` and `commit`, are the subject of the next two sections.

4.4 Rendering

`render(keys, wordEnded)` turns a key sequence into the text that should be on screen. It is a `const` member function: it reads the options and nothing else.

```
std::u16string Engine::render(const std::u16string& keys, bool
wordEnded) const {
    Parser parser(options_);
    Word w = parser.parse(keys);
    completeUo(w.letters);
    const bool show = !options_.checkSpelling || plausible(w.letters,
w.tone, false);
    if (!show) return w.typed;  // not Vietnamese: show what was typed
    // ... end-of-word corrections (Chapter 5) ...
    const size_t tonePos = w.tone == Tone::None ? w.letters.size()
                         : tonePosition(w.letters,
options_.modernToneStyle);
    std::u16string out;
    for (size_t i = 0; i < w.letters.size(); i++) {
        const Letter& l = w.letters[i];
        // ... a VNI digit typed literally is copied as is ...
        const bool toned = i == tonePos;
        if (options_.encoding == Encoding::UnicodeCombining && toned) {
            out.push_back(composeLetter(l.base, l.mark, Tone::None,
l.upper));
            out.push_back(combiningTone(w.tone));
        } else {
            out.push_back(composeLetter(l.base, l.mark, toned ? w.tone :
Tone::None, l.upper));
        }
    }
    return out;
}
```

In order, rendering:

1. parses the keys into a `Word` (Chapter 3);
2. completes *ư* + *o* to *ươ* before a final consonant (`uwon` → *ươn*);
3. checks plausibility, and if the word cannot be Vietnamese, returns the keys as typed (Chapter 5);
4. at the end of a word, applies two corrections: *ươ* at the very end becomes *uơ*, and a word that is a prefix but not a complete syllable is shown as typed (Chapter 5);
5. finds the vowel that carries the tone (Chapter 6);
6. composes each letter into its Unicode form.

Everything that can make one key sequence look different from another is in this function or in what it calls. There is no other path from keys to text.

4.5 Committing: the minimal edit

`commit(next, typed)` compares the new word with `display_` and produces the edit that turns one into the other:

```
Edit Engine::commit(std::u16string next, char32_t typed) {
    Edit edit;
    size_t common = 0;
    while (common < display_.size() && common < next.size()
           && display_[common] == next[common]) common++;
    edit.backspaces = static_cast<int>(display_.size() - common);
    edit.insert = next.substr(common);
    edit.handled = !(edit.backspaces == 0 && edit.insert.size() == 1
                     && edit.insert[0] == typed);
    display_ = std::move(next);
    return edit;
}
```

The edit keeps the longest common prefix of the old and new words, deletes the rest of the old word, and inserts the rest of the new one. Because all changes happen at the end of the word (the caret is at the end), the common prefix is the right thing to keep: any edit must at least delete back to the first character that changes.

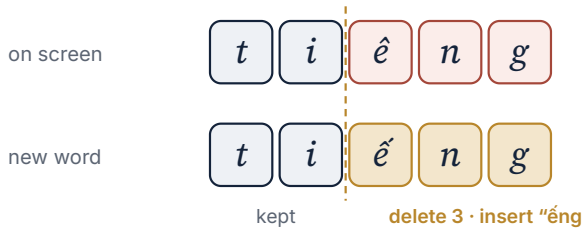


Figure 4.2 Committing *tiêng* → *tiếng*. The common prefix *ti* is kept; three characters are deleted and *ếng* is inserted.

The last line before the assignment is what makes most keys free. If the edit is "delete nothing, insert exactly the key that was typed", then the application would produce the same result on its own. The engine reports `handled = false`, and the platform lets the original key event through without sending anything. In the regression corpus of Chapter 12, 88.1 per cent of keys end this way (Chapter 14). Only the remaining 11.9 per cent cause synthetic events, and those delete 1.39 characters on average.

The edit is also minimal in a second sense: it never touches anything outside the current word. The engine has no knowledge of the text before the word, so it cannot damage it.

4.6 A full trace

The table below shows every key of `dduowcj` followed by a space, as the engine handles it. "Handled" says whether the platform must send an edit; "Delete" and "Insert" are the edit; "Screen" is the text that results. The values were printed by the trace program in Appendix F.

Key	Handled	Delete	Insert	Screen
`d`	no	0		*d*
`d`	yes	1	*đ*	*đ*
`u`	no	0		*đu*
`o`	no	0		*đuo*
`w`	yes	2	*ươ*	*đươ*

Key	Handled	Delete	Insert	Screen
`c`	no	0		*đươc*
`j`	yes	2	*ợc*	*được*
`␣`	no	0		*được ␣*

Seven letter keys produced three edits. The largest deleted two characters. The space at the end changed nothing and went through.

A second trace shows the gate at work. `windows` is not Vietnamese, but its first key is a Telex mark key:

Key	Handled	Delete	Insert	Screen
`w`	yes	0	*ư*	*ư*
`i`	no	0		*ưi*
`n`	yes	2	*win*	*win*
`d o w s`	no	0		*windows*

The lone `w` becomes *ư*, because a word can start with *ư*. The *i* keeps it plausible (*ưi* is a Vietnamese rhyme). The *n* does not: no syllable begins *ưin*. From that key on, render returns the keys as typed, and the commit replaces *ưi* with *win*. The later `w` and `s`, which would be mark keys in a Vietnamese word, simply type their letters, because the word they belong to is no longer Vietnamese. Without spelling checks the same keys produce *ứindow*: the second `w` is not used, because the vowel group is the *i* alone, and the final `s` puts an acute tone on the *ư*.

4.7 Word breaks

A word-break character ends the current word. Before the break goes through, the engine renders the word one last time with `wordEnded = true`, which enables the end-of-word corrections of Chapter 5, and checks for a text shortcut (Chapter 7). If the final rendering differs from `display_`, the edit includes the break character itself:

```
if (done ≠ display_) {
    edit = commit(done + static_cast<char16_t>(ch), ch);
```

```
    display_.pop_back();  // the break key is not part of the word
}
```

Including the break in the edit keeps the order right. If the engine sent the corrected word and then let the space through as a separate event, a fast typist's next letter could, in some applications, arrive between them. With the space inside the edit, one delivery carries both.

Three traces from Appendix F show the cases:

- `thuowr` ␣ → *thuở* ␣. The word was *thưở* while it was being typed; at the end, *ưở* with nothing after it becomes *ươ*. The edit deletes two characters and inserts *ở* followed by the space.
- `use` ␣ → *use* ␣. While typing, `s` was a plausible acute tone (*ú*), then *úe* was still a plausible prefix (it could still become *uế*). At the end of the word, *ue* is not a complete syllable, so the word reverts to the keys typed.
- `pass` ␣ → *pass* ␣. The double `s` undid the tone and left *pas* on screen; at the end, the engine decides this was an English word with a double letter and restores both *s* keys (Chapter 5).

After the break, the engine clears `raw_`, `typed_` and `display_`. If the break was a single space after a non-empty word, it first saves the word in the `prev` fields so that Backspace can reopen it.

4.8 Resetting

The engine cannot see the screen. It must be told when its belief about the current word stops being true. The platform calls `reset()` whenever the caret may have moved or the text may have changed without the engine's knowledge: a mouse click, an arrow or navigation key, a keyboard shortcut, a change of application or window, or a held key that auto-repeats. `reset()` clears the current word and the remembered previous word. The next key starts a new word from an empty `display_`.

Resetting too often is harmless: at worst a word typed after a reset cannot be corrected by marks typed before it. Resetting too rarely is not harmless, because the next edit would delete characters that

are not the current word. The platform layers therefore err on the side of resetting. Chapter 9 and Chapter 10 list exactly which events reset the engine on each platform.

4.9 Properties of the design

Rebuilding from keys gives the engine a set of properties that are easy to state and easy to test.

Determinism. For given options, the displayed word is a function of `raw_` alone. Two engines fed the same keys show the same word.

No drift. The only state that survives from one key to the next is the key list and the engine's record of what it last showed. There is no derived state (no "the tone is on letter 3") that a bug could leave inconsistent with the key list.

Order independence of marks. Because the whole word is re-parsed, a mark typed after the letters it modifies has the same effect as one typed straight after its letter, wherever the parser rules allow it. `tieengs`, `tienges` and `tiengse` all show *tiếng*.

Locality of edits. Every edit is confined to the current word, and its size is bounded by the word's length.

Silence by default. A key whose only effect is to insert itself produces no synthetic event.

The test suite checks these properties directly. The round trip of Chapter 12 types every syllable with every tone and compares the result; the pass-through test checks that plain letters are never rewritten; the doubled-key test checks the exact edit produced by each key of `ddoox`.

4.10 Limits

The engine transforms words of at most 32 raw keys. A Vietnamese syllable needs at most about ten keys even in the most generous Telex spelling, so the limit only affects long tokens, which are better left alone. Edits returned through the C API are limited to 256 UTF-16 units (`DOKEY_MAX_EDIT`), which matters only for text short-

cuts; an edit that would exceed it is dropped and the word is reset (Chapter 11).

The engine processes characters in the Basic Multilingual Plane. A word break outside it (an emoji, for example) ends the word but is never placed inside an edit. Non-ASCII characters typed directly, such as a Vietnamese letter from another keyboard layout, are word breaks and pass through unchanged.

THE ENGINE

5

Syllable Validation as a Gate

Chapter 3 described when a mark key *can* be applied: there must be a vowel to carry it, the key must not have been undone, and so on. This chapter describes when a mark key *may* be applied. Dokey's answer is a single rule, applied at two points: a mark is allowed only if the word can still become a Vietnamese syllable. The same rule, applied to the whole word when it is drawn, decides whether the word is shown as Vietnamese at all. Together these two applications form a gate between the keys and the screen.

5.1 Why a gate

Without a gate, an input method applies every mark it can. That is correct for Vietnamese text and wrong for almost everything else. In Telex the mark keys `s`, `f`, `r`, `x`, `j`, `w` and the doubled vowels are common in English. The word `windows` contains three of them.

There are two common responses. One is to give the user a quick way to switch the input method off, and leave the rest to them. The other is to keep a list of English words and leave those alone. The first puts the work on the user; the second covers only the words on the list and fails on names, new terms and identifiers.

Dokey's gate works from the other side. It does not need to know English at all. It knows what Vietnamese syllables can look like, and it refuses to produce anything else. Any word that cannot be Vietnamese is shown exactly as typed, whether it is English,

French, a product name or a variable name. The only words the gate cannot protect are those whose keys also spell a Vietnamese syllable, and those are a fixed, testable set.

5.2 The dictionary

The gate needs a fast answer to three questions about a string of lower-case letters with diacritics and without tone:

1. Is it the beginning (a prefix) of some syllable?
2. Could it become the beginning of some syllable if diacritics were added to its plain letters?
3. Is it a complete syllable?

Dokey answers them from a dictionary built once, on first use, from the tables of initials and rhymes in `src/vietnamese.cpp`. The builder combines every initial with every rhyme under the spelling constraints of Chapter 2 and adds each resulting syllable together with all of its prefixes:

```
void add(const std::u16string& syllable) {
    syllables.push_back(syllable);
    complete.insert(syllable);
    for (size_t n = 1; n ⩽ syllable.size(); n++) {
        std::u16string prefix = syllable.substr(0, n);
        if (marked.insert(prefix).second)
            byStripped[stripMarks(prefix)].push_back(prefix);
        stripped.insert(stripMarks(prefix));
    }
}
```

The result is four structures:

- `complete`: the set of complete syllables, 3,957 strings;
- `marked`: every prefix of every syllable, with diacritics, 4,390 strings;
- `stripped`: the same prefixes with diacritics removed;
- `byStripped`: a map from each stripped prefix to the marked prefixes that share it. The stripped prefix *duo*, for example, maps to *đuô* and *đươ*, among others.

The counts are from the measurement program in Appendix F. On my machine the dictionary was built in 11.3 and 3.9 milliseconds in two timed runs; it is built once per process, the first time the engine needs it, and held in a function-local static.

The three questions then become lookups:

```
bool isSyllablePrefix(const std::u16string& lowerMarked) {
    return dictionary().marked.count(lowerMarked) != 0;
}

bool isCompatibleSyllablePrefix(const std::u16string& lowerMarked) {
    const auto& groups = dictionary().byStripped;
    auto it = groups.find(stripMarks(lowerMarked));
    if (it == groups.end()) return false;
    for (const std::u16string& prefix : it->second) {
        bool ok = true;
        for (size_t i = 0; ok && i < prefix.size(); i++) {
            // A marked letter must match; a plain letter may still get
its mark.
            const char16_t c = lowerMarked[i];
            ok = c == prefix[i] || c < 0x80;
        }
        if (ok) return true;
    }
    return false;
}

bool isSyllable(const std::u16string& lowerMarked) {
    return dictionary().complete.count(lowerMarked) != 0;
}
```

A compatible prefix is one where every letter that already carries a diacritic matches, and every plain letter could still receive one. *đuo* is compatible: its *đ* matches and its *u* and *o* may still become *ư* and *ơ*. *đuă* is not: no syllable has *ă* after *đu*.

5.3 Plausibility

The engine combines the lookups with the tone rule of Chapter 2 in one function:

```
bool plausible(const std::vector<Letter>& w, Tone tone, bool
requireMarks) {
    if (w.empty()) return true;
    if (!toneAllowed(w, tone)) return false;
    std::u16string s = lowerMarked(w);
```

```
    if (isSyllablePrefix(s)) return true;
    if (requireMarks) return false;
    return isCompatibleSyllablePrefix(s);
}
```

A word is **plausible** if its tone is allowed (no grave, hook or tilde on a checked syllable) and its letters are a prefix of some syllable, either exactly or, when `requireMarks` is false, after adding diacritics to its plain letters.

The `requireMarks` flag distinguishes the two places where the gate is used.

The gate on mark keys

When the parser tries a diacritic (circumflex, horn, breve or stroke), it calls `plausible` with `requireMarks = true`. The trial word, with the new diacritic, must be an exact prefix of a syllable. A diacritic is a deliberate claim about a letter, and the engine accepts it only if that claim leads somewhere.

When the parser tries a tone, it calls `plausible` with `requireMarks = false`. The letters may still be waiting for their diacritics. In `dduocjw` the tone key `j` arrives when the word is *đuoc*; the exact string *đuoc* is not a prefix of any syllable, but it is compatible with *đươc*, and the tone is accepted. The later `w` then adds the horns and the word becomes *được*. Users type tones in many positions, and the gate should not punish one order over another.

If the gate refuses a mark, the parser treats the key as a plain letter. `hocf` does not become an impossible *hòc*: the grave tone is refused on a checked syllable, and the `f` is appended as a letter. The word *hocf* is then not plausible, and the next stage shows the keys as typed.

The gate on the whole word

When `render` draws the word, it calls `plausible` once more on the parsed result, with `requireMarks = false`. If the answer is no, render returns `w.typed`, the keys exactly as typed, instead of the parsed letters:

```
const bool show = !options_.checkSpelling || plausible(w.letters,
w.tone, false);
if (!show) return w.typed;  // not Vietnamese: show what was typed
```

This is the step that restores English words. As soon as the keys stop being a possible start of a Vietnamese syllable, the screen shows the keys. The trace of `windows` in Chapter 4 shows the moment: at the *n*, the word *ưin* is no longer plausible, and the screen changes from *ưi* to *win*.

Because the check happens on every key, it can also go the other way. A word that was not plausible can become plausible again as more keys arrive, if the rules allow it, and the screen follows. In practice this is rare, because a string that is not a prefix of any syllable rarely becomes one by adding letters.

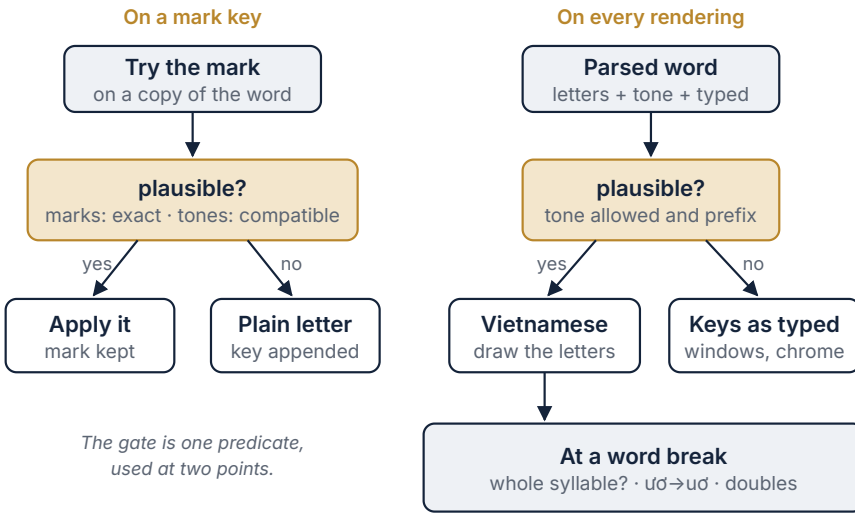


Figure 5.1 The two gates. A mark is applied only if the trial word is still plausible; the whole word is shown as Vietnamese only if it is plausible, and at the end of the word only if it is a complete syllable.

5.4 Corrections at the end of a word

When a word break arrives, the engine renders the word with `wordEnded = true`, which enables three further rules.

A finished word must be a whole syllable

While a word is being typed, a prefix is enough: *tiê* is fine because *tiêng* may follow. When the word ends, a prefix is no longer enough:

```
if (wordEnded && options_.checkSpelling && hasVowel(w.letters)
    && !isSyllable(lowerMarked(w.letters)))
    return w.typed;
```

This is why `use ␣` gives *use* and not *úe*: *ue* is a prefix (of *uê*, *uế*…) but not a syllable. Words with no vowel, such as a lone *đ*, are kept: they are not syllables, but a user who typed `dd` and a space presumably wanted *đ*.

ươ at the end becomes uơ

The diphthong *ươ* is written *uơ* when nothing follows it (Chapter 2). During typing the parser cannot know whether a final consonant is coming, so it shows *ươ*. When the word ends with *ươ* as its last two letters, render removes the horn from *u*. `thuowr ␣` therefore ends as *thuở*, and `huow ␣` as *huơ*. A stand-alone *ư* typed with a lone `w` is left alone.

English words with a double letter

The third rule is the most specific, and it exists because Telex itself creates the problem. In Telex, repeating a tone key undoes the tone and prints the letter once: `ass` → *as*. That is the convention, and a Vietnamese user relies on it. But English is full of double *s*, *f* and *r*: *pass*, *class*, *message*, *different*, *error*, *password*. Typed in Telex, `pass` gives *pas*, because the second *s* undid the acute tone that the first *s* added.

Dokey keeps the Telex convention for short words and restores the double letter for longer English words, using a rule that depends only on the keys. When a word ends, the engine looks for this situation:

- the word has at least four keys, and Quick Telex did not alter it;
- a tone key was cancelled by repeating it somewhere in the word;

- the word, as rendered, is exactly the keys as typed minus the cancelled key (in other words, the result is not Vietnamese and lost a letter to the undo).

If all three hold, the engine finds the position of the cancelling key and looks at the next key:

- If the doubled letter ends the word, or is followed by a vowel, *w* or *y*, the double is treated as an English double letter and every key is restored: *pass*, *message*, *different*, *password*.
- If the doubled letter is followed by a consonant, the double is treated as the Telex way to type the letter once: `tesst` → *test*, `marrketing` → *marketing*. To keep a double letter before a consonant, the user types it three times: `offfline` → *offline*, `passsport` → *passport*.

Words of fewer than four keys keep the Telex convention: `ass` → *as*.

The rule was refined with data. The project's test notes record a scan of 12,518 words containing *ss*, *ff* or *rr* from the macOS system dictionary; the scan found that doubles before a consonant (*offline*, *passport*) are the cases that need the third key. The regression corpus of Chapter 12 locks in the behaviour for the words listed in its data files.

5.5 What the gate cannot do

The gate protects every word whose keys cannot spell Vietnamese. It cannot protect a word whose keys *do* spell Vietnamese, because in that case the keys are ambiguous and the engine has no way to tell what the user meant. `test` is how a Vietnamese typist writes *tét*; `this` is how they write *thí*.

The regression corpus records these cases explicitly. Its English file has 1,171 entries (1,138 distinct words, since a few appear twice). Of those entries, 74 produce a Vietnamese syllable; they are 73 distinct words, because *list* appears twice. All are listed in Appendix E; a few examples:

Keys	Result	Keys	Result
`this`	*thí*	`did`	*đi*

Keys	Result	Keys	Result
`there`	*thể*	`data`	*dât*
`test`	*tét*	`room`	*rôm*
`was`	*ủa*	`wrong`	*ưởng*
`best`	*bét*	`six`	*sĩ*

The list is part of the test: the corpus test fails if any of these results changes, in either direction. A user who types English often and finds these words in their way has two remedies, both outside the engine: switch to English mode for a moment, or add the word to the list of excluded applications if it occurs in one program. A text shortcut also works for frequent cases.

The remaining 1,097 entries are kept exactly as typed. The test suite checks all 1,171 on every build (Chapter 12).

5.6 Spelling checks off

The gate can be turned off. With `checkSpelling` false, `ok()` in the parser always returns true and render never falls back to the typed keys. Every mark is applied wherever the parser rules allow, and the end-of-word corrections that depend on the dictionary (complete syllable, double letters) are skipped. The *ươ* to *ươ* correction still applies.

This mode exists for users who type non-standard Vietnamese, such as old spellings, dialect forms or deliberate misspellings, and who do not type English with the input method on. It also serves as a control in measurement: in the performance measurements of Chapter 14, turning the gate off made the engine about a third faster per key on the same corpus.

5.7 Summary

The gate is one predicate, *plausible*, built on a dictionary of syllable prefixes generated from the spelling rules. Applied to each mark key, it refuses diacritics that lead nowhere and tones that a checked syllable cannot take. Applied to the whole word, it shows

the keys as typed as soon as they stop being Vietnamese. At the end of a word, a stricter version requires a complete syllable, fixes *ươ* to *uơ*, and restores English double letters. None of this needs a list of English words. The only words it cannot protect are those that are also Vietnamese, and the test suite names every one of them that occurs in its corpus.

THE ENGINE

6

Tone Placement and Output Encoding

The parser of Chapter 3 decides which letters a word has and which tone it carries. It does not decide where the tone is drawn. That decision is made at the last moment, when the word is rendered into text, by a single function that looks only at the finished letters. This chapter describes that function, the tables that turn a letter, a diacritic and a tone into a Unicode character, and the two output encodings.

6.1 Why placement is computed, not stored

In an incremental input method the tone is usually attached to a letter when the tone key arrives, and moved when later keys change the vowels. Every rule that changes vowels must then also remember to move the tone. That is a well-known source of bugs: a word such as *hoà* typed as `hoaf`, then extended with `n`, must become *hoàn* in either style (the tone moves from *a* to *a* in the modern style, but from *o* to *a* in the traditional style), and Backspace must move it back.

In Dokey the tone is never attached to a letter in the engine's state. The `Word` produced by the parser holds one `Tone` value for the whole word. Every time the word is rendered, `tonePosition` computes the index of the vowel that carries it, from the letters as they are now.

When a later key changes the letters, the next rendering simply computes a new position. There is nothing to move.

The trace of `tiengse` in Chapter 3 shows this. After `s` the word is *tieng* with an acute tone; the rules put it on the last vowel, giving *tiéng*. After the final `e` the word is *tiêng*; the same rules now find a vowel with a diacritic and put the tone on *ê*, giving *tiếng*. No code moved the tone. It was placed twice, by the same function, on two different words.

6.2 The placement rules

`SPEC.md` states the rules over the vowel group, which excludes the *u* of *qu* and the *i* of *gi* when another vowel follows (Chapter 3). The implementation follows the specification line by line:

```
size_t tonePosition(const std::vector<Letter>& w, bool modern) {
    size_t b, e;
    vowelRun(w, b, e);
    if (e == b) return w.size();
    // 1. A vowel with a diacritic takes the tone (the last one: ươ →
ơ).
    for (size_t i = e; i-- > b;) {
        if (w[i].mark ≠ Mark::None) return i;
    }
    const size_t len = e - b;
    // 2. Closed syllable: last vowel of the nucleus.
    if (e < w.size()) return e - 1;
    // 3./4./5. Open syllable.
    if (len ⩾ 3) return b + 1;
    if (len == 2) {
        char a = w[b].base, c = w[b + 1].base;
        bool modernPair = (a == 'o' && (c == 'a' || c == 'e')) || (a ==
'u' && c == 'y');
        return (modern && modernPair) ? b + 1 : b;
    }
    return b;
}
```

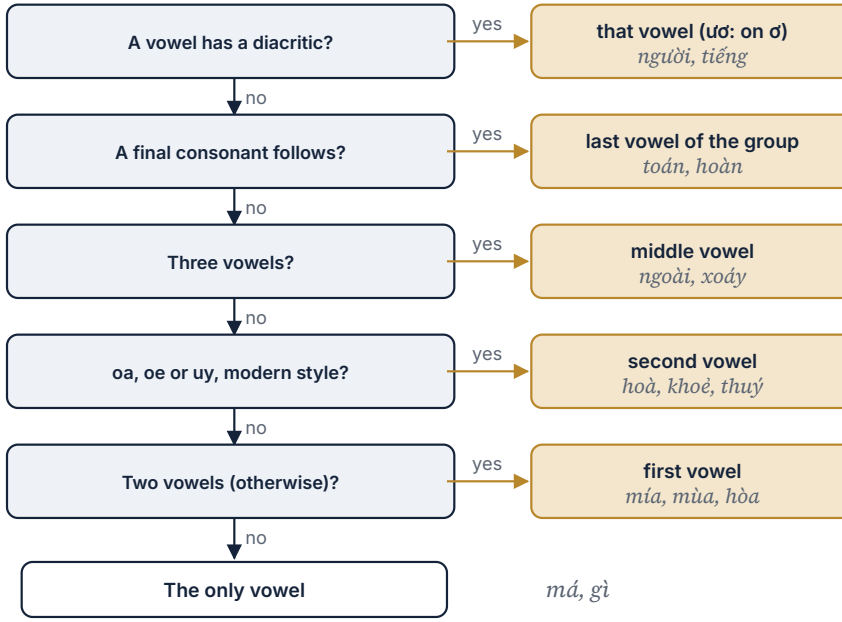


Figure 6.1 Tone placement as a decision tree. The first rule that applies decides the vowel.

Each branch corresponds to a rule of Chapter 2.

Rule 1: a vowel with a diacritic. The loop runs from the end of the vowel group backwards and returns the first vowel that has a diacritic. Running backwards matters for *ươ*, where both vowels have a horn: the later one, *ơ*, is found first, which gives *người* and *được*. In words with a single marked vowel (*tiếng*, *Việt*, *cười*) the direction makes no difference.

Rule 2: a closed syllable. If any letter follows the vowel group, the syllable has a final, and the tone goes on the last vowel of the group: *toán*, *hoàn*, *huýt*, *khoắt* (where rule 1 would already apply).

Rule 3: three vowels. In an open syllable with a three-vowel group, the middle vowel takes the tone: *ngoài*, *xoáy*, *khuỷu*.

Rule 4: two vowels. In an open syllable with a two-vowel group, the first vowel takes the tone (*mía*, *mùa*, *táy*), except for *oa*, *oe* and *uy*, where the style decides: the second vowel in the modern style (*hoà*, *khoẻ*, *thuý*), the first in the traditional style (*hòa*, *khỏe*, *thúy*).

Rule 5: one vowel. The only vowel takes it.

If the word has no vowel group at all, the function returns the length of the word, which is not a valid index, and no letter receives the tone. The parser never produces a toned word without a vowel (a tone key needs a vowel to be accepted), so this is only a safety value.

The special initials

Two details of `vowelRun` affect placement. The *u* of *qu* is skipped whenever a vowel follows it, so *quá* puts the tone on *a* and *quý* on *y*. The *i* of *gi* is skipped only when it carries no diacritic and a vowel follows, so *giá* puts the tone on *a*, while in *gì* the *i* is the whole vowel group and takes the tone itself. The spelling *gi* + *ì* = *gì* was established by the dictionary builder (Chapter 2); here the same fact shows up as a vowel group of one letter.

6.3 Testing placement independently

Tone placement is easy to get almost right. The round-trip test of Chapter 12 therefore checks it against a second implementation, written separately in the test file from the specification and not from the engine's code. The test's version, `expectedWithTone`, works on Unicode strings rather than on `Letter` values:

```
static std::u16string expectedWithTone(const std::u16string& w, int
tone, bool modern) {
    if (!tone) return w;
    size_t start = 0;
    if (w.size() > 2 && w.compare(0, 2, u"qu") == 0 && isV(w[2])) start
= 2;
    else if (w.size() > 2 && w.compare(0, 2, u"gi") == 0 && isV(w[2]))
start = 2;
    else while (start < w.size() && !isV(w[start])) start++;
    size_t end = start;
    while (end < w.size() && isV(w[end])) end++;
    std::u16string nucleus = w.substr(start, end - start);
    size_t pos;
    size_t marked = nucleus.find_last_of(u"ăâêôơư");
    if (marked != std::u16string::npos) pos = marked;
    else if (end < w.size()) pos = nucleus.size() - 1;
    else if (nucleus.size() == 3) pos = 1;
    else if (nucleus.size() == 2)
        pos = (modern && (nucleus == u"oa" || nucleus == u"oe" ||
```

```
nucleus == u"uy")) ? 1 : 0;
    else pos = 0;
    // ... compose the toned letter at start + pos ...
}
```

The two implementations agree for all 3,957 syllables and every tone each can carry, in the modern style. Hand-written cases cover the traditional style (`hoaf` → *hòa*, `khoer` → *khỏe*, `thuys` → *thúy*, `hoanf` → *hoàn*).

6.4 From letters to code points

Once the tone position is known, each letter is turned into one UTF-16 code unit by `composeLetter`:

```
char16_t composeLetter(char base, Mark mark, Tone tone, bool upper) {
    if (base == 'd' && mark == Mark::Stroke) return upper ? u'Đ' : u'đ';
    int row = vowelRow(base, mark);
    if (row >= 0) return (upper ? kUpper : kLower)[row][static_cast<int>
(tone)];
    return static_cast<char16_t>(upper ? base - 'a' + 'A' : base);
}
```

The tables `kLower` and `kUpper` have one row per vowel letter (*a, ă, â, e, ê, i, o, ô, ơ, u, ư, y*) and one column per tone (none, acute, grave, hook, tilde, dot). Their 144 entries are the precomposed code points of the Unicode standard; Appendix B lists them all. `vowelRow` finds the row from the base letter and the diacritic. Consonants other than *đ* are returned as themselves, with case.

The reverse function, `decomposeLetter`, searches the same tables to split a character back into base, diacritic, tone and case. The engine uses it in three places: to strip diacritics when building the dictionary, to upper-case text shortcut expansions, and to turn the word on screen back into keys after Backspace (Chapter 7).

Because both directions use the same two tables, they cannot disagree. A letter composed by the engine always decomposes to the values it was composed from.

6.5 The combining encoding

With `Encoding::UnicodeCombining`, the toned vowel is written as two code units: the vowel with its diacritic but without tone, followed by the combining tone mark:

Tone	Combining character
acute	U+0301 COMBINING ACUTE ACCENT
grave	U+0300 COMBINING GRAVE ACCENT
hook	U+0309 COMBINING HOOK ABOVE
tilde	U+0303 COMBINING TILDE
dot	U+0323 COMBINING DOT BELOW

Letters with a diacritic stay precomposed in this encoding: *ê* is U+00EA, not *e* + U+0302. The result is neither NFC nor NFD but a mixed form that some older Vietnamese software expects. The test suite checks it with one case: `vieetj ␣` must produce the bytes of *v, i, ê, U+0323, t* followed by a space.

The combining encoding has two consequences elsewhere in the system.

First, the word on screen is longer than the number of letters, so an edit's `backspaces` count includes the combining marks. The engine's `display_` holds the same code units as the screen, so its edits are still exact, as long as the application deletes one code unit per Backspace. Applications that delete a whole grapheme cluster per Backspace (a base letter with its combining marks) would delete too much; this is one reason the combining encoding is not the default.

Second, the engine cannot resume a word after Backspace in this encoding. `backspaceImpl` resets the word when the encoding is not precomposed Unicode, because the text on screen no longer maps one-to-one onto letters (Chapter 7). On macOS the platform layer also avoids selection-based delivery with the combining encoding, because the Left Arrow key moves by whole characters and would select a different number of code units than the edit expects (Chapter 9).

6.6 Upper case and tones

Case is a property of each letter. `composeLetter` simply selects the `kUpper` table for capital letters. Tones and diacritics are the same on capitals as on small letters, so *TIẾNG*, *ĐƯỜNG* and *Ở* need no special handling. Text shortcuts upper-case their expansion with `decomposeLetter` and `composeLetter` in turn, so that a shortcut typed in capitals expands with the tones kept: `VN.` with the shortcut `vn = Việt Nam` gives *VIỆT NAM.* (Chapter 7).

6.7 Summary

Tone placement in Dokey is a pure function of the finished letters, computed every time the word is drawn. It implements the five written rules in fewer than twenty lines, with the modern and traditional styles differing only for *oa*, *oe* and *uy*. Letters are turned into code points through two 12 × 6 tables that serve both composition and decomposition. The precomposed encoding keeps one code unit per letter, which makes every later edit simple to count; the combining encoding is available for software that needs it, at the cost of some editing features.

THE ENGINE

7

Editing: Backspace, Reopening, Undo and Shortcuts

Typing is never only forward. Users delete letters, go back to fix a mark they forgot, change their minds about a whole word, and type the same phrases many times a day. Version 0.4 of Dokey added a set of editing features that address these habits. Each is implemented in the engine, behind the same C API, so that the macOS and Windows applications behave identically. This chapter describes them and shows how they fit the pure-rebuild model of Chapter 4.

7.1 Backspace inside a word

Backspace is the one key that the rebuild model cannot handle by appending to the key list. Deleting the last *letter* on screen does not correspond to deleting the last *key*: the last key of `tieengs` is `s`, which added a tone, and the last letter of *tiếng* is *g*. Removing `s` from the keys would remove the tone and keep the *g*; the user expects the opposite.

Dokey's answer is to stop treating the old keys as the truth after a Backspace. The engine takes the word as it is now on screen, minus its last character, and writes a fresh key sequence that would produce exactly that text: the canonical keys of Chapter 3. It then continues from those keys as if the user had typed them.

```
std::u16string remaining = display_.substr(0, display_.size() - 1);
std::u16string keys;
if (remaining.empty() || !encodeKeys(remaining, options_.method, keys))
{
    reset();  // cannot resume this word: the next key starts a new one
    return Edit();
}
std::u16string next = render(keys, false);
if (next == remaining) {
    raw_ = keys;
    typed_ = keys;
    display_ = remaining;
    return Edit();
}
```

If rendering the canonical keys gives back exactly the remaining text, the engine adopts them and lets the Backspace go through as a normal key. The next key continues the word: `tieengs` ⌫ ⌫ `n` ␣ gives *tiến* ␣, because after two Backspaces the screen shows *tiế*, its canonical keys are `tiees`, and `n` extends them to `tieesn`, which renders as *tiến*.

If the canonical keys cannot be found (the word contains a character that is not a Vietnamese letter, or the encoding is the combining one), the engine resets. The Backspace still goes through; the engine simply forgets the word, and the next key starts a new one. This is the safe failure: the engine never edits text it does not understand.

Backspace that moves the tone

Sometimes the remaining letters render differently from what is on screen. The common case is a tone that should move. In the traditional style, *hoàn* has its tone on *a* because a final follows; delete the *n* and the word *hoà* should become *hòa*, with the tone on *o*. A plain Backspace would leave *hoà*, which is wrong in that style.

For this case the engine offers a second Backspace entry point, `backspaceEdit()`. When the letters are the same but the rendering differs, it returns an edit that the platform applies *instead of* the Backspace:

```
// Same letters, different tone position ("hoàn" → "hòa" in the old
style):
```

```
// the platform applies the corrected word instead of a plain Backspace.
if (allowEdit && next.size() == remaining.size()) {
    Edit edit = commit(next, 0);
    raw_ = keys;
    typed_ = keys;
    return edit;
}
```

The trace from Appendix F, in the traditional style:

Key	Handled	Delete	Insert	Screen
`h o a n`	no	0		*hoan*
`f`	yes	2	*àn*	*hoàn*
⌫	yes	3	*òa*	*hòa*

The edit deletes three characters (*oàn*) and inserts *òa*, which includes the deletion of *n* that the user asked for. In the modern style the same Backspace is a plain one: *hoà* is already correct.

The older entry point, `backspace()`, still exists for platforms that cannot swallow a Backspace and replace it with an edit. It updates the engine's view in the same way but never returns an edit; if the tone would move, it resets instead.

7.2 Reopening the previous word

A common slip is to finish a word, press Space, and only then notice a missing tone. In most input methods the word is now gone: the engine has started a new word, and the user must delete the whole word and type it again.

Dokey keeps the previous word for exactly one more key. When a word ends with a single space, the engine saves its raw keys, typed keys and display:

```
hasPrevious_ = ch == U' ' && hadWord && !expanded && !literal_ &&
raw_.size() <= kMaxWordKeys;
if (hasPrevious_) {
    prevRaw_ = raw_;
    prevTyped_ = typed_;
    prevDisplay_ = display_;
}
```

If the very next key is Backspace, which deletes that space, the engine restores the saved word as the current word. The user can then type the missing mark:

Keys	Screen
`x i n`	*xin*
␣	*xin* ␣
⌫	*xin* (the word is reopened)
`f`	*xìn*

Any other key clears the saved word. So do a second space, a mouse click, an arrow key or a change of application, because each of those means the caret is no longer right after the word. The rule is deliberately narrow: one word, one space, one Backspace. A broader rule (reopen any word the caret lands in) would require the engine to know the text around the caret, which it does not.

7.3 Undo with Escape

Sometimes the engine's guess is wrong in the other direction: the user meant the keys, not the Vietnamese word. A user typing a code identifier such as `tieengs` (perhaps a test string) sees *tiếng* and wants the keys back.

`Engine::undo()` replaces the current word with the keys exactly as typed and puts the rest of the word in literal mode:

```
Edit Engine::undo() {
    if (display_.empty() || literal_ || typed_ == display_) return
Edit();
    Edit edit = commit(typed_, 0);
    literal_ = true;
    hasPrevious_ = false;
    return edit;
}
```

If the word has no marks (the screen already shows the keys), there is nothing to undo, the function returns *not handled*, and the platform lets Escape go through to the application, where it may close a dialog or leave a mode as usual. Both applications bind undo to

Escape, and both leave the option off by default, because many applications already use Escape for something.

In literal mode, every further letter of the word is appended as typed, with no parsing at all: `vieejt` ⌫ `s` ␣ gives *vieejts* ␣. The next word starts normally: `vieejt` ⌫ ␣ `vieejt` ␣ gives *vieejt việt* ␣.

7.4 Quick Telex

Quick Telex, described in Chapter 3, is an option that turns a doubled consonant at the start of a word into a two-letter initial: `cc` → *ch*, `gg` → *gi*, `kk` → *kh*, `nn` → *ng*, `pp` → *ph*, `qq` → *qu*, `tt` → *th*. It lives in `Engine::key`, before the parser:

```
if (options_.quickTelex && options_.method == InputMethod::Telex
    && raw_.size() == 1 && typed_.size() == 2) {
    static const char16_t from[] = u"cgknpqt";
    static const char16_t to[]   = u"hihghuh";
    // ... if the second key equals the first and is in `from`,
    //     append the matching letter of `to` to raw_ instead ...
}
```

Two details are worth noting. The expansion changes `raw_` but not `typed_`, so undo and the double-letter rule of Chapter 5 still see what the user pressed. And the expansion applies only to the second key of the word, so it cannot fire inside English words such as `app`, `accept` or `Google`.

7.5 Text shortcuts

Text shortcuts (macros) replace an abbreviation with a longer text when a word break follows it: `btw` ␣ → *by the way* ␣, `vn` ␣ → *Việt Nam* ␣. The platform passes the shortcut list to the engine as UTF-8 lines of the form `shortcut = text` or `shortcut: text`, with `#` for comments:

```
dokey_set_macros(engine,
    "# my shortcuts\n"
    "btw = by the way\n"
    "vn: Việt Nam\n");
```

The C API parses the lines, trims spaces and tabs around both sides, and skips empty lines and lines without a separator. The en-

gine stores shortcuts in lower case and ignores any shortcut longer than 32 keys.

When a word break arrives, `expandMacro` compares the keys *as typed* (not the rendered word) with each shortcut, ignoring case. If one matches, the engine commits the expansion followed by the break character. Case follows the abbreviation:

Typed	Result
`btw ␣`	*by the way ␣*
`Btw,`	*By the way,*
`BTW ␣`	*BY THE WAY ␣*
`VN.`	*VIỆT NAM.*

A shortcut typed in all capitals (and longer than one letter) expands in capitals; one that starts with a capital expands with its first letter capitalised. Upper-casing uses the Vietnamese letter tables of Chapter 6, so tones and diacritics are kept.

Matching on the typed keys has a useful consequence: a shortcut matches only when the user typed exactly its keys. `kos ␣` does not trigger the shortcut `ko`, and a shortcut whose keys are also a Vietnamese word in Telex expands instead of the word. It also means the shortcut is chosen before the spelling gate's end-of-word rules run, so an abbreviation such as `vn`, which is not a syllable, expands rather than being restored as typed.

Through the C API, an expansion is limited to 256 UTF-16 units including the break character (`DOKEY_MAX_EDIT`, Chapter 11). Both applications keep the engine's shortcuts off while the user edits the shortcut list itself, so that typing `btw = by the way` into the editor does not expand the left-hand side.

7.6 How the features interact

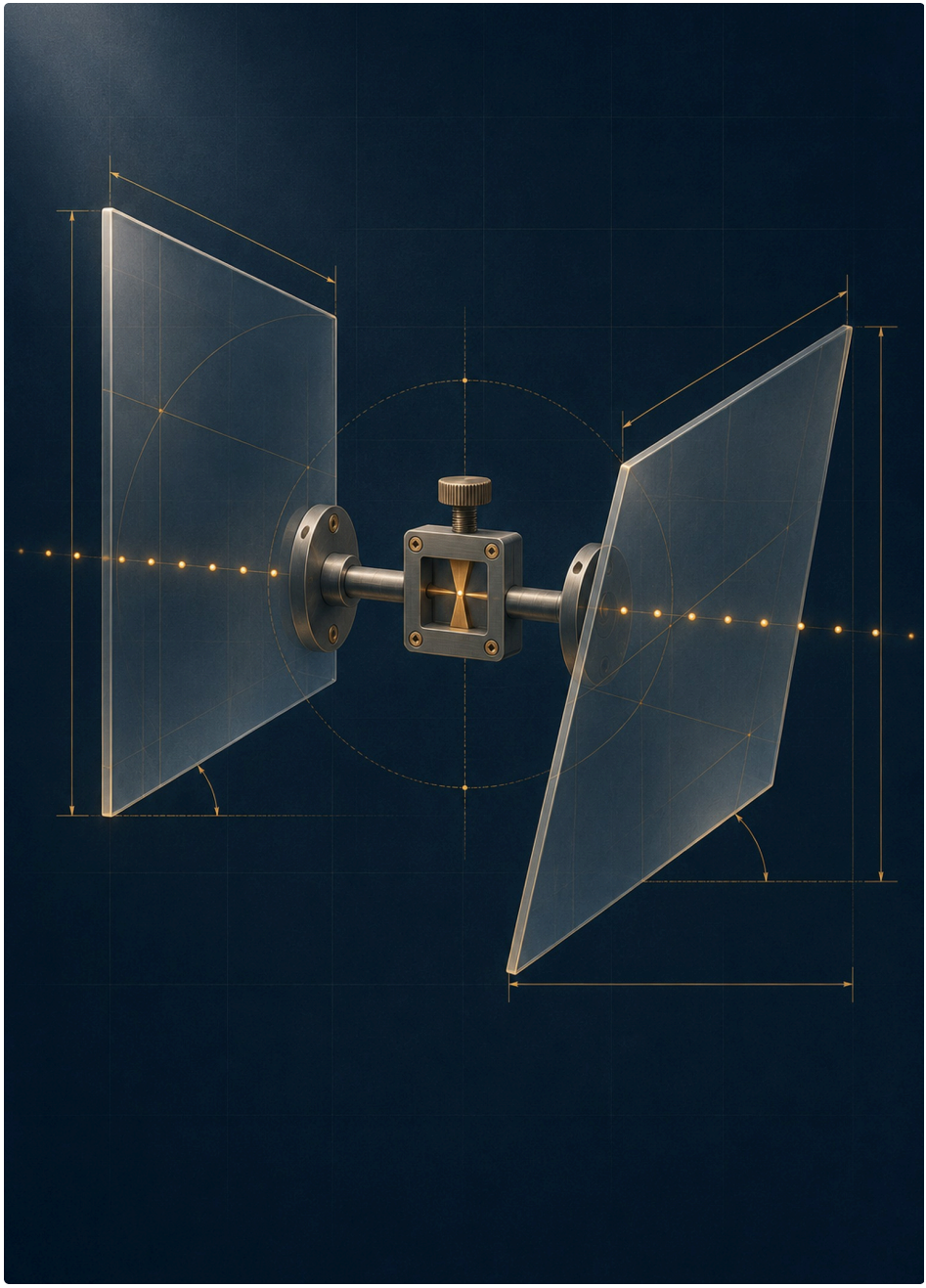
Because all the features act on the same three strings (`raw_`, `typed_`, `display_`) and all produce edits through `commit`, their interactions are few and explicit:

- After undo, Backspace works on the literal word one character at a time, and the word is never reopened.
- A word that ended in a shortcut expansion is not saved for reopening; Backspace after it is an ordinary Backspace.
- Quick Telex changes `raw_` only, so undo shows the doubled consonant the user typed.
- `reset()` clears everything, including the saved previous word and literal mode.

The test suite's `editing()` group checks each of these features through the same edit-returning entry points that the applications use (Chapter 12). The cases quoted in this chapter are taken from it.

7.7 Summary

The editing features show how the rebuild model extends beyond forward typing. Backspace converts the screen back into canonical keys and continues from there, which lets a tone move to its correct place. Reopening keeps one word for one Backspace. Undo replaces the word with its keys. Quick Telex and shortcuts transform keys before or instead of the parser. Each feature is a few dozen lines in the engine, and each returns the same kind of minimal edit as an ordinary key.



PART III

The Platforms

Chapters 8–11

8

Minimal Edits and Per-Application Delivery

The engine ends its work with an edit: delete *n* characters, insert this text. Everything up to that point is deterministic and testable in isolation. Everything after it depends on the operating system, the application that has keyboard focus, the text control inside that application, the speed of the machine and the speed of the user. This chapter describes the edit as a contract between the engine and the platform, the ways in which delivering it goes wrong, and the strategies Dokey uses to deliver it intact. Chapters 9 and 10 then describe how each platform implements those strategies.

8.1 The edit as a contract

From the platform's point of view, the engine offers a narrow promise:

- If `dokey_key` returns 0, the original key should go through unchanged. The engine has already accounted for it.
- If it returns 1, the original key must be swallowed, and the platform must delete `backspaces` characters immediately before the caret and then insert `text`. After that, the application's text will match the engine's `display_`.

The promise holds only if the engine's belief about the current word is true when the edit is applied. That is the platform's respon-

sibility. It must call `dokey_reset` whenever the caret moves or the text changes behind the engine's back, and it must make sure that each edit is applied completely and in order before the user's next key is processed.

The engine helps by keeping edits small. Because it diffs the old and new words (Chapter 4), a typical edit replaces one or two characters. In the regression corpus of Chapter 12, 11.9 per cent of keys produce an edit, and those edits delete 1.39 characters on average. In the Vietnamese sentences alone, 22.7 per cent of keys produce an edit, deleting 1.57 characters on average (Chapter 14). Fewer and shorter edits mean fewer synthetic events, and fewer synthetic events mean fewer chances for something to go wrong.

8.2 Four ways delivery fails

The Dokey repository records the failures that its manual tests found. Each is a real case, with the keys typed and the text that appeared. Each has a different cause, and each led to a change in delivery.

Reordering in multi-process applications

Chromium-based browsers and Electron applications handle keyboard input in more than one process. The browser process receives the key events from the operating system and forwards them to a renderer process that owns the page or the editor. When an input method sends Backspace events followed by new characters back to back, and the renderer is busy, the events can be applied out of order or merged in ways the input method did not intend. The symptom is jumbled or doubled letters.

The general remedy is to keep a correction together and ahead of the user's next key. On macOS, Dokey inserts its synthetic events at the position of its own event tap, which places them ahead of any later key (Chapter 9). On Windows, it sends each correction in a single `SendInput` call and holds the user's next keys until the correction has passed through the hook (Chapter 10).

Editors that ignore synthetic Backspace

Rich-text editors built on ProseMirror and similar frameworks (the editor family used in many chat and note-taking applications) handle text input through browser input events rather than raw key events. In the project's tests on macOS 14, version 0.3.0 of Dokey typed `Haxy` into such an editor running in Microsoft Edge and produced *Hayãy*: the editor did not apply the synthetic Backspace, so the old letters stayed, and the new text *ãy* was inserted after them.

The fix, introduced in version 0.3.1, is **select-and-replace**: instead of *n* Backspace presses, send *n* presses of Shift+Left Arrow, which selects the old letters, and then type the new text, which replaces the selection. Typing over a selection is an operation every text control supports in the same way. Replaying the same event sequence with select-and-replace gave the correct *Hãy*. With the installed 0.3.1 build, typing into the ProseMirror editor gave fully correct text in 6 of 7 runs; the failing run was the first word right after the page loaded, and the second word onward was always correct.

Held keys and repeat suppression

A synthetic key event carries a virtual key code as well as its text. Early versions of Dokey on macOS sent the replacement text with the key code of the key just typed. When a user types fast, the previous key is often still held down when the next one arrives: in `Haxy`, the `a` key may still be down when `x` is pressed. A synthetic key-down for a key that is already down looks like an auto-repeat, and Chromium-based applications dropped it. `Haxy` gave *Hy* and `maf` gave *m*.

The fix, in version 0.3.2, is to send text with the code of a key that is *not* currently held, preferring keys that are rarely held during typing (Chapter 9). The project's notes record the result in the chat box of an Electron application: *Hãy hãy mà và hãy mà ô kìa…*, correct throughout, where the same typing had given *Hy* and *m* before.

Inline completions in address bars

Browser address bars, search fields and some other controls show an *inline completion*: as the user types, the control suggests the rest of a URL or term, and selects the suggested part after the caret. If the input method now sends a Backspace, the first Backspace deletes the selected suggestion, not the user's letter. On Windows, `thaayf` came out as *thaầy* and `ddoox` as *đđỗ*: one old letter survived because one Backspace was spent on the suggestion.

The fix is a **guard character**. Before deleting, the platform types one extra character. Typing replaces the selected suggestion with that character, and the control has nothing left selected. The platform then deletes one more character than the edit asked for, removing the guard. The first version of this fix, in 0.5.3, used U+200B ZERO WIDTH SPACE as the guard, because it is invisible. Tests in the real address bars of Chrome and Edge then showed that the browsers ignore U+200B when computing their suggestion, and brought the same completion back. Version 0.5.4 switched to U+00B7 MIDDLE DOT, which the address bar treats as a real character, so its suggestion ends. The dot appears for a moment and is deleted at once.

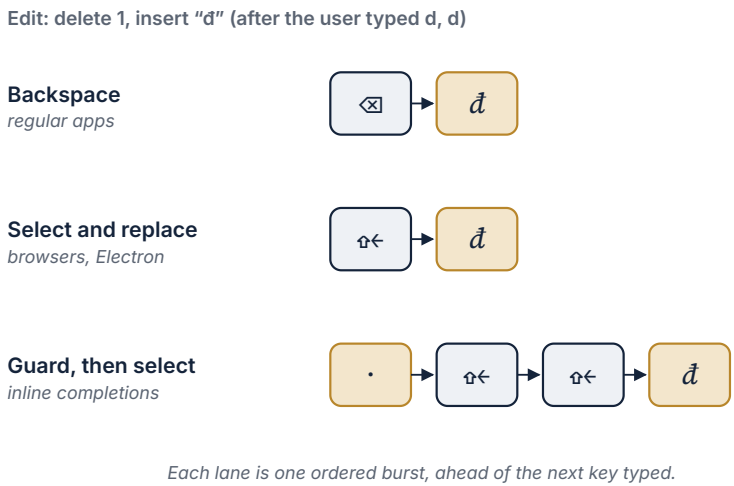


Figure 8.1 Three ways to deliver the same edit (delete 1, insert đ). Backspace delivery, select-and-replace, and select-and-replace with a guard character for controls that hold an inline completion.

8.3 Delivery strategies

From these cases, Dokey's delivery is assembled from a few independent choices.

How to remove the old letters. Either Backspace n times, or Shift+Left Arrow n times followed by typing over the selection. Backspace works in every control that accepts it and is the only option in terminal emulators, where Shift+Left Arrow is sent to the program as an escape sequence. Select-and-replace works in editors that ignore synthetic Backspace and is robust against reordering, because the replacement text and the selection are one operation for the application.

Whether to clear a selection first. In controls with inline completions, type a guard character first and remove it with one extra deletion step.

How many characters per event. A synthetic key event can carry a short string of UTF-16 code units. Sending the whole insertion in one event is fastest; some editing surfaces keep up better with one character per event. A surrogate pair must never be split across two events.

Whether to pause, and where. A short pause after the deletions and before the new text gives a slow renderer time to apply the deletions. A pause between characters helps surfaces that process each event slowly. Pauses cost latency, and on macOS a pause inside the event tap delays the user's next key, so they are used sparingly.

Which key code to attach to text. The text events must carry a key code, and it must not be one the user is holding.

8.4 Classifying applications

The right combination depends on the application, so both platforms classify the application in front of the user. The classes and their default strategies are summarised in Table 8.1. The exact values belong to the platform chapters; the table shows the shape.

Table 8.1 Default delivery by application class (Dokey 0.5.4).

Class	Examples	macOS	Windows
Browser	Chrome, Edge, Brave, Firefox, Arc; launchers with inline completion	select-and-replace, guard character, up to 20 units per event, no pauses	Backspace with guard, one `SendInput`, 12 ms pause before the new text (Chromium windows)
Electron or IDE	chat and note applications, VS Code, JetBrains IDEs	select-and-replace, up to 20 units per event, no pauses	as browser (same window class)
Regular	text editors, office applications, terminals	Backspace, up to 20 units per event, no pauses	Backspace with guard, one `SendInput`
Google Docs, Sheets, Slides	recognised from the window title	select-and-replace, one unit per event, 2 ms per key, 6 ms before new text	one key per `SendInput`, 2 ms per key, 6 ms before new text

A slower profile ("slow machine compatibility" in the user interface) doubles or adds pauses for users with older hardware. Both applications also let an advanced user override the defaults without rebuilding: on macOS through a `tuning` dictionary in the user defaults, on Windows through registry values (Chapters 9 and 10).

8.5 Why delivery is not in the engine

All of this could, in principle, be pushed into the engine: the engine could return a list of synthetic events instead of an edit. Dokey keeps it out for three reasons.

First, the strategies depend on facts only the platform knows: which application has focus, what its window class or bundle identifier is, which keys are physically held, whether a control has a selection. The engine would need all of these as inputs, and its tests would need to simulate them.

Second, the edit is the right unit for testing. The engine's tests can check, for every key sequence, that applying the edits to a text buffer gives the expected text. The delivery code's tests can check, for every edit, that the generated events are the expected sequence. Each half is tested against a simple model of the other.

Third, the same engine serves different platforms with different event models. The macOS application sends `CGEvent` objects with Unicode strings attached; the Windows application sends `INPUT` structures with `KEYEVENTF_UNICODE`. A future platform will have its own. The edit is the part they share.

8.6 Summary

The engine's minimal edit is a small, exact promise: delete n characters, insert this text, and the screen will match. Keeping that promise across real applications required four separate fixes, each found by testing in real software: keeping corrections ahead of the user's next key, replacing selections instead of sending Backspace, never reusing a held key's code, and clearing inline completions with a guard character. Dokey combines these into per-application delivery profiles. The next two chapters show how each platform captures keys, classifies applications and sends the events.

THE PLATFORMS

9

The macOS Platform Layer

The macOS application connects Dokey Core to the keyboard and to every application on the system. It is written in Swift with AppKit, builds as one universal binary for Intel Macs from macOS 10.13 and Apple Silicon Macs from macOS 11, and calls the engine only through the C API of Chapter 11. This chapter follows a key press from the keyboard to the screen: how the key is captured, which keys reach the engine, how the application in front is classified, and how the resulting edit is delivered as synthetic events. The code is in `apps/macos/Sources/`, mainly `KeyboardTap.swift` and `AppClassifier.swift`.

9.1 Capturing keys with an event tap

macOS offers two broad ways for a program to change what the user types. The first is the Input Method Kit, used by the input sources listed in System Settings; an input method built on it owns the text it is composing and hands it to the application as *marked text* until it is committed. The second is a **Quartz event tap**, a hook that sees keyboard and mouse events as they flow through the window server and can drop, modify or add events.

Dokey uses an event tap. Its edits are ordinary key events, so they work the same way in every application that accepts typing, including terminals and web pages, and there is never uncommitted text waiting for the application to accept it. The price is that the tap needs Accessibility permission, which the user grants once in

System Settings, and that Dokey must do itself the work that marked text would otherwise do: make sure its corrections land exactly where they should.

The tap is created at the session level and placed at the head of the chain of taps:

```
let mask: CGEventMask =
    (1 << CGEventType.keyDown.rawValue) |
    (1 << CGEventType.flagsChanged.rawValue) |
    (1 << CGEventType.leftMouseDown.rawValue) |
    (1 << CGEventType.rightMouseDown.rawValue) |
    (1 << CGEventType.otherMouseDown.rawValue)
guard let tap = CGEvent.tapCreate(tap: .cgSessionEventTap,
                                  place: .headInsertEventTap,
                                  options: .defaultTap,
                                  eventsOfInterest: mask,
                                  callback: tapCallback,
                                  userInfo:
Unmanaged.passUnretained(self).toOpaque()) else {
    return false
}
```

It listens to key presses, to changes of modifier keys (for the Vietnamese/English switch), and to mouse button presses (which move the caret and must reset the engine). It does not listen to key releases: the engine needs only key-downs.

Its own thread

The tap's run-loop source is added to the run loop of a dedicated thread, not the main thread:

```
let thread = Thread {
    CFRunLoopAddSource(CFRunLoopGetCurrent(), runLoopSource,
.commonModes)
    CGEvent.tapEnable(tap: tap, enable: true)
    CFRunLoopRun()
}
thread.name = "Dokey event tap"
thread.qualityOfService = .userInteractive
thread.start()
```

The main thread runs the menu bar item, the control panel and the update checker. If the tap shared that thread, opening a menu or drawing a window would delay every keystroke system-wide. On

its own thread, the tap's latency depends only on the engine and the delivery code. Settings that the main thread changes (the input method, the delivery profile for the front application, the switch hotkey) reach the tap thread through a small `TapState` structure guarded by a lock.

Keeping the tap alive

macOS disables an event tap that takes too long to respond, and it disables taps temporarily while secure input is active. The tap callback receives a notification in both cases (`tapDisabledByTimeout`, `tapDisabledByUserInput`), and Dokey re-enables the tap at once and starts a new word. As a second line of defence, a timer on the main thread checks every two seconds whether the tap is still enabled and switches it back on if not. The same timer creates the tap as soon as the user grants Accessibility permission, so Dokey starts working without being relaunched.

9.2 Which keys reach the engine

The tap's `keyDown` handler decides, for each key press, whether the engine sees it, whether the engine is reset, or whether the key simply passes.

1. **Dokey's own events pass.** Every event Dokey posts carries the tag `0x444F4B45` ("DOKE") in the `eventSourceUserData` field. The tap recognises its own output and lets it through untouched, so it never processes a correction as if the user had typed it.
2. **The switch hotkey toggles the mode.** By default the switch is ⌃⇧ pressed and released on its own; ⌥Z, ⌃Space and ⌘⇧Space are alternatives.
3. **In English mode, everything passes.**
4. **Auto-repeat resets.** A key held down produces repeated keydowns; the engine treats what follows as a new word.
5. **Shortcuts reset.** Any key with ⌘, ⌃ or ⌥ held goes straight to the application, and the engine starts a new word.
6. **Backspace** goes to `dokey_backspace_edit`. If the engine returns an edit (a tone moved, Chapter 7), Dokey swallows the Backspace

and delivers the edit; otherwise the Backspace passes.

7. **Return, keypad Enter and Tab** go to the engine as the characters `\r` and `\t`. If the engine returns an edit, Dokey removes the break character from the end of the edit's text, delivers the rest, and then posts a copy of the original key, tagged as its own. The word is finished before the key reaches the application.
8. **Escape** goes to `dokey_undo` if the user has enabled undo with Escape and the word has marks; otherwise it resets the engine and passes.
9. **Forward Delete, arrows, Home, End, Page Up and Page Down** reset the engine and pass.
10. **Everything else** is converted to a character with the current keyboard layout. If the key produces exactly one printable character, it goes to `dokey_key`; if not (function keys, control characters, half of a surrogate pair), the engine is reset and the key passes.

Converting with the current layout means Dokey works with any Latin keyboard layout: the engine receives the letter the user sees on the key, not a fixed US position. The one exception is the Korean prototype (Appendix G), which types by key position and reads letters from the key code.

Mouse button presses reset the engine, and so does a change of the front application, which the main thread observes through `NSWorkspace` notifications and passes to the tap.

9.3 Classifying the front application

When the front application changes, the main thread classifies it into one of three kinds (`AppKind`): browser, Electron/IDE, or normal. The classification looks first at the bundle identifier, then, for applications it does not know, at the frameworks the application ships:

```
static func classify(bundleID id: String, bundleURL: URL?) → AppKind {
    if browserPrefixes.contains(where: { id.hasPrefix($0) }) { return
.browser }
    if electronPrefixes.contains(where: { id.hasPrefix($0) }) { return
```

```
.electron }
    guard let url = bundleURL else { return .normal }
    return classify(frameworks:
url.appendingPathComponent("Contents/Frameworks"))
}
```

The browser list contains the bundle prefixes of Chromium-based browsers, Firefox and two launchers whose search fields behave like an address bar. The Electron/IDE list contains common Electron applications, the VS Code family and JetBrains IDEs. For any other application, the classifier lists `Contents/Frameworks`: an `Electron Framework.framework` or `Chromium Embedded Framework.framework` makes it an Electron application, and a Chromium-family framework makes it a browser. Results are cached per bundle identifier.

Two further rules refine the class at the moment of delivery:

- **Spotlight.** Spotlight's search field never becomes the front application, so the classifier cannot see it. If the front application is classed as normal, the tap asks the Accessibility API which application owns keyboard focus; if it is Spotlight, the edit is delivered with the browser profile, because Spotlight's field has an inline completion.
- **Google Docs, Sheets and Slides.** These web applications type into a hidden editing surface that keeps up best with one character per event and short pauses. The main thread reads the front window's title; if it contains the name of one of these applications (in English or Vietnamese), a dedicated site profile replaces the browser profile.

9.4 Delivery profiles

Each class maps to a `DeliveryProfile`:

```
struct DeliveryProfile {
    var select: Bool          // Shift+Left Arrow and type over, or
Backspace
    var unitsPerEvent: Int    // UTF-16 units per text event
    var step: useconds_t      // pause after each event
    var settle: useconds_t    // pause before the first new letter
    var tail: useconds_t      // pause after the last letter
```

```
    var textKey: Int          // key code for text events (-2 automatic,
-1 the typed key)
    var textUp: Bool          // also send key-up for text events
    var clearSelection = false // type the guard character first
}
```

Table 9.1 macOS delivery profiles in Dokey 0.5.4, from `AppClassifier.swift`.

Profile	select	units	step	settle	clear selection
Browser	yes	20	0	0 (2 ms slow)	yes
Electron/IDE	yes	20	0	0 (2 ms slow)	no
Normal	no	20	0	0	no
Google Docs family	yes	1	2 ms (4 ms slow)	6 ms (10 ms slow)	no

Earlier versions used pauses for browsers and Electron applications, and the application's README from that period still lists them (1.5 ms per key and 4 ms before typing for browsers). Version 0.3.1 removed them. The source comment explains why: the events are posted at the tap's own position, so they are already ahead of the user's next key, and sleeping inside the tap only delays that next key. The slow-machine switch now adds a 2 ms settle for browsers and Electron applications only.

Every value can be overridden without a rebuild. Dokey reads a `tuning` dictionary from its user defaults every two seconds, with keys of the form `<kind>.<field>`:

```
defaults write com.dhtoan.dokey.mac tuning -dict electron.settle -int
5000
defaults delete com.dhtoan.dokey.mac tuning
```

Values are clamped to safe ranges (for example, pauses to at most 100 or 200 milliseconds and units to between 1 and 20).

9.5 Delivering an edit

`deliver` turns one edit into events. It runs on the tap thread, inside the callback for the key that caused the edit, and it posts every event with `CGEventTapPostEvent`. Events posted that way enter the event stream immediately after the tap, which means they reach the application before any event that arrives at the tap later, in-

cluding the user's next key. That ordering is the macOS answer to the reordering problem of Chapter 8.

The steps are:

1. **Choose the profile**: the site profile if one applies, otherwise the class profile; the browser profile if Spotlight has focus. For the Korean prototype in the VS Code family, selection is turned off, because the editor's integrated terminal reads Shift+Left Arrow as an escape sequence.
2. **Decide whether to select.** Selection is used only if the profile allows it, the edit inserts text, and the encoding is precomposed Unicode. With combining tone marks, Left Arrow moves over a whole character, which may be two code units, so Backspace is used instead.
3. **Delete.** The deletion steps are computed by a small pure function:

```
swift static func deletionSteps(backspaces: Int, select: Bool, clearSelection: Bool) → [DeletionStep] { guard backspaces > 0 else { return [] } let count = clearSelection ? backspaces + 1 : backspaces let steps = [DeletionStep](repeating: select ? .selectLeft : .backspace, count: count) return clearSelection ? [.guardCharacter] + steps : steps }
```

With `clearSelection`, the guard character U+00B7 is typed first, replacing any inline completion, and one more deletion step removes it. Each Shift+Left Arrow is posted with only the Shift flag (plus the flags macOS uses for arrow keys), and each Backspace with no flags at all, so a Shift key the user is physically holding cannot turn a Backspace into Shift+Backspace. 4. **Settle**, if the profile has a settle pause and the edit both deletes and inserts. 5. **Type the text** in chunks of at most `unitsPerEvent` UTF-16 units, never splitting a surrogate pair. Each chunk is one key event whose Unicode string is set with `keyboardSetUnicodeString`. The event carries a key code that is not currently held, chosen by `freeTextKey`. 6. **Replay the original key** for Return and Tab, tagged as Dokey's own.

The free text key

A text event must carry some virtual key code, and section 8.2 showed what happens when it is the code of a key still held down. `freeTextKey` asks the system which keys are down at this moment and picks the first free one from a short list of keys that are rarely held while typing: *b*, *v*, *n*, comma, period, grave accent, semicolon.

```
private func freeTextKey() → CGKeyCode {
    for key: CGKeyCode in [11, 9, 45, 43, 47, 50, 41]
        where !CGEventSource.keyState(.hidSystemState, key: key) {
        return key  // b, v, n, comma, period, grave, semicolon
    }
    return 11
}
```

The application sees a key-down for, say, the *b* key, carrying the string *ã*. Because *b* is not physically down, no application treats it as a repeat.

The Korean prototype makes one exception, found by measurement and recorded in the source: in Chromium, a letter key carrying a Hangul character was sometimes treated like Korean IME input, which lost the next selection. Korean text in browsers and Electron applications is therefore tagged with the Space key code, unless Space itself is held.

Event source and flags

Synthetic events are created from a private event source (`CGEventSource(stateID: .privateState)`), so their modifier state is independent of the keys the user holds. Text events carry no modifier flags. Combined with the tag in `eventSourceUserData`, this makes Dokey's events self-contained: they mean the same thing whatever the user is doing at the time.

9.6 Secure input

When an application enables secure input (password fields, or Terminal's Secure Keyboard Entry), macOS stops delivering key events to every event tap. This is a deliberate security feature of the operating system, and no input method built on an event tap

can type in those fields. Dokey's tap is disabled for the duration and re-enabled afterwards by the mechanisms described above.

9.7 Per-application memory and exclusions

Dokey remembers the Vietnamese or English mode separately for each application (an option, on by default). Switching to English in a code editor and back to Vietnamese in a chat application then needs no further key presses. A user can also exclude an application entirely from the menu bar; an excluded application always types English until the user presses the switch hotkey inside it, which removes the exclusion. Both features change only the `vietnamese` flag in `TapState`; the engine is not involved.

9.8 Diagnostics

For diagnosing lost letters, Dokey has a debug log, off by default:

```
defaults write com.dhtoan.dokey.mac debugLog -bool true
```

With it on, the tap appends one line per physical key (time only, not which key) and one line per delivered edit, with the application class, the deletion method, the number of characters deleted and the inserted text, to `~/Library/Logs/Dokey.log`. Because the inserted text is logged, the file contains the words typed. It never leaves the machine, and the application's documentation says so.

The binary also has command-line modes used in the build and in CI: `--self-test` calls the engine through the C API from Swift and checks Telex, VNI, English words, end-of-word correction, the traditional tone style and the application classifier; `--render-panel` and `--render-marketing` draw the control panel to image files for review without opening a window.

9.9 Summary

The macOS layer captures keys with a session-level event tap on its own thread, converts each key with the current layout, and passes printable keys to the engine. It classifies the front application by

bundle identifier and frameworks, refines the class for Spotlight and Google's editors, and delivers each edit through a profile: select-and-replace or Backspace, an optional guard character, chunked Unicode text events carrying a free key code, all posted at the tap's position so they arrive before the user's next key.

THE PLATFORMS

10

The Windows Platform Layer

The Windows application is a native Win32 program written in C++, with a control panel drawn in GDI+. It builds for x86, x64 and ARM64 with a statically linked C runtime, needs no administrator rights and no separate runtime installation, and calls the engine through the same C API as the macOS application. This chapter describes how it captures keys, how it keeps its corrections in order, and how it builds the synthetic input for each edit. The code is in `apps/windows/src/`, mainly `keyboard.cpp` and `inject.h`.

An important caveat applies to this whole chapter, and the project states it in its own documentation: as of version 0.5.4, the Windows application has not been run on a physical Windows PC. It was written on macOS, and every test described here ran on GitHub Actions virtual machines (Windows Server 2022, x64). The x86 and ARM64 builds have been compiled and checked for architecture but never run. Chapter 13 lists the gaps.

10.1 Capturing keys with a low-level hook

Windows offers several ways to see keystrokes system-wide. Dokey uses the **low-level keyboard hook**, installed with `SetWindowsHookExW(WH_KEYBOARD_LL, ...)`. A low-level hook procedure is called for every key event before it is delivered to any application, and it can swallow the event by returning a non-zero value. Unlike other hook types, a low-level hook is not injected into other processes: the system calls it in the installing thread's context, by

sending that thread a message. The hook therefore works for 32-bit and 64-bit applications alike, from a single process.

That design has a cost. The installing thread must pump messages, and it must answer each hook call quickly. If a low-level hook procedure takes too long, Windows skips it, and after repeated timeouts it may remove the hook silently. Dokey's design follows from that constraint.

Two threads

Dokey runs its keyboard work on two threads of its own, separate from the user interface thread:

- The **input thread** owns the engine, installs the keyboard hook and a low-level mouse hook, and registers WinEvent hooks for foreground and focus changes. The hook procedures run on this thread. They call the engine and queue work, and they never block.
- The **sender thread** takes jobs from a queue and calls `SendInput` for each one, in order.

The input thread runs at `THREAD_PRIORITY_TIME_CRITICAL`, so that it answers hook calls before any other thread in the process. Settings changes from the user interface reach it as a posted thread message carrying a copy of the settings, which the input thread applies and frees. No setting is ever read across threads without that handoff.

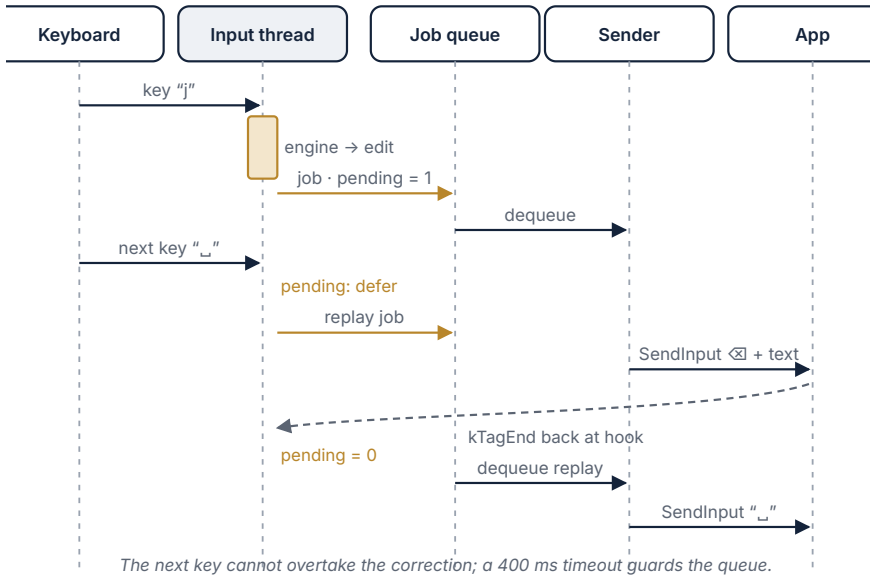


Figure 10.1 The Windows threads. The hook procedure on the input thread runs the engine and queues jobs; the sender thread calls `SendInput`; keys typed while a job is pending are queued behind it.

10.2 Which keys reach the engine

The hook procedure looks at each key-down and decides:

1. **Dokey's own events** are recognised by a tag in `dwExtraInfo` and passed through (see below).
2. **Modifier keys** pass; Dokey only notes whether Ctrl and Shift alone are being pressed, for the Ctrl+Shift switch.
3. **The switch hotkeys** (Ctrl+Shift released on its own, or Alt+Z) toggle Vietnamese and English. For Alt+Z, Dokey also sends an unassigned virtual key so that releasing Alt does not open the window's menu bar.
4. `VK_PACKET` **events**, which carry text injected by another program, reset the engine.
5. **When typing is off**, when the front application is on the exclusion list, or when an East Asian IME is open, keys pass.
6. **Shortcuts** with Ctrl, Alt or Windows held reset the engine and pass.

7. **Backspace** goes to `dokey_backspace_edit`; an edit, if returned, replaces the Backspace.
8. **Escape**, if undo with Escape is enabled, goes to `dokey_undo`.
9. **Neutral keys** (Caps Lock, Num Lock, Scroll Lock, media and volume keys) pass without resetting the word.
10. **Everything else** is translated to a character with `ToUnicodeEx`, using the keyboard layout of the foreground window's thread. A key that produces no single character (arrows, Home, Delete, function keys) resets the engine. A key that produces a character goes to `dokey_key`.

The translation call passes flag `0x4`, which asks `ToUnicodeEx` not to change the keyboard state. Without it, translating a dead key in the hook would consume the dead key, and accented characters typed with a dead-key layout would break in the application.

Mouse clicks (from the low-level mouse hook) reset the engine. So do changes of foreground window and changes of focus to a different window, reported by WinEvent hooks registered out of context. The focus hook ignores focus events within the same window, because Chromium raises them for pop-ups and accessibility while the user is still typing in the same field.

10.3 Keeping corrections in order

The ordering problem of Chapter 8 has a particular shape on Windows. When the hook swallows a key and queues a correction, the correction is sent by `SendInput` from the sender thread. `SendInput` places the events in the system's input stream, where they are processed, like hardware input, by the low-level hooks (including Dokey's own) and then delivered to the foreground thread. If the user presses another key in the meantime, that key could reach the application before Dokey's correction.

Dokey prevents this with a **pending** state:

- A job is pending from the moment it is queued until the hook sees the *last* event of that job come back through the hook.

- While any job is pending, keys that would normally pass straight through are not passed. They are swallowed and queued as **replay jobs** behind the pending job, and the sender thread later sends them with `SendInput` in order.

```
// Lets the key through, or queues it behind a pending job.
bool PassOrDefer(const KBDLLHOOKSTRUCT& k) {
    if (!IsPending()) return false;
    Enqueue(ReplayJob(k));
    g_swallowed[k.vkCode] = true;
    return true;
}
```

Every key that Dokey swallows has its virtual key code recorded, and the matching key-up is swallowed too, so the application never sees a key-up without its key-down.

Recognising its own events

Every `INPUT` structure that Dokey sends carries one of two tags in `dwExtraInfo`:

```
constexpr ULONG_PTR kTagMid = 0x444F4B00;  // "DOK\0"
constexpr ULONG_PTR kTagEnd = 0x444F4B01;
```

All events of a job carry `kTagMid` except the last, which carries `kTagEnd`. When the hook sees a key-up with `kTagEnd`, it knows the job has passed through the input stream, and it decrements the pending count. The hook never feeds tagged events to the engine.

A timeout

If Dokey's own events never come back (because User Interface Privilege Isolation blocked them from reaching a window of higher integrity, or because another hook swallowed them), the pending state would block the keyboard. `IsPending` therefore gives up after 400 milliseconds without progress and clears the count. A lost correction is a bad outcome; a frozen keyboard would be a worse one.

10.4 Building the input for an edit

`BuildInputs` in `inject.h` turns a job into an array of `INPUT` structures. It is a pure function with its own unit tests (Chapter 12). For an edit that deletes n characters and inserts some text, it produces, in order:

1. If a guard character is configured and $n > 0$: the guard as a Unicode key-down and key-up (`KEYEVENTF_UNICODE` with `wScan` set to the character).
2. n Backspace presses, plus one more if a guard was typed.
3. The text, one Unicode key-down and key-up per UTF-16 code unit.
4. If the job replays a key (Enter, Tab, or a deferred key): that key's down and up, with its original scan code and extended-key flag.

The last event is tagged `kTagEnd`. The function also reports the index where the deletions end, so that the sender can pause there.

Unicode key events (`KEYEVENTF_UNICODE`) deliver a character without a virtual key code, as a `VK_PACKET` message. They do not depend on the keyboard layout and never collide with keys the user is holding, which is why the held-key problem of Chapter 8 does not arise on Windows in the same form.

The guard character

The guard character addresses inline completions (Chapter 8). On Windows it is applied to every edit that deletes characters, not only in browsers, because inline completions appear in many places: browser address bars, Excel cells, File Explorer's address field. The default is U+00B7 MIDDLE DOT, stored in the registry value `SelectionGuard`, which can also be set to 0 to turn the guard off or to 65535 to use the Delete key instead of a character:

```
constexpr char16_t kSelectionGuard = 0x00B7;  // MIDDLE DOT
constexpr char16_t kGuardDelete = 0xFFFF;     // SelectionGuard value:
the Delete key instead
```

The history of this value is told in Chapter 8: U+200B was tried first in version 0.5.3, and the real address bars of Chrome and Edge ig-

nored it.

Sending

The sender thread sends each job in one of three ways:

- **One call.** The normal case: all events of the job go to a single `SendInput` call. Windows guarantees that the events of one call are inserted into the input stream serially, without other input interleaved.
- **Two calls with a pause.** In windows whose class name starts with `Chrome_WidgetWin_` (Chrome, Edge, Brave, Opera and Electron applications), an edit that both deletes and inserts is split at the end of the deletions, with a pause in between. The pause defaults to 12 milliseconds and is stored in the registry value `ChromiumPauseMs`.
- **One key at a time.** In Google Docs, Sheets and Slides, recognised from the window title in English or Vietnamese, each key's down and up are sent with their own `SendInput` call, with 2 milliseconds between keys and 6 milliseconds before the new text. With a Chromium pause of 30 milliseconds or more (the slow-machine setting), these become 4 and 12 milliseconds.

If `SendInput` reports that it inserted fewer events than requested, the end marker will never come back, and the sender clears the job's pending state itself.

10.5 Other input methods

Dokey steps aside while another input method that composes text is active: a Chinese, Japanese or Korean IME, or Microsoft's own Vietnamese Telex keyboard. The user interface thread asks the focused window's default IME window whether it is open, using `ImmGetDefaultIMEWnd` and `SendMessageTimeoutW` with `IMC_GETOPENSTATUS`, and passes the answer to the input thread. The timeout keeps a hung application from blocking Dokey. While the answer is yes, every key passes untouched.

10.6 Limits of the platform

- **Integrity levels.** A program running without administrator rights cannot send input to a window running with them; Windows blocks it through User Interface Privilege Isolation. Dokey runs as a normal user and therefore does not type in elevated windows. The pending timeout keeps this from freezing the keyboard.
- **Layouts.** Characters come from the foreground window's keyboard layout. Non-Latin layouts produce non-ASCII characters, which the engine treats as word breaks, so they pass untouched.

10.7 Settings and diagnostics

Settings are stored under `HKCU\Software\Dokey`. Besides the options visible in the control panel, two values exist only in the registry: `ChromiumPauseMs` and `SelectionGuard`, described above, and `DebugLog`. With `DebugLog` set to 1, Dokey writes a log to `%TEMP%\dokey-debug.log`. Unlike the macOS log, it records only events and counts (a key arrived, an edit with so many deletions and characters, a reset and its reason, the class name of a newly focused window), never which key was typed or what text was inserted.

10.8 Summary

The Windows layer captures keys with a low-level keyboard hook on a time-critical thread that never blocks, sends corrections from a second thread with `SendInput`, and keeps them in order by holding back the user's later keys until the correction's tagged end marker has come back through the hook. It types a guard character before every deletion to clear inline completions, splits edits with a short pause in Chromium windows, and types one key at a time in Google's editors. Its tests run on virtual machines; it has not yet been run on real hardware.

THE PLATFORMS

11

The C API

The engine is written in C++, but the applications that use it are written in Swift and in C++ with Win32, and a future application might be written in Kotlin through JNI or in any language with a C foreign-function interface. The boundary between them is a small C API declared in `include/dokey/dokey.h`. This chapter is a reference for that API: its types, each function's contract, the rules for threads and memory, and complete examples.

11.1 Design principles

The API follows a few rules that make it easy to bind from other languages.

- **One opaque handle.** All state lives behind `dokey_engine*`. The caller never sees a C++ type.
- **Plain structures.** Options and edits are fixed-size C structures with `int` and `uint16_t` fields. There are no pointers inside them and nothing to free.
- **No callbacks.** The engine never calls back into the platform. Every function returns its result.
- **No allocation by the caller.** The caller passes a `dokey_edit` by pointer; the engine fills it.
- **No errors to handle.** Every function accepts a null engine pointer and does nothing. Invalid options fall back to defaults. An edit that cannot be expressed is dropped and the word reset.

- **UTF-16 out, code points in.** Keys go in as Unicode code points (`uint32_t`); text comes out as UTF-16, which is what both macOS (`UniChar`) and Windows (`WCHAR`) use for keyboard events.

11.2 Types

```
typedef struct dokey_engine dokey_engine;

enum { DOKEY_TELEX = 0, DOKEY_VNI = 1 };
enum { DOKEY_UNICODE = 0, DOKEY_UNICODE_COMBINING = 1 };
enum { DOKEY_LANG_VIETNAMESE = 0, DOKEY_LANG_KOREAN = 1 };

typedef struct {
    int method;            /* DOKEY_TELEX or DOKEY_VNI */
    int encoding;          /* DOKEY_UNICODE or DOKEY_UNICODE_COMBINING
*/
    int modern_tone_style; /* 1: hoà, khoẻ, thuý */
    int check_spelling;    /* 1: keep non-Vietnamese words as typed */
    int quick_telex;       /* 1: cc gg kk nn pp qq tt at a word start →
ch gi kh ng ph qu th */
} dokey_options;

#define DOKEY_MAX_EDIT 256

typedef struct {
    int backspaces;                 /* characters to delete first */
    int length;                     /* UTF-16 units in text */
    uint16_t text[DOKEY_MAX_EDIT];  /* text to insert */
} dokey_edit;
```

`dokey_options` fields are interpreted leniently: any `method` other than `DOKEY_VNI` means Telex, any `encoding` other than `DOKEY_UNICODE_COMBINING` means precomposed, and the three flags are true when non-zero. A null options pointer gives the defaults: Telex, precomposed Unicode, modern tone style, spelling checks on, Quick Telex off.

`dokey_edit` is 520 bytes on common platforms (two `int` fields and 256 `uint16_t`). It is meant to live on the caller's stack and be reused for every key.

11.3 Functions

dokey_create

```
dokey_engine* dokey_create(const dokey_options* options);
```

Creates an engine with the given options (or defaults if `options` is null) and returns its handle. The Vietnamese syllable dictionary is shared by all engines in the process and built on first use (Chapter 5), so the first key typed after process start pays a one-time cost of a few milliseconds.

dokey_destroy

```
void dokey_destroy(dokey_engine* engine);
```

Frees the engine. The handle must not be used afterwards.

dokey_set_options

```
void dokey_set_options(dokey_engine* engine, const dokey_options*
options);
```

Replaces the options and resets the engine, so the next key starts a new word. Text shortcuts are kept.

dokey_key

```
int dokey_key(dokey_engine* engine, uint32_t ch, dokey_edit* out);
```

The main entry point. Pass every character the user types: letters, digits, punctuation, space, and also Enter as `'\r'` and Tab as `'\t'`, so that they can end words.

- Returns **0** if the platform should let the original key through unchanged. `out→backspaces` and `out→length` are set to 0.
- Returns **1** if the platform must swallow the original key, delete `out→backspaces` characters before the caret, and insert the `out->length` UTF-16 units of `out→text`.

When the key is a word break and the engine returns 1, the break character is the last unit of `out→text`. A platform that delivers Enter or Tab as a real key press, as both Dokey applications do, removes it from the text and replays the original key after the edit (Chapters 9 and 10).

If an edit would need more than `DOKEY_MAX_EDIT` units, which can only happen with a long text shortcut, the C API returns 0 and resets the word. The original key then goes through, and the shortcut does not expand.

dokey_backspace and dokey_backspace_edit

```
void dokey_backspace(dokey_engine* engine);
int  dokey_backspace_edit(dokey_engine* engine, dokey_edit* out);
```

Call one of these when the user presses Backspace.

`dokey_backspace` is for platforms that always let Backspace through. It updates the engine's view of the word (Chapter 7) and returns nothing.

`dokey_backspace_edit` is for platforms that can replace a Backspace with an edit. It returns 1 when the platform must swallow the Backspace and apply `out` instead; the edit already includes the deletion the user asked for. This happens when deleting a letter moves the tone (*hoàn* ⌫ → *hòa* in the traditional style). It returns 0 when the Backspace should go through. It is also the function that reopens the previous word when Backspace deletes the single space after it.

dokey_undo

```
int dokey_undo(dokey_engine* engine, dokey_edit* out);
```

Replaces the current word with the keys exactly as typed (*tiếng* → `tieengs`) and puts the rest of the word in literal mode. Returns 1 and the edit to apply, or 0 when there is nothing to undo, in which case the platform should let the triggering key (Escape in both applications) through.

dokey_set_macros

```
void dokey_set_macros(dokey_engine* engine, const char* utf8_lines);
```

Sets the text shortcuts from UTF-8 text with one shortcut per line, in the form `shortcut=expansion` or `shortcut:expansion`. Spaces and tabs around both sides are trimmed, `\r\n` line endings are accepted, and lines that are empty, start with `#`, or have no separator are skipped. Passing null or an empty string turns shortcuts off. Matching is case-insensitive, and the case of the typed abbreviation decides the case of the expansion (Chapter 7).

dokey_reset

```
void dokey_reset(dokey_engine* engine);
```

Starts a new word and forgets the previous one. Call it whenever the caret may have moved or the text may have changed without the engine's knowledge: mouse clicks, arrow and navigation keys, keyboard shortcuts, changes of application, window or focus, held keys, switching typing on or off. When in doubt, reset.

dokey_set_language

```
void dokey_set_language(dokey_engine* engine, int language);
```

Selects Vietnamese (the default) or the prototype Korean composer (Appendix G). Starts a new word. In Korean mode, the Vietnamese options, text shortcuts and undo have no effect.

11.4 Threads and memory

An engine is not thread-safe. Each engine must be used from one thread at a time, and both applications call it only from their input thread (the event tap thread on macOS, the hook thread on Windows). Different engines may be used from different threads.

The shared syllable dictionary is a function-local static constant, initialised on first use. C++11 and later guarantee that such initiali-

sation is thread-safe, and the dictionary is never written after it is built.

The engine allocates memory internally for its strings, with C++ `new`, and frees it in `dokey_destroy`. If allocation fails, the C++ exception will propagate through the C API, which C callers cannot catch; on the platforms Dokey targets, the engine's few small allocations are not expected to fail.

11.5 A complete example in C

The program below types a short Telex sequence through the API and applies each edit to a text buffer, exactly as a platform would apply it to a text field. It is the same loop that the test helper `Engine::simulate` runs.

```
#include <dokey/dokey.h>
#include <stdio.h>

static uint16_t text[1024];
static int len = 0;

static void type(dokey_engine* e, const char* keys) {
    dokey_edit edit;
    for (const char* k = keys; *k; k++) {
        if (dokey_key(e, (unsigned char)*k, &edit)) {
            len -= edit.backspaces < len ? edit.backspaces : len;
            for (int i = 0; i < edit.length; i++) text[len++] =
edit.text[i];
        } else {
            text[len++] = (unsigned char)*k;   /* the key goes through
*/
        }
    }
}

int main(void) {
    dokey_options opt = { DOKEY_TELEX, DOKEY_UNICODE, 1, 1, 0 };
    dokey_engine* e = dokey_create(&opt);
    type(e, "Xin chaof cacs banj ");
    dokey_destroy(e);
    /* text now holds "Xin chào các bạn " in UTF-16 */
    printf("%d UTF-16 units\n", len);
    return 0;
}
```

11.6 Binding from Swift

The macOS application imports `dokey.h` through a bridging header and wraps the handle in a small class (`apps/macos/Sources/Engine.swift`). Because Swift sees the fixed-size `text` array as a tuple, the wrapper copies each edit into a Swift value with a `[UInt16]` array. The wrapper also serialises calls with a lock, so that the main thread can change options while the tap thread types.

The C API added `dokey_set_language` as a function instead of a new field in `dokey_options` for a Swift-specific reason recorded in the design notes: Swift constructs `dokey_options` with a member-wise initialiser that lists every field, so each new field would force a change at every call site.

11.7 Command-line tool

The library ships with a command-line tool, `dokey`, built from `tools/dokey_cli.cpp`. It converts Telex or VNI text given as arguments or on standard input:

```
$ dokey "Xin chaof cacs banj"
Xin chào các bạn
$ dokey --vni "Vie65t Nam"
Việt Nam
$ dokey --old-tone "hoaf binhf"
hòa bình
```

Its options mirror `dokey_options`: `--vni`, `--old-tone`, `--no-spell`, `--combining` and `--quick-telex`. The tool appends a space to each line so that the last word is finished, then removes it. It is useful for checking the engine's behaviour on a word without a keyboard, and for reporting bugs with exact inputs.

11.8 Stability

The C API has grown by addition since version 0.1: `dokey_backspace_edit`, `dokey_undo`, `dokey_set_macros` and the `quick_telex` field arrived in 0.4, and `dokey_set_language` in 0.5. No function has changed meaning. The `dokey_options` structure did

grow a field in 0.4, which is a source-compatible change for C callers that initialise it by position with fewer fields (the remainder is zeroed) but requires recompilation. A platform layer that is built together with the engine, as both Dokey applications are, is unaffected.



PART IV

The Evidence

Chapters 12–16

THE EVIDENCE

12

Testing Strategy

An input method is tested every time someone types with it, and its failures are visible at once. That makes it tempting to test by typing. Dokey's testing strategy starts from the opposite end: the engine is a pure function of its inputs, so most of its behaviour can be checked automatically, exhaustively where the input space is small and by representative corpora where it is not. Typing tests are reserved for what only typing can reveal, which is the delivery of edits into real applications. This chapter describes each layer of tests, what it checks and how it is built. The next chapter reports the results and lists what remains untested.

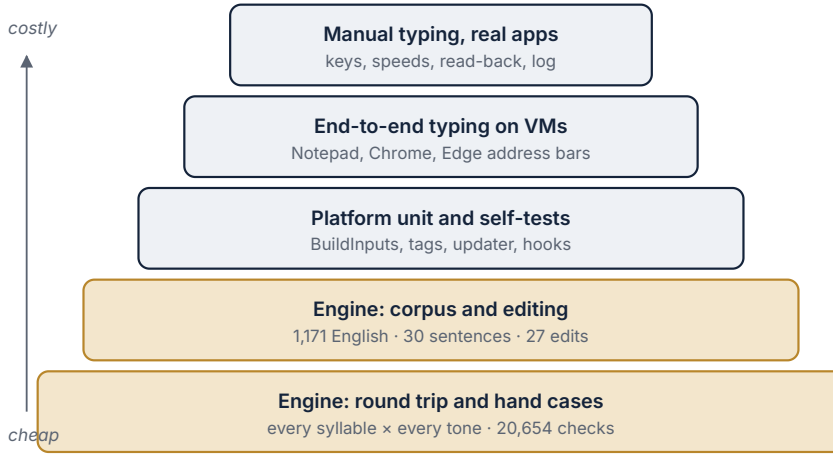


Figure 12.1 The layers of Dokey's tests, from exhaustive engine checks at the bottom to manual typing in real applications at the top.

12.1 Principles

Four principles shape the tests.

Test the engine through its edits. The engine's output is a sequence of edits, not a word. Almost every engine test therefore types a key sequence through the same loop a platform uses (apply each edit to a text buffer, or let the key through) and compares the final buffer with the expected text. A test that compared only `currentWord()` would miss an edit that deletes one character too many.

Write expectations independently. An expectation computed by the code under test proves nothing. The hand-written cases are literal strings. The round trip computes its expectations with a second implementation of key encoding and tone placement, written in the test file from the specification.

Make the data the test. The English and Vietnamese regression corpora are plain text files in `tests/data/`. Adding a word to a file adds a test. The list of English words that become Vietnamese is also a file, and the test fails if the engine's behaviour changes for any of them, in either direction.

Run everywhere, on every change. The core tests run on Linux, macOS and Windows in continuous integration after every push and pull request.

12.2 The engine test program

All engine tests are in one program, `tests/engine_tests.cpp`, built by CMake as `dokey_tests` and run by `ctest`. It needs nothing but the library and the data directory, whose path is compiled in. Each check increments a counter; the program prints the first 60 failures and exits with a non-zero status if any check failed. At the commit described in this book it performs 20,654 checks.

Hand-written cases

The first group is a table of key sequences and expected results for Telex: 119 cases that cover the tone keys, every diacritic, *qu* and *gi*, *ươ* in all its typing orders, undo by repetition, the double-letter rules, English words, punctuation, numbers and Backspace. Smaller tables cover the traditional tone style (4 cases), VNI (13 cases) and the combining encoding (1 case). A few representative rows:

Keys	Expected	What it checks
`dduo`	*đuo*	a half-typed word stays Vietnamese
`dduocjw␣`	*được␣*	tone before horn
`thuowr␣`	*thuở␣*	*ươ* → *uơ* at the end
`aw␣`	*aw␣*	*ă* alone is not a syllable
`tiengs␣`	*tiengs␣*	no circumflex typed: not a word
`tesst␣`	*test␣*	Telex double before a consonant
`password␣`	*password␣*	English double before a vowel
`2026 laf nawm␣`	*2026 là năm␣*	digits break words in Telex
`tieengs⌫⌫n␣`	*tiến␣*	resume after Backspace
`a11␣` (VNI)	*a1␣*	VNI undo

The round trip

The round trip is the largest group and the closest to exhaustive. For every syllable in the engine's dictionary and every tone that syllable may carry, it types the syllable in canonical Telex, ends the word with a space, and compares the result with an independently computed expectation:

```
for (const std::u16string& syl : allSyllables()) {
    // "oo" needs a third o in Telex (booong → boong); covered by hand
cases.
    if (syl.find(u"oo") ≠ std::u16string::npos) continue;
    for (int tone = 0; tone ⩽ 5; tone++) {
        if (isStop(syl) && tone ≠ 0 && tone ≠ 1 && tone ≠ 5)
continue;
        std::u16string expected = expectedWithTone(syl, tone, true) + u"
";
        check(telexKeys(syl, tone) + " ", toUtf8(expected), o,
"roundtrip");
    }
}
```

Both helper functions, `telexKeys` and `expectedWithTone`, are written in the test file. `telexKeys` writes each marked letter as its Telex pair and the tone key last; `expectedWithTone` implements the placement rules of Chapter 6 on Unicode strings (the listing is in that chapter). Neither calls the engine's parser, encoder or placement code.

Two exclusions are deliberate and visible in the code. Syllables containing *oo* (*boong*, *xoong*) are skipped because typing them in Telex needs three *o* keys, since `oo` alone makes *ô*; hand-written cases cover them. Checked syllables are typed only with no tone, acute and dot below, because the other tones are not allowed on them; the gate's refusal is covered by other cases.

The round trip iterates over the 3,964 entries that the dictionary builder generates, including the seven duplicates noted in Chapter 2, so a few syllables are typed twice. The test prints a one-line summary: `round-trip: 3964 syllables x tones, 0 failures`.

Editing features

The `editing()` group drives the engine through the edit-returning entry points that the applications use: `backspaceEdit()` for

Backspace (written `\b` in the test strings) and `undo()` for Escape (`\x1b`). Its 27 checks cover reopening the previous word, Backspace that moves the tone in both styles, *ứ* + *o* completion, the Central Highlands spellings, undo and literal mode, Quick Telex on and off, and text shortcuts with their case rules. The cases quoted in Chapter 7 come from this group.

Regression corpus

The `corpus()` group reads three files:

- `english.txt`: common English words, developer vocabulary and brand names, 1,171 entries in all (1,138 distinct words), each of which is typed followed by a space;
- `english_known.txt`: the 73 distinct words among them (74 entries) whose keys spell a Vietnamese syllable, each with the result the engine must produce (`this → thí`);
- `vietnamese_telex.txt`: 30 real Vietnamese sentences with their Telex keys, typed with tone keys at the end of each syllable as most people type.

Every English word must come out exactly as typed unless it is in the known list, in which case it must come out exactly as listed. Every sentence must come out exactly. The group also fails if the files are missing or suspiciously small (fewer than 1,000 words or 25 sentences), so that a broken data path cannot make the test pass by testing nothing.

C API, pass-through and edit shapes

Three small groups check properties that the larger groups take for granted:

- **C API.** Typing `aa` through `dokey_key` must return 1 with one Backspace and the single unit U+00E2. A second check sets shortcuts through `dokey_set_macros` (including a comment line, a `\r\n` line ending and the `:` separator), expands `btw`, and checks the shape of the edit; then it types `tieengs` and checks that `dokey_undo` returns an edit.

- **Pass-through.** Typing `b`, `a`, `n` must produce three unhandled keys: plain letters never cause synthetic events.
- **Doubled-key edits.** For `ddoox`, the exact edit of every key is specified: not handled, then delete 1 and insert *đ*, not handled, delete 1 and insert *ô*, delete 1 and insert *ỗ*. An application that only ever sees these edits must end with *đỗ*. This test was added when the address-bar failure of Chapter 8 was investigated, to establish that the engine's edits were correct and the fault lay in delivery.

12.3 The Korean composer's tests

The prototype Korean composer has its own program, `tests/hangul_tests.cpp`, with 53,204 checks. Besides hand-written cases, it generates 20,000 random words (250,522 keys in the run for this book) and checks invariants on every key: every key is rewritten (Korean composition replaces the last syllable on almost every key), deletions and insertions stay small (0.33 characters deleted and 1.04 inserted per key on average), and Enter is never handled. Appendix G describes the composer.

12.4 Platform tests

The platform layers cannot be tested without their operating systems, but most of their logic can still be tested without a person.

macOS

The macOS application has a built-in self-test, run by `Dokey --self-test` at the end of every build and in continuous integration on macOS 15 with Apple Silicon. It calls the engine through the C API from Swift (checking Telex, VNI, English words, end-of-word correction and the traditional tone style), runs the application classifier on known bundle identifiers, and draws the menu bar icon. The build also verifies the universal binary's architectures, the bundle's property list, the ad-hoc signature and the disk image (Chapter 15). The deletion-step function of Chapter 9 is a pure static function so that it can be checked the same way.

Windows

The Windows application has three automated layers, all run by the `windows-app.yml` workflow on a `windows-2022` virtual machine:

1. **Unit tests** (`apps/windows/tests/inject_tests.cpp` and `updater_tests.cpp`), built for x64 and x86 and run with `ctest`. The injection tests check the `INPUT` sequences that `BuildInputs` produces for given jobs, the `dwExtraInfo` tags, the guard character and the engine contract the hook relies on. The updater tests check version parsing and comparison.
2. **Self-test.** `Dokey.exe --self-test` installs and removes the keyboard and mouse hooks, for x64 and x86.
3. **End-to-end typing** (`apps/windows/tests/e2e.ps1`). The script starts Dokey and a target application, types Telex as plain key presses with `SendInput`, exactly as a physical keyboard would, and reads the text back. It deliberately avoids PowerShell's `SendKeys`, which calls `BlockInput` and would also block Dokey's own `SendInput`. Targets include Notepad (read back from its edit control), a local test page in Chrome with an ordinary text box, a box with an inline completion and a rich editor (read back through the page title, which the page keeps equal to its text), and, since version 0.5.4, the real address bars of Chrome and Edge with browsing history present so that completions appear. Text is typed at a normal pace and at full speed.

The same workflow builds the installer, installs it silently, checks the Start menu shortcut and the installed files, and uninstalls it, also silently.

Rendering checks

Both applications can draw their control panels to image files from the command line (`--render-panel`). Continuous integration runs these commands so that a layout change that crashes or produces an empty image is caught, and the images are kept for review.

12.5 Manual typing tests

Some failures appear only with a real keyboard, a real application and a real person typing at real speed. The project keeps a written record of each manual test in `docs/TESTING.md`, including the failures. The method is consistent:

- type a fixed set of words that exercise the risky cases (*Hãy*, *Việt*, *tiếng*, *đường*, *người*, *thuở*, *windows*);
- type them at several speeds, from slow (120 ms per key) to faster than people type (8 ms per key), using simulated key presses at the HID level where possible;
- read the text back from the application, not from the screen;
- compare with Dokey's own debug log, which records what was sent.

This method found every delivery failure described in Chapter 8. It is also the most expensive layer, and the one with the most gaps, which the next chapter lists.

12.6 Continuous integration

Three workflows run the tests:

Workflow	Trigger	Runs
`ci.yml`	every push to `main`, every pull request	core tests on `ubuntu-latest`, `macos-15`, `windows-2022`; macOS application build and bundle checks
`windows-app.yml`	changes to the Windows application or the core, and release tags (called by `release.yml`)	builds for x86, x64 and ARM64; unit tests (x64, x86); self-tests; updater check; panel rendering; end-to-end typing; installer install and uninstall
`release.yml`	version tags	core tests on each platform before packaging (Chapter 15)

The core is built with `-Wall -Wextra` on Clang and GCC and `/W4` on MSVC.

12.7 Summary

Dokey's tests are layered by cost. At the bottom, a single program checks 20,654 engine behaviours in 0.18 seconds of run time on my machine, including every syllable with every tone and more than a thousand English words. Above it, platform unit tests check event sequences without an operating system's input stack, self-tests check hooks and taps, and end-to-end scripts type into real applications on virtual machines. At the top, manual typing tests on real hardware find the delivery failures that nothing else can, and every one of them is recorded with its keys and results.

THE EVIDENCE

13

Evaluation and What Is Not Tested

This chapter reports the results of the tests described in Chapter 12, in two parts. The first part is the automated evaluation, which I reran for this book at the commit the book describes. The second part is the record of manual typing tests kept in the Dokey repository, which covers real applications on real hardware. The chapter ends with the list of what has not been tested, which is as important to a reader deciding whether to rely on the software as the list of what has.

13.1 Automated results

Method

I cloned the repository at commit `675038e` (3 November 2026, the commit after the version 0.5.4 release commit; the engine and tests are unchanged between them) and built the two test programs on Windows 11 with Clang 21.1.0 at `-O2`, with `-Wall -Wextra`. The machine is a desktop with an Intel Core i5-4590 processor (four cores, 3.3 GHz, released in 2014). The compiler was the Clang bundled with the `ziglang` 0.16 Python package, invoked as `python -m ziglang c++`; the published working paper reports the same compiler version on the same machine. The test data path was compiled in as CMake would set it. Each program ran once and reported its own counts.

Results

Table 13.1 Automated test results at commit `675038e`, run for this book.

Test group	Size	Result
Hand-written Telex cases	119	all passed
Traditional tone style	4	all passed
VNI cases	13	all passed
Combining encoding	1	passed
Round trip: every syllable typed with every allowed tone	3,964 syllables × tones	0 failures
Editing features (Backspace, reopen, undo, Quick Telex, shortcuts)	27	all passed
English and developer words	1,171 entries: 1,097 kept as typed, 74 expected Vietnamese	0 failures
Vietnamese sentences typed in Telex	30 sentences	0 failures
C API, pass-through, doubled-key edits	8	all passed
Vietnamese engine total	**20,654 checks**	**20,654 passed**
Korean composer (prototype)	53,204 checks, including 20,000 random words	53,204 passed

The compiler emitted three warnings, all in the test file (`-Wcharacter-conversion`, about passing a `char16_t` where a `char32_t` is expected, which is harmless for the ASCII and BMP characters involved). The library itself compiled without warnings.

Comparison with the published paper

The working paper reported 20,635 checks, all passing, for version 0.4.0 at commit `5f84bd5`. The 19 additional checks at commit `675038e` come from cases added with versions 0.5.1 to 0.5.4: fourteen hand-written cases (`ddoox`, `thaayf`, the Telex double-letter rule with `tesst`, `marrketing` and `offfline`, and the English words that must keep their doubles) and the five-step doubled-key edit test for `ddoox`. No earlier case was removed or changed its expected result. The regression corpus is unchanged at 1,171 words and 30 sentences.

13.2 Manual typing results

The project's manual tests are recorded in `docs/TESTING.md` and in the release notes, in Vietnamese. They were carried out on a MacBook with an Intel processor running macOS 14, and, for Windows, on GitHub Actions virtual machines. I summarise them here in English, with the results as recorded. Typing was simulated at the HID level (as if from a keyboard) unless the record says the typing was by hand.

Table 13.2 Manual and end-to-end typing results recorded by the project.

Version	Setting	Result
0.3.0	TextEdit, Terminal, textarea and contenteditable in Edge; 8–120 ms per key	correct (*Hãy Việt tiếng đường người*)
0.3.0	ProseMirror editor in Edge; Edge address bar	failed: *Hayãy*; address bar sometimes *Haãy*
0.3.1	same event sequence with select-and-replace: ProseMirror, textarea, contenteditable, address bar	correct (*Hãy*)
0.3.1	installed build; TextEdit, textarea and contenteditable in Edge; 8–80 ms per key	correct (*Hãy Việt tiếng đường người thuở windows*)
0.3.1	ProseMirror editor in Edge	6 of 7 runs fully correct; the failure was the first word right after page load
0.3.1	Edge address bar	read back once, correct (*Việt Nam*); the automated read-back was not reliable
0.3.1	chat box of an Electron application, typed by hand, no pauses	a long sentence correct, checked against Dokey's log
0.3.2	fast typing with the vowel key still held (*Haxy*, *maf*), same chat box	correct after the free-key fix (*Hãy hãy mà và hãy mà ô kìa…*); before it, *Hy* and *m*
0.4.0	Windows, Notepad, VM: `Haxy xin ⌫f btw ␣`	correct: *Hãy xìn by the way* ␣
0.5.0-beta	installed build, macOS 14; Telex into TextEdit, Edge and Safari at normal speed and 20–25 ms per key, also under full CPU load	26 of 26 sentences correct
0.5.0-beta	VNI in TextEdit and Edge	4 of 4 sentences correct
0.5.0-beta	Telex and VNI typed by hand in an Electron chat application	correct
0.5.3	Windows VM: Notepad, an ordinary Chrome text box, a box with inline completion; normal, full speed and overlapping keys	correct (*thầy đi học bế tôi*)
0.5.4	Windows VM: the real Chrome and Edge address bars with browsing history, a Gmail- or Messenger-style editor	correct (*đỗ*, *thầy*, *đi học*) at normal and full speed
0.5.1	macOS self-update from a 0.4.9 test build to the real 0.5.0 release	worked: download, SHA-256 and signature check, replace, relaunch

The record also includes tests of the Korean prototype in VS Code (an Electron application): correct in the editor at 90, 25 and 10 ms

per key, correct in the chat panel at 90 and 25 ms, and correct in the integrated terminal at 25, 40 and 90 ms, with errors at 10 to 15 ms per key, faster than people type.

13.3 What is not tested

The following gaps are taken from the project's own documentation and from the limitations section of the working paper. They are stated without softening, because they decide where Dokey can be relied on today.

Windows

- **No real Windows PC.** The Windows application has not been run on a physical Windows 10 or 11 computer. All Windows tests ran on GitHub Actions virtual machines running Windows Server 2022 (x64).
- **x86 and ARM64 have never run.** These builds are compiled and their architecture is checked; they have not been executed. x86 on 32-bit Windows and ARM64 devices are untested.
- **Applications.** Typing has not been tested in Word, other Office applications, VS Code or other Electron applications on Windows. Excel and File Explorer, which have inline completions, are untested.
- **User interface.** The tray menu, the switch shortcuts, start with Windows, and the control panel have not been tested by hand.

macOS

- **Chrome** has not been tested directly; Edge, which shares the Chromium engine, has.
- **Older systems and other hardware.** macOS 10.13 to 10.15 and Apple Silicon hardware have not been tested; the application builds for both and ships the Swift runtime needed by macOS before 10.14.4.
- **Slow machines.** The slow-machine profile has not been tested on a slow Mac. Its pauses are initial estimates.

- **0.4 features by hand.** Backspace and Escape through the event tap, Spotlight, Google Docs and the update check had not been typed by hand when the record was last updated. The engine side of these features is covered by the automated tests.
- **Manual features.** Start at login, per-application memory and the switch shortcuts have not been tested by hand.
- **Address bars.** The guard character on macOS (0.5.4) has been tested only on a model of a text field with an inline completion, not on Chrome's real address bar.

Known limitations

These are not gaps in testing but behaviours that are known and documented:

- **Electron terminals.** In the integrated terminal of VS Code and similar applications, Shift+Left Arrow does not select text, so select-and-replace delivery can place marks wrongly there. The Korean prototype switches to Backspace in the VS Code family; the Vietnamese profile does not yet.
- **Combining encoding.** With combining tone marks, Dokey uses Backspace delivery everywhere and cannot resume a word after Backspace.
- **Secure input.** On macOS, password fields and Terminal's Secure Keyboard Entry block every event tap, including Dokey's.
- **Elevated windows.** On Windows, Dokey cannot type into windows running as Administrator unless it runs as Administrator itself.
- **Ambiguous words.** English words whose keys spell a Vietnamese syllable (73 distinct words in the regression corpus, Appendix E) become Vietnamese.

Not evaluated at all

- **No user study.** No group of users has typed with Dokey under observation, and there are no measurements of typing speed or error rates with it.
- **No comparison.** Dokey has not been compared with other Vietnamese input methods in this book or in the working paper.

Any statement that it is better or worse than another input method would be unsupported.

- **Not code-signed.** The releases are signed ad hoc on macOS and not signed on Windows. This affects installation and permissions, not typing, and is planned for version 1.0.

13.4 Summary

The engine passes all 20,654 of its checks at the commit described in this book, including every syllable with every allowed tone and every word of the regression corpus. The manual record shows four delivery failures found and fixed, and correct typing in the macOS applications and Windows test environments that were tried. The gaps are equally clear: the Windows application awaits its first real hardware, several browsers and older macOS versions are untried, and there is no user study or comparison. Chapter 16 returns to these as future work.

THE EVIDENCE

14

Performance

Rebuilding the whole word on every keystroke is, on its face, wasteful. An incremental engine does a constant amount of work per key; Dokey re-parses every key of the word, runs dictionary lookups for every trial mark, and composes the whole word again, only to send nothing for most keys. This chapter measures what that costs, explains where the time goes, and shows why the cost does not matter in practice. It also measures the other quantity that matters for an input method's performance, which is not CPU time at all but the number of synthetic events it sends.

14.1 What to measure

Three quantities describe the engine's cost.

- **Time per key**: how long `dokey_key` takes, on average, for realistic input. This bounds the latency the engine adds to each keystroke.
- **Edit rate**: the fraction of keys for which the engine returns an edit, and the size of those edits. This determines how much work the platform layer has to do and how many synthetic events reach applications.
- **Scaling**: how the time per key grows with the length of the word, since the rebuild re-parses the whole word.

14.2 Method

All measurements were made for this book on the machine described in Chapter 13: an Intel Core i5-4590 desktop (four cores, 3.3 GHz, 2014) running Windows 11, with Clang 21.1.0 at `-O2`, at commit `675038e`. The machine is an ordinary desktop, not a dedicated benchmark host. Some runs were made while other work was using it and some while it was idle; the tables say which.

The main benchmark is the program published with the working paper, `bench.cpp`, reproduced in Appendix F. It builds one input text from the repository's own regression corpus: the 30 Telex sentences of `vietnamese_telex.txt` followed by the 1,171 words of `english.txt`, each followed by a space. That text is 9,389 keys long. The program feeds the whole text through the C API 200 times per run, with Telex, precomposed Unicode, the modern tone style and spelling checks on, and measures the elapsed time with `std::chrono::steady_clock`. It makes seven runs, each with a fresh engine, sorts their per-key times, and reports the minimum, median and maximum. During the first pass of the first run it also counts how many keys returned an edit and how many characters those edits deleted.

The corpus is deliberately mixed. English words exercise the gate and the restoration of typed keys, which is the most expensive path; the Vietnamese sentences exercise marks and tones. The mix is roughly four English keys to one Vietnamese key (7,618 to 1,771), which makes the benchmark pessimistic for a user who types mainly Vietnamese.

A second program, `bench2.cpp` (Appendix F), uses the same timing method to separate the corpus into its two parts, to compare option settings, and to measure scaling with word length.

14.3 Results

Time per key

I ran `bench.cpp` six times on 6 November 2026. The first three invocations ran while other work was using the machine; the last three

ran later the same evening with the machine otherwise idle.

Table 14.1 Time per key for the mixed corpus (`bench.cpp`, seven runs per invocation).

Invocation	Machine	Minimum	Median	Maximum
1	in use	885 ns	919 ns	1,660 ns
2	in use	853 ns	959 ns	1,427 ns
3	in use	790 ns	848 ns	908 ns
4	idle	647 ns	687 ns	928 ns
5	idle	661 ns	719 ns	779 ns
6	idle	664 ns	691 ns	716 ns

On the idle machine the medians lie between 0.69 and 0.72 microseconds per key, the same range the working paper reported for version 0.4.0 on this machine (medians of 803 and 689 nanoseconds, 659 to 900 nanoseconds over 14 runs). Under background load they rose to between 0.85 and 0.96 microseconds. The spread is a reminder that a desktop in ordinary use is a noisy instrument; it does not change any conclusion of this chapter.

Splitting the corpus and varying the options gives the following medians, from three invocations of `bench2.cpp` on the idle machine:

Table 14.2 Median time per key by input and options (`bench2.cpp`, three invocations).

Input and options	Keys	Median per key
Vietnamese sentences only	1,771	628–668 ns
English words only	7,618	659–772 ns
Whole corpus, spelling checks off	9,389	450–479 ns
Whole corpus, combining encoding	9,389	629–779 ns

Vietnamese text is a little cheaper per key than English, because English words that leave the gate are rendered twice at the end of the word (once to decide and once to restore). Turning spelling checks off saves about a third of the time, which is the cost of the dictionary lookups. The combining encoding costs about the same as the default.

Edit rate

The edit statistics do not depend on timing and were identical in every run.

Table 14.3 How often the engine sends an edit, and how large it is.

Input	Keys with an edit	Deletions per edit (mean)
Whole corpus	11.9 %	1.39
Vietnamese sentences only	22.7 %	1.57
English words only	9.4 %	1.28
Whole corpus, spelling checks off	15.7 %	—

In Vietnamese text, fewer than one key in four causes an edit, and an edit deletes about one and a half characters. In English text with spelling checks on, fewer than one key in ten does. Turning spelling checks off *increases* the edit rate, because marks are then applied inside English words and must be drawn.

These numbers matter more than the time per key. Each edit becomes several synthetic events: on macOS one Shift+Left Arrow or Backspace per deleted character plus one or more text events, and on Windows two events per deleted and inserted character. A design that sent the whole word again after every key would multiply the event count several times over, and every event is a chance for the reordering and dropping described in Chapter 8.

14.4 Scaling with word length

Because each key re-parses the whole word, the time per key grows with the length of the word. `bench2.cpp` measures this by typing words of a fixed length, built by repeating the letters of *nghieeng*, 200 times each, followed by a space, and timing every key.

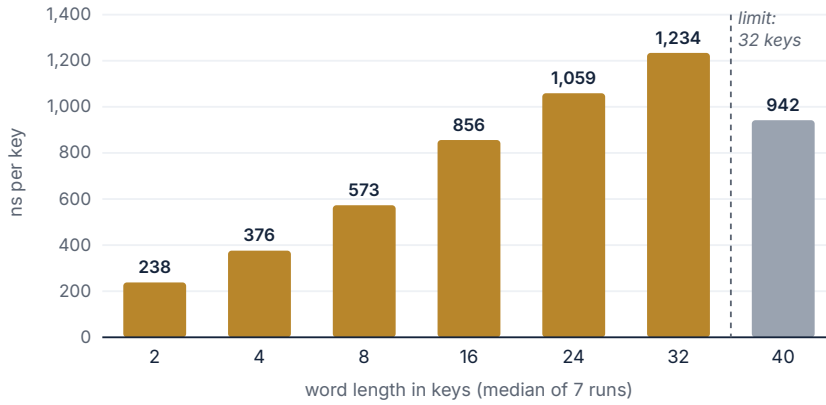


Figure 14.1 Median time per key against word length, for words built from the letters of *nghieeng*. The drop at 40 keys is the 32-key limit: beyond it the engine stops transforming and appends keys without rendering.

Table 14.4 Median time per key by word length (first `bench2.cpp` invocation on the idle machine).

Word length (keys)	Median per key
2	238 ns
4	376 ns
8	573 ns
16	856 ns
24	1,059 ns
32	1,234 ns
40	942 ns

The other two invocations gave 1,228 and 1,308 nanoseconds per key at 32 keys.

From 2 to 32 keys the time per key grows roughly linearly with the length, as expected for a re-parse of the whole word on each key: the total cost of typing a word of n keys grows with n^2. At 40 keys the per-key time falls, because the engine stops transforming a word after 32 keys (Chapter 4); keys 33 to 40 are simply appended, and the average over the word drops.

These synthetic words are a stress test. Real Vietnamese syllables are short: the longest in the dictionary has seven letters, and in

canonical Telex it is typed with nine keys (`nghieengf`). English words in the corpus average about 5.5 letters. At those lengths the engine spends well under a microsecond per key.

14.5 Where the time goes

The cost per key has three parts, in decreasing order of size for typical input:

1. **Dictionary lookups.** Each trial mark, each tone and each rendering calls `plausible`, which converts the letters to a string and looks it up in a hash set; when the exact prefix is not found and marks may still be added, `isCompatibleSyllablePrefix` strips the marks and scans the candidates that share the stripped form. The one-third saving with spelling checks off is mostly this.
2. **Parsing.** The parser walks every key of the word, and each mark key scans the vowel group. Each step copies a small vector of letters when it tries a mark.
3. **Composition and diffing.** Composing each letter is a table lookup; diffing two short strings is negligible.

The code makes no attempt to optimise any of these. It allocates short-lived strings and vectors freely and copies the letter vector for every trial. The measurements show that this is the right trade: the code is simple to read and to test, and its cost is far below anything a user can perceive.

14.6 Why the cost does not matter

A very fast typist enters about 150 words per minute, or about 12 to 15 keystrokes per second, one every 65 to 80 milliseconds. The engine's median cost is under one microsecond per key: less than one part in 60,000 of the interval between keys. Even the stress case of a 32-key token, at about 1.2 to 1.3 microseconds per key, is negligible.

The latencies that users can feel come from elsewhere: from the pauses that some delivery profiles insert (milliseconds, Chapter 8),

from the application's handling of synthetic events, and from the operating system's event pipeline. The engine is not on that list.

The dictionary is built once per process. In two timed runs, building it took 11.3 and 3.9 milliseconds, paid once, on the first key after Dokey starts. It holds about 4,400 short prefix strings and 3,957 syllables in hash sets; I did not measure its memory footprint.

14.7 Summary

On a 2014 desktop processor, Dokey's engine takes a median of 0.69 to 0.72 microseconds per key on a mixed English and Vietnamese corpus when the machine is idle, and 0.63 to 0.67 microseconds on Vietnamese text alone. The cost per key grows linearly with word length and is capped by the 32-key limit. More important than the time is the edit rate: 88 per cent of keys in the mixed corpus, and 77 per cent in Vietnamese text, cause no synthetic events at all, and the edits that are sent delete about one and a half characters. The rebuild design costs nothing that a user can notice and keeps the platform's work to a minimum.

THE EVIDENCE

15

Building and Packaging

This chapter explains how Dokey is built from source, how each platform's application is packaged, and how a release is produced and delivered to users, including the self-update mechanism. It is written for readers who want to build the software themselves and for readers interested in how a one-person project ships native applications for two operating systems from a single repository.

15.1 Repository layout

```
Dokey/
├── include/dokey/   public API: dokey.h (C), engine.hpp (C++),
hangul.hpp
├── src/             engine, Vietnamese tables, Hangul composer, C API
├── tools/           dokey command-line tool
├── tests/           engine and Hangul tests; data/ holds the corpora
├── apps/
│   ├── macos/       Swift + AppKit application, build.sh, install.sh
│   └── windows/     Win32 application, CMake project, Inno Setup script
├── docs/            test record, design notes, images
├── site/            project web page generator
└── .github/         workflows and release notes
```

The core library has no dependencies beyond a C++17 compiler and its standard library. Each application depends only on its operating system's SDK.

15.2 Building the core

The core uses CMake 3.16 or later:

```
cmake -S . -B build -DCMAKE_BUILD_TYPE=Release
cmake --build build --config Release
ctest --test-dir build -C Release
```

This builds the static library `dokey_core`, the command-line tool `dokey`, and the two test programs, and runs the tests. On MSVC the sources are compiled with `/utf-8 /W4`, because the source files contain Vietnamese text in UTF-8; elsewhere with `-Wall -Wextra`. `cmake --install` installs the library, the tool, the headers and the documentation into the usual directories.

The library is small enough to build without CMake. The measurements in this book were made by compiling the four source files directly:

```
c++ -std=c++17 -O2 -Iinclude src/engine.cpp src/vietnamese.cpp \
    src/hangul.cpp src/c_api.cpp tests/engine_tests.cpp \
    -DDOKEY_TEST_DATA="\"$PWD/tests/data\"" -o dokey_tests
```

15.3 Building the macOS application

The macOS application builds with the Command Line Tools alone; Xcode is not required. A single script does everything:

```
apps/macos/build.sh          # → apps/macos/build/Dokey.app
apps/macos/build.sh --dmg    # also a disk image
```

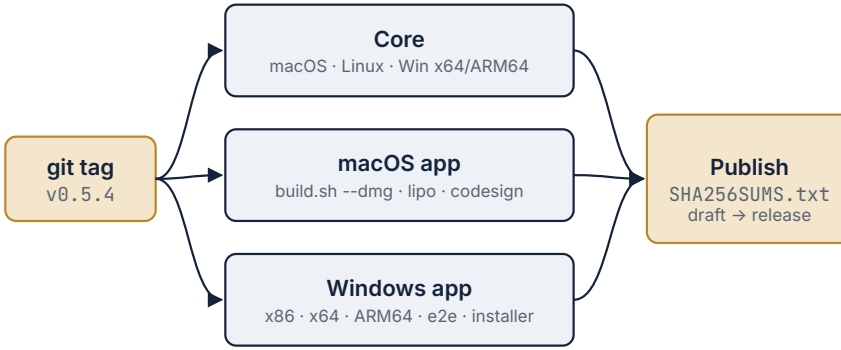


Figure 15.1 From source to release. The core and both applications are built and tested on GitHub Actions; a version tag publishes all artefacts with a checksum file.

The script performs these steps:

1. **Core, per architecture.** For each architecture (by default `x86_64` and `arm64`), compile the core sources with `clang++` for the right minimum system (macOS 10.13 for Intel, macOS 11 for Apple Silicon) and archive them into a static library with `libtool`.
2. **Application, per architecture.** Compile all Swift sources with `swiftc` in whole-module optimisation mode, with a bridging header that imports `dokey.h`, and link against that architecture's core library and `libc++`.
3. **Universal binary.** Merge the two executables with `lipo`.
4. **Bundle.** Write `Info.plist` with the version and a build number (the number of commits), and copy the licence.
5. **Icon.** Compile and run a small Swift program that draws the application icon at every required size, and convert the result with `iconutil`.
6. **Swift runtime for old systems.** macOS before 10.14.4 has no Swift runtime of its own. For the Intel slice, the script copies the back-deployment runtime from the toolchain into the bundle's `Frameworks` directory; newer systems ignore it.
7. **Sign.** Sign the bundle ad hoc (`codesign --sign -`) and verify the signature.
8. **Self-test.** Run `Dokey --self-test` (Chapter 12).

9. **Disk image.** With `--dmg`, create a compressed HFS+ image containing the application, the licence and a link to `/Applications`.

The script also contains a workaround for a defect in some Command Line Tools releases, which ship two module maps that both define the same Swift module and break `import Cocoa`. Instead of editing system files, it hides the duplicate through a virtual file-system overlay passed to the compiler.

Installing without a warning

An application downloaded through a browser is marked with a quarantine attribute, and because Dokey is not notarised by Apple, opening it shows an "unidentified developer" warning. The project ships an install script, published with each release as `install-macos.sh`, which downloads or takes the disk image, copies the application into `/Applications`, removes the quarantine attribute and opens it. The record in Chapter 13 includes a test of this script with a quarantined disk image.

Each release has a new ad-hoc signature, so macOS treats it as a new application and the user must grant Accessibility permission again after every update. Code signing with a developer certificate, which would keep the permission across updates, is planned for version 1.0.

15.4 Building the Windows application

The Windows application is a separate CMake project in `apps/windows`, which includes the core:

```
cmake -S apps/windows -B build/x64 -A x64
cmake --build build/x64 --config Release
ctest --test-dir build/x64 -C Release
```

Replacing `-A x64` with `-A Win32` or `-A ARM64` builds the other architectures. The build uses Visual Studio 2022 with the static C runtime, so the executable needs no Visual C++ Redistributable. The application's icon is drawn by a Python script; the tray icon and the control panel are drawn at run time with GDI+, scaled to the display's DPI.

The installer is built with Inno Setup from `apps/windows/installer/Dokey.iss`, given the version and a directory with the three builds. One installer serves all machines: it detects the processor and installs the x86, x64 or ARM64 executable. By default it installs for the current user without administrator rights, with an option to install for all users; it creates a Start menu entry, optionally starts Dokey with Windows, and registers an uninstaller.

15.5 Continuous integration and release

Releases are produced by GitHub Actions, never on a developer's machine. Pushing a tag such as `v0.5.4` starts `release.yml`, which runs these jobs:

1. **Create** a draft release, with the release notes from `.github/release-notes/<tag>.md`.
2. **Build the core** for macOS (universal, deployment target 10.13), Linux x64, Windows x64 and Windows ARM64. Each job runs the core tests (except ARM64, which cannot run on the x64 runner), installs into a package directory and uploads a `.tar.gz` or `.zip`.
3. **Build the macOS application** with `build.sh --dmg`, verify the universal binary and the disk image, and upload the image and the install script.
4. **Build the Windows application** by calling `windows-app.yml` with the tag: all three architectures, unit tests, self-tests, end-to-end typing, the installer, and a silent install and uninstall; then upload the installer.
5. **Publish**: download every artefact, compute `SHA256SUMS.txt`, upload it, and turn the draft into a published release.

Every artefact a user downloads has therefore been built by the same workflow from the tagged commit, and its checksum is published alongside it.

15.6 Self-update

From version 0.5.1 on macOS and 0.5.2 on Windows, Dokey updates itself. Both implementations follow the same pattern.

1. **Check.** Shortly after launch and then every few hours (on Windows, about 20 seconds after launch and at most twice a day), request the latest release from GitHub's public API, `https://api.github.com/repos/dhtoan/Dokey/releases/latest`. This is the only network connection Dokey makes, and the user can turn it off.
2. **Notify.** If the release is newer and the user has not chosen to skip it, show a system notification and a menu item. The dialog shows the release notes.
3. **Download** the disk image (macOS) or installer (Windows) for that release.
4. **Verify.** Compare the file's SHA-256 digest with the digest GitHub reports for the release asset. On macOS, also mount the image read-only, copy the application out, and verify its code signature with `codesign --verify --deep --strict`. On Windows, also check that the installer's file version matches the release's version.
5. **Install and relaunch.** On macOS, replace the application in place, keeping the old copy until the new one is in position, and relaunch. On Windows, run the installer over the existing installation and restart.

If any step fails, the running version is left untouched. The project record includes a test of the macOS path: a test build numbered 0.4.9 found the real 0.5.0 release, showed its notes, downloaded it, verified it, replaced itself and relaunched.

15.7 Versioning

Versions follow the pattern *major.minor.patch*. The core and both applications share one version number, set in `CMakeLists.txt` and `build.sh` and passed to the Windows build from the tag. A release may change only one platform; its notes then say which. The re-

lease notes, written in Vietnamese for the project's users, are kept in the repository so that each tag's notes are versioned with its code.

15.8 Summary

Dokey builds from one repository with standard tools: CMake for the core and the Windows application, a shell script and the Command Line Tools for the macOS application, and Inno Setup for the Windows installer. All releases are built, tested and checksummed by GitHub Actions from a tag. The applications update themselves from GitHub Releases after verifying a SHA-256 digest and, on macOS, a code signature, and leave the running version untouched if anything fails.

THE EVIDENCE

16

Lessons and Future Work

The preceding chapters described what Dokey does and how. This last chapter steps back to draw out the lessons that seem likely to apply beyond this one program, and to set out the work that remains. It also records, briefly, how the project was carried out, since the pace of its development is unusual and is visible in its history.

16.1 Lessons

Make the core a function

The most consequential decision in Dokey was to make the engine's output a pure function of the raw keys. It cost a re-parse per key, which Chapter 14 showed to be under a microsecond. In return it removed the class of bugs in which an input method's internal model drifts from the screen, it made tone placement a single function instead of a rule that every edit must remember, and it made the engine testable by tables of inputs and outputs. When the platform layers later ran into delivery failures, the doubled-key test of Chapter 12 could show in a few lines that the engine's edits were right and the fault lay elsewhere.

The general lesson is that when a system's state is a small summary of a short history, keeping the history and recomputing the summary is often cheaper, in total, than maintaining the summary. The history of a word being typed is a few keys.

Validate by what is possible, not by what is listed

The spelling gate does not know English. It knows which strings can begin a Vietnamese syllable, a set generated from 29 initials and 156 rhymes under a handful of constraints. That small, rule-derived set protects every word that cannot be Vietnamese, including names and identifiers that no list would contain. Its failures are also exactly predictable: the words whose keys spell a syllable. The regression corpus names 73 of them, and the test fails if that list changes.

A list of exceptions grows without end and fails silently on whatever it omits. A generated set of possibilities is finite, can be checked exhaustively (the round trip types every member), and fails only where the input is truly ambiguous.

Separate deciding from delivering

The engine decides what the text should be; the platform decides how to get it there. The boundary between them is a single small structure: delete *n*, insert this. Each side can be tested against a trivial model of the other. The engine's tests apply edits to a buffer; the Windows injection tests turn edits into event arrays; neither needs an operating system. And when a platform needed a new delivery strategy (select-and-replace, a guard character, a free key code), the engine did not change.

Send as little as possible

Most of the delivery failures in Chapter 8 have one thing in common: they happen to synthetic events. The best defence is not to send them. Because the engine diffs old and new words and reports *not handled* when a key only inserts itself, 88 per cent of keys in the mixed corpus send nothing, and the edits that remain are short. Every failure mode is less likely when there are fewer events for it to act on.

Test where the failures are

The engine's tests found few bugs after the first version; the manual typing tests found every delivery failure. That is not an argu-

ment against automated tests, which keep the engine's behaviour fixed while it changes, but it is a reminder that the hardest part of an input method is not the language. It is the path from the input method to the application, which varies by application, by operating system version, by machine speed and by the speed of the person typing. The project's written record of each manual test, with its keys, its results and its failures, is what made those failures fixable. Writing down what failed, and keeping it in the repository next to the code, cost little and repaid itself every time.

Say what is not tested

Chapter 13 lists the gaps plainly: no real Windows PC, no Chrome on macOS, no older macOS, no user study, no comparison. A user deciding whether to rely on Dokey needs that list more than any other part of the evaluation. Publishing it costs nothing in credibility; omitting it would have cost a great deal the first time a user found a gap.

16.2 How the project was built

Dokey's history is short. The first commit is dated 29 October 2026 (Indochina Time) and contained the engine, the specification, the tests and continuous integration. The tag for version 0.3.0, the first release with both the macOS and the Windows application, is dated the same day. Version 0.4.0, with the editing features of Chapter 7 and the regression corpus, was released on 1 November; the Korean prototype and self-update came on 3 November with versions 0.5.0 to 0.5.4. The repository holds 82 commits up to the commit this book describes.

That pace was possible for two reasons. The first is the design: a pure engine with a written specification and a large automatic test suite can be changed quickly without fear, because every change is checked against tens of thousands of cases in under a second. The second is that the work was done with an AI coding assistant, Claude, which wrote and revised much of the code and the tests under the author's direction, while the author set the design, re-

viewed the changes, and carried out the tests on real machines. The project's history should be read with both in mind.

16.3 Future work

The project's roadmap and limitations point to the following work, roughly in order of importance.

Test on real Windows hardware. The Windows application must be run on physical Windows 10 and 11 computers, on x86 and ARM64 devices, and typed into Chrome, Edge, Office and Electron applications. Until then, its delivery pauses are estimates and its behaviour outside the tested applications is unknown.

Broaden the macOS matrix. Chrome, Apple Silicon hardware, macOS 10.13 to 10.15, and a slow machine for the slow-machine profile. Each is a short session of the manual method in Chapter 12.

Fix delivery in Electron terminals. In the integrated terminal of VS Code and similar applications, select-and-replace does not work. The Korean prototype already switches to Backspace for the VS Code family; the Vietnamese profile should do the same, ideally by detecting the terminal surface rather than the application.

Legacy encodings. TCVN3, VNI Windows and CP1258 are still found in older documents and some government systems. They are listed on the roadmap together with an encoding converter. In the rebuild design they are a change to the composition step of Chapter 6 only.

Code signing and notarisation. Signing with an Apple developer certificate would remove the installation warning, keep Accessibility permission across updates, and open a path to the Mac App Store. A Windows code-signing certificate would remove the SmartScreen warning. Both are planned for version 1.0.

A golden test for refactoring. The design notes for the multi-language engine (Appendix G) propose splitting the engine into a shared core and per-language rule packs. They also propose the safeguard: record every edit the current engine produces for about

a million random key sequences under every combination of options, and require that the refactored engine reproduce them exactly. That tool has not been written.

A user study and a comparison. An honest measure of Dokey's value to users requires people typing real text, with error rates and speeds recorded, and a comparison with the input methods they use now. That is a different kind of work from everything in this book, and it has not been started.

16.4 Closing

An input method is a small program that sits in the most personal place on a computer: between a person and every word they write. It earns its place by being invisible. Dokey tries to be invisible in two ways: by never showing a word that the keys did not mean, and by never letting its corrections arrive damaged. The first is a matter of rules, and the rules fit in a few hundred lines. The second is a matter of patience with every application in which people type, and that work continues.



REFERENCE

Appendices

Appendices A–G

APPENDIX

A

Rule Tables

This appendix collects, in one place, the rules that the engine and the platform layers follow. It is a reference to Chapters 3 to 10; each table names the code that implements it.

A.1 Telex key actions

Table A.1 Telex actions (`Parser::feed` in `src/engine.cpp`).

Key	Class	Action	Succeeds when
`s`	tone	acute tone	tone class not undone; word has a vowel; result plausible
`f`	tone	grave tone	as above; not a checked syllable
`r`	tone	hook tone	as above; not a checked syllable
`x`	tone	tilde tone	as above; not a checked syllable
`j`	tone	dot-below tone	as for `s`
`z`	tone	remove the tone	the word has a tone and the class is not undone
`a`	circumflex *a*	*a* → *â* on the last eligible *a* of the vowel group	the trial word is an exact syllable prefix
`e`	circumflex *e*	*e* → *ê*	as above
`o`	circumflex *o*	*o* → *ô*	as above
`w`	horn and breve	*uo* → *ươ*; else last *o*/*u* → *ơ*/*ư*, or *a* → *ă*; with no vowel group, the letter *ư*	the trial word is an exact syllable prefix
`d`	stroke	first letter *d* → *đ*	the first letter is *d*; the trial word is an exact prefix
other letters	—	append the letter	always

A.2 VNI key actions

Table A.2 VNI actions.

Key	Class	Action
`1 2 3 4 5`	tone	acute, grave, hook, tilde, dot below
`0`	tone	remove the tone
`6`	circumflex	last eligible *a*, *e* or *o* → *â*, *ê*, *ô*
`7`	horn	*uo* → *ươ*; else last *o*/*u* → *ơ*/*ư*
`8`	breve	last *a* → *ă*
`9`	stroke	first letter *d* → *đ*
letters	—	append the letter

A digit at the start of a word in VNI begins a number and is not transformed.

A.3 Undo by repetition

Table A.3 What repeating a key does.

Situation	Result	Example
Same mark key again, immediately after it applied	mark removed, key printed as a letter, class disabled for the rest of the word	`aaa` → *aa*, `ass` → *as*, `ddd` → *dd*, `a11` → *a1*
Lone `w` again	the *ư* becomes the letter *w*	`ww` → *w*
Same tone key later, not immediately	key printed as a letter	
Different tone key	tone replaced	`asf` → *à*
After the class was undone	key always printed as a letter	—

A.4 Tone placement

Table A.4 Tone placement (`tonePosition`), first matching rule wins.

Rule	Condition	Tone on	Examples
1	a vowel in the group has a diacritic	the last such vowel	*tiếng*, *người*, *được*
2	a final follows the vowel group	the last vowel of the group	*toán*, *hoàn*, *huýt*
3	three vowels, open syllable	the middle vowel	*ngoài*, *xoáy*
4a	two vowels *oa*, *oe*, *uy*, modern style	the second vowel	*hoà*, *khoẻ*, *thuý*
4b	two vowels, otherwise	the first vowel	*mía*, *mùa*, *hòa* (traditional)
5	one vowel	that vowel	*má*, *gì*

The vowel group excludes the *u* of *qu* and the *i* of *gi* when another vowel follows.

A.5 End-of-word rules

Table A.5 Rules applied when a word ends (`Engine::render` with `wordEnded`, `Engine::key`).

Rule	Condition	Effect	Example
Text shortcut	typed keys match a shortcut	expansion plus the break character	`btw` ␣ → *by the way* ␣
Not a syllable	spelling on; word has a vowel; not a complete syllable	keys as typed	`use` ␣ → *use* ␣
ươ final	the word ends in *ươ*	*uơ*	`thuowr` ␣ → *thuở* ␣
English double	≥ 4 keys; a tone undone by repetition; result not Vietnamese; double ends the word or precedes a vowel, *w* or *y*	all keys as typed	`pass` ␣ → *pass* ␣
Telex double	same, but the double precedes a consonant	letter once	`tesst` ␣ → *test* ␣

A.6 Word-break characters

Every character that is not a word key ends the current word. Word keys are `a–z` and `A–Z` in Telex, and also `0–9` in VNI. Breaks therefore include space, Enter (`\r`), Tab (`\t`), all punctuation, Telex digits, and every non-ASCII character.

A.7 Quick Telex

Table A.6 Quick Telex expansions (option, off by default; first two keys of a word only).

`cc`	`gg`	`kk`	`nn`	`pp`	`qq`	`tt`
ch	*gi*	*kh*	*ng*	*ph*	*qu*	*th*

A.8 Reset triggers

Table A.7 Events that start a new word, by platform.

Event	macOS	Windows
Mouse button press	yes	yes
Arrow, Home, End, Page Up, Page Down	yes	yes
Forward Delete	yes	yes
Escape (without undo)	yes	yes
Shortcut with ⌘ ⌃ ⌥ / Ctrl Alt Win	yes	yes
Key auto-repeat	yes	—
Change of application	yes	yes (foreground)
Change of focus window	—	yes
Vietnamese/English switch	yes	yes
Text injected by another program (`VK_PACKET`)	—	yes
Event tap disabled by the system	yes	—

A.9 Delivery profiles

Table A.8 Delivery by class (Dokey 0.5.4). Pauses in milliseconds; "slow" is the slow-machine setting.

Class	Platform	Remove old letters	Guard	Text sent as	Pauses
Browser	macOS	Shift+←	U+00B7	≤ 20 units per event	none (2 before text when slow)
Electron/IDE	macOS	Shift+←	—	≤ 20 units per event	none (2 before text when slow)
Normal	macOS	Backspace	—	≤ 20 units per event	none
Google Docs family	macOS	Shift+←	—	1 unit per event	2 per key, 6 before text (4 and 10 slow)
Chromium window	Windows	Backspace	U+00B7	two `SendInput` calls	12 before text
Other window	Windows	Backspace	U+00B7	one `SendInput` call	none
Google Docs family	Windows	Backspace	U+00B7	one key per call	2 per key, 6 before text (4 and 12 slow)

APPENDIX

B

Unicode Code Points for Vietnamese Letters

This appendix lists the precomposed code point of every Vietnamese vowel letter with every tone, in lower and upper case, as stored in the tables `kLower` and `kUpper` of `src/vietnamese.cpp`. The tables were extracted from the source by the script `build/gen_appendices.py`, so they match the engine exactly. The consonant *đ* is U+0111 and *Đ* is U+0110.

B.1 Lower case

Table B.1 Lower case vowels by tone (precomposed, NFC).

Letter	no tone	acute (sắc)	grave (huyền)	hook (hỏi)	tilde (ngã)	dot (nặng)
a	a U+0061	á U+00E1	à U+00E0	ả U+1EA3	ã U+00E3	ạ U+1EA1
ă	ă U+0103	ắ U+1EAF	ằ U+1EB1	ẳ U+1EB3	ẵ U+1EB5	ặ U+1EB7
â	â U+00E2	ấ U+1EA5	ầ U+1EA7	ẩ U+1EA9	ẫ U+1EAB	ậ U+1EAD
e	e U+0065	é U+00E9	è U+00E8	ẻ U+1EBB	ẽ U+1EBD	ẹ U+1EB9
ê	ê U+00EA	ế U+1EBF	ề U+1EC1	ể U+1EC3	ễ U+1EC5	ệ U+1EC7
i	i U+0069	í U+00ED	ì U+00EC	ỉ U+1EC9	ĩ U+0129	ị U+1ECB
o	o U+006F	ó U+00F3	ò U+00F2	ỏ U+1ECF	õ U+00F5	ọ U+1ECD
ô	ô U+00F4	ố U+1ED1	ồ U+1ED3	ổ U+1ED5	ỗ U+1ED7	ộ U+1ED9
ơ	ơ U+01A1	ớ U+1EDB	ờ U+1EDD	ở U+1EDF	ỡ U+1EE1	ợ U+1EE3
u	u U+0075	ú U+00FA	ù U+00F9	ủ U+1EE7	ũ U+0169	ụ U+1EE5
ư	ư U+01B0	ứ U+1EE9	ừ U+1EEB	ử U+1EED	ữ U+1EEF	ự U+1EF1
y	y U+0079	ý U+00FD	ỳ U+1EF3	ỷ U+1EF7	ỹ U+1EF9	ỵ U+1EF5

B.2 Upper case

Table B.2 Upper case vowels by tone (precomposed, NFC).

Letter	no tone	acute (sắc)	grave (huyền)	hook (hỏi)	tilde (ngã)	dot (nặng)
A	A U+0041	Á U+00C1	À U+00C0	Ả U+1EA2	Ã U+00C3	Ạ U+1EA0
Ă	Ă U+0102	Ắ U+1EAE	Ằ U+1EB0	Ẳ U+1EB2	Ẵ U+1EB4	Ặ U+1EB6
Â	Â U+00C2	Ấ U+1EA4	Ầ U+1EA6	Ẩ U+1EA8	Ẫ U+1EAA	Ậ U+1EAC
E	E U+0045	É U+00C9	È U+00C8	Ẻ U+1EBA	Ẽ U+1EBC	Ẹ U+1EB8
Ê	Ê U+00CA	Ế U+1EBE	Ề U+1EC0	Ể U+1EC2	Ễ U+1EC4	Ệ U+1EC6
I	I U+0049	Í U+00CD	Ì U+00CC	Ỉ U+1EC8	Ĩ U+0128	Ị U+1ECA
O	O U+004F	Ó U+00D3	Ò U+00D2	Ỏ U+1ECE	Õ U+00D5	Ọ U+1ECC
Ô	Ô U+00D4	Ố U+1ED0	Ồ U+1ED2	Ổ U+1ED4	Ỗ U+1ED6	Ộ U+1ED8
Ơ	Ơ U+01A0	Ớ U+1EDA	Ờ U+1EDC	Ở U+1EDE	Ỡ U+1EE0	Ợ U+1EE2
U	U U+0055	Ú U+00DA	Ù U+00D9	Ủ U+1EE6	Ũ U+0168	Ụ U+1EE4
Ư	Ư U+01AF	Ứ U+1EE8	Ừ U+1EEA	Ử U+1EEC	Ữ U+1EEE	Ự U+1EF0
Y	Y U+0059	Ý U+00DD	Ỳ U+1EF2	Ỷ U+1EF6	Ỹ U+1EF8	Ỵ U+1EF4

B.3 Combining tone marks

Tone	Code point	Name
acute (sắc)	U+0301	COMBINING ACUTE ACCENT
grave (huyền)	U+0300	COMBINING GRAVE ACCENT
hook (hỏi)	U+0309	COMBINING HOOK ABOVE
tilde (ngã)	U+0303	COMBINING TILDE
dot (nặng)	U+0323	COMBINING DOT BELOW

APPENDIX

C

Initials and Rhymes

Dokey Core builds its syllable dictionary from two hand-written lists in `src/vietnamese.cpp`: 29 initials (counting the empty initial) and 156 rhymes, plus 7 rhymes that follow only *qu*. They are reproduced here exactly, in the order of the source. Chapter 2 describes the constraints under which they are combined.

C.1 Initials

Table C.1 The 29 initials.

(none)	*b*	*c*	*ch*	*d*	*đ*	*g*
gh	*gi*	*h*	*k*	*kh*	*l*	*m*
n	*ng*	*ngh*	*nh*	*p*	*ph*	*qu*
r	*s*	*t*	*th*	*tr*	*v*	*x*
kr						

The initial *kr* is used only in Central Highlands place names (*Krông Búk*, *Krông Pắc*).

C.2 Rhymes

Table C.2 The 156 rhymes as written after an ordinary initial.

a	*ac*	*ach*	*ai*	*am*	*an*	*ang*	*anh*
ao	*ap*	*at*	*au*	*ay*	*ăc*	*ăk*	*ăm*
ăn	*ăng*	*ăp*	*ăt*	*âc*	*âm*	*ân*	*âng*
âp	*ât*	*âu*	*ây*	*e*	*ec*	*em*	*en*
eng	*eo*	*ep*	*et*	*ê*	*êch*	*êm*	*ên*
ênh	*êp*	*êt*	*êu*	*i*	*ia*	*ich*	*im*
in	*inh*	*ip*	*it*	*iu*	*iêc*	*iêm*	*iên*
iêng	*iêp*	*iêt*	*iêu*	*o*	*oc*	*oi*	*om*
on	*ong*	*op*	*ot*	*oa*	*oac*	*oach*	*oai*
oam	*oan*	*oang*	*oanh*	*oao*	*oap*	*oat*	*oay*
oăc	*oăm*	*oăn*	*oăng*	*oăt*	*oe*	*oen*	*oeo*
oet	*ooc*	*oong*	*ô*	*ôc*	*ôi*	*ôm*	*ôn*
ông	*ôp*	*ôt*	*ơ*	*ơi*	*ơm*	*ơn*	*ơp*
ơt	*u*	*ua*	*uc*	*uk*	*ui*	*um*	*un*
ung	*up*	*ut*	*uân*	*uâng*	*uât*	*uây*	*uê*
uêch	*uênh*	*uơ*	*uy*	*uych*	*uynh*	*uyp*	*uyt*
uya	*uyu*	*uyên*	*uyêt*	*uôc*	*uôi*	*uôm*	*uôn*
uông	*uôt*	*ư*	*ưa*	*ưc*	*ưi*	*ưm*	*ưn*
ưng	*ưt*	*ưu*	*ươc*	*ươi*	*ươm*	*ươn*	*ương*
ươp	*ươt*	*ươu*	*y*				

The rhymes *ăk* and *uk* are used only in Central Highlands place names (*Đắk Lắk*, *Búk*).

C.3 Rhymes after qu only

quy, *quyên*, *quyêt*, *quynh*, *quych*, *quyt*, *quyp*.

APPENDIX

D

The Complete Syllable Inventory

This appendix lists all 3957 distinct syllables, without tone, that Dokey Core accepts at commit `675038e`. They were dumped from the engine by the program `eval/inventory.cpp` (Appendix F) and grouped here by initial. As Chapter 2 explains, the list contains every string permitted by the spelling rules, not only syllables attested in Vietnamese words. Checked syllables (ending in *c*, *ch*, *p*, *t* or *k*) can take only the acute and dot-below tones; all others can take all six.

D.1 No initial (156)

a	*ăp*	*ê*	*oa*	*oe*	*ôp*
ac	*ăt*	*êch*	*oac*	*oen*	*ôt*
ach	*âc*	*êm*	*oach*	*oeo*	*ơ*
ai	*âm*	*ên*	*oai*	*oet*	*ơi*
am	*ân*	*ênh*	*oam*	*oi*	*ơm*
an	*âng*	*ếp*	*oan*	*om*	*ơn*
ang	*âp*	*ết*	*oang*	*on*	*ợp*
anh	*ât*	*êu*	*oanh*	*ong*	*ợt*
ao	*âu*	*i*	*oao*	*ooc*	*u*
ap	*ây*	*ia*	*oap*	*oong*	*ua*
at	*e*	*ich*	*oat*	*op*	*uân*
au	*ec*	*im*	*oay*	*ot*	*uâng*
ay	*em*	*in*	*oắc*	*ô*	*uât*
ăc	*en*	*inh*	*oắm*	*ôc*	*uây*
ăk	*eng*	*ip*	*oắn*	*ôi*	*uc*
ăm	*eo*	*it*	*oắng*	*ôm*	*uê*
ăn	*ep*	*iu*	*oắt*	*ôn*	*uêch*
ăng	*et*	*o*	*oc*	*ông*	*uênh*

ui	*uôn*	*uych*	*ưa*	*ươm*	*y*
uk	*uông*	*uyên*	*ưc*	*ươn*	*yêc*
um	*uôt*	*uyêt*	*ưi*	*ương*	*yêm*
un	*uơ*	*uynh*	*ưm*	*ươp*	*yên*
ung	*up*	*uyp*	*ưn*	*ươt*	*yêng*
uôc	*ut*	*uyt*	*ưng*	*ươu*	*yêp*
uôi	*uy*	*uyu*	*ươc*	*ưt*	*yêt*
uôm	*uya*	*ư*	*ươi*	*ưu*	*yêu*

D.2 Initial b (156)

ba	*bâu*	*biêt*	*boc*	*bơt*	*buya*
bac	*bây*	*biêu*	*boe*	*bu*	*buych*
bach	*be*	*bim*	*boen*	*bua*	*buyên*
bai	*bec*	*bin*	*boeo*	*buân*	*buyêt*
bam	*bem*	*binh*	*boet*	*buâng*	*buynh*
ban	*ben*	*bip*	*boi*	*buât*	*buyp*
bang	*beng*	*bit*	*bom*	*buây*	*buyt*
banh	*beo*	*biu*	*bon*	*buc*	*buyu*
bao	*bep*	*bo*	*bong*	*buê*	*bư*
bap	*bet*	*boa*	*booc*	*buêch*	*bưa*
bat	*bê*	*boac*	*boong*	*buênh*	*bưc*
bau	*bêch*	*boach*	*bop*	*bui*	*bưi*
bay	*bêm*	*boai*	*bot*	*buk*	*bưm*
băc	*bên*	*boam*	*bô*	*bum*	*bưn*
băk	*bênh*	*boan*	*bôc*	*bun*	*bưng*
băm	*bêp*	*boang*	*bôi*	*bung*	*bươc*
băn	*bêt*	*boanh*	*bôm*	*buôc*	*bươi*
băng	*bêu*	*boao*	*bôn*	*buôi*	*bươm*
băp	*bi*	*boap*	*bông*	*buôm*	*bươn*
băt	*bia*	*boat*	*bôp*	*buôn*	*bương*
bâc	*bich*	*boay*	*bôt*	*buông*	*bươp*
bâm	*biêc*	*boăc*	*bơ*	*buôt*	*bươt*
bân	*biêm*	*boăm*	*bơi*	*buơ*	*bươu*
bâng	*biên*	*boăn*	*bơm*	*bup*	*bưt*
bâp	*biêng*	*boăng*	*bơn*	*but*	*bưu*
bât	*biêp*	*boăt*	*bơp*	*buy*	*by*

D.3 Initial c (123)

ca	*canh*	*căk*	*câm*	*co*	*coang*
cac	*cao*	*căm*	*cân*	*coa*	*coanh*
cach	*cap*	*căn*	*câng*	*coac*	*coao*
cai	*cat*	*căng*	*câp*	*coach*	*coap*
cam	*cau*	*căp*	*cât*	*coai*	*coat*
can	*cay*	*căt*	*câu*	*coam*	*coay*
cang	*căc*	*câc*	*cây*	*coan*	*coăc*

coăm	*coong*	*cơn*	*cuk*	*cuya*	*cứng*
coăn	*cop*	*cợp*	*cum*	*cuych*	*cước*
coăng	*cot*	*cơt*	*cun*	*cuyên*	*cười*
coăt	*cô*	*cu*	*cung*	*cuyêt*	*cươm*
coc	*côc*	*cua*	*cuôc*	*cuynh*	*cươn*
coe	*côi*	*cuân*	*cuôi*	*cuyp*	*cương*
coen	*côm*	*cuâng*	*cuôm*	*cuyt*	*cướp*
coeo	*côn*	*cuât*	*cuôn*	*cuyu*	*cướt*
coet	*công*	*cuây*	*cuông*	*cư*	*cươu*
coi	*côp*	*cuc*	*cuôt*	*cưa*	*cứt*
com	*côt*	*cuê*	*cươ*	*cực*	*cưu*
con	*cơ*	*cuêch*	*cup*	*cưi*	
cong	*cơi*	*cuênh*	*cut*	*cưm*	
cooc	*cơm*	*cui*	*cuy*	*cưn*	

D.4 Initial ch (156)

cha	*châu*	*chiêt*	*choc*	*chơt*	*chuya*
chac	*chây*	*chiêu*	*choe*	*chu*	*chuych*
chach	*che*	*chim*	*choen*	*chua*	*chuyên*
chai	*chec*	*chin*	*choeo*	*chuân*	*chuyêt*
cham	*chem*	*chinh*	*choet*	*chuâng*	*chuynh*
chan	*chen*	*chip*	*choi*	*chuât*	*chuyp*
chang	*cheng*	*chit*	*chom*	*chuây*	*chuyt*
chanh	*cheo*	*chiu*	*chon*	*chuc*	*chuyu*
chao	*chep*	*cho*	*chong*	*chuê*	*chư*
chap	*chet*	*choa*	*chooc*	*chuêch*	*chưa*
chat	*chê*	*choac*	*choong*	*chuênh*	*chưc*
chau	*chêch*	*choach*	*chop*	*chui*	*chưi*
chay	*chêm*	*choai*	*chot*	*chuk*	*chưm*
chăc	*chên*	*choam*	*chô*	*chum*	*chưn*
chăk	*chênh*	*choan*	*chôc*	*chun*	*chưng*
chăm	*chêp*	*choang*	*chôi*	*chung*	*chươc*
chăn	*chêt*	*choanh*	*chôm*	*chuôc*	*chươi*
chăng	*chêu*	*choao*	*chôn*	*chuôi*	*chươm*
chăp	*chi*	*choap*	*chông*	*chuôm*	*chươn*
chăt	*chia*	*choat*	*chôp*	*chuôn*	*chương*
châc	*chich*	*choay*	*chôt*	*chuông*	*chươp*
châm	*chiêc*	*choăc*	*chơ*	*chuôt*	*chươt*
chân	*chiêm*	*choăm*	*chơi*	*chuơ*	*chươu*
châng	*chiên*	*choăn*	*chơm*	*chup*	*chưt*
châp	*chiêng*	*choăng*	*chơn*	*chut*	*chưu*
chât	*chiêp*	*choăt*	*chơp*	*chuy*	*chy*

D.5 Initial d (156)

da	*dac*	*dach*	*dai*	*dam*	*dan*

dang	*den*	*dinh*	*doeo*	*dua*	*duych*
danh	*deng*	*dip*	*doet*	*duân*	*duyên*
dao	*deo*	*dit*	*doi*	*duâng*	*duyêt*
dap	*dep*	*diu*	*dom*	*duât*	*duynh*
dat	*det*	*do*	*don*	*duây*	*duyp*
dau	*dê*	*doa*	*dong*	*duc*	*duyt*
day	*dêch*	*doac*	*dooc*	*duê*	*duyu*
dăc	*dêm*	*doach*	*doong*	*duêch*	*dư*
dăk	*dên*	*doai*	*dop*	*duênh*	*dưa*
dăm	*dênh*	*doam*	*dot*	*dui*	*dưc*
dăn	*dêp*	*doan*	*dô*	*duk*	*dưi*
dăng	*dêt*	*doang*	*dôc*	*dum*	*dưm*
dăp	*dêu*	*doanh*	*dôi*	*dun*	*dưn*
dăt	*di*	*doao*	*dôm*	*dung*	*dưng*
dâc	*dia*	*doap*	*dôn*	*duôc*	*dươc*
dâm	*dich*	*doat*	*dông*	*duôi*	*dươi*
dân	*diêc*	*doay*	*dôp*	*duôm*	*dươm*
dâng	*diêm*	*doăc*	*dôt*	*duôn*	*dươn*
dâp	*diên*	*doăm*	*dơ*	*duông*	*dương*
dât	*diêng*	*doăn*	*dơi*	*duôt*	*dươp*
dâu	*diêp*	*doăng*	*dơm*	*duơ*	*dươt*
dây	*diêt*	*doăt*	*dơn*	*dup*	*dươu*
de	*diêu*	*doc*	*dợp*	*dut*	*dưt*
dec	*dim*	*doe*	*dơt*	*duy*	*dưu*
dem	*din*	*doen*	*du*	*duya*	*dy*

D.6 Initial đ (156)

đa	*đăt*	*đêm*	*đip*	*đoăng*	*đôn*
đac	*đâc*	*đên*	*đit*	*đoăt*	*đông*
đach	*đâm*	*đênh*	*điu*	*đoc*	*đôp*
đai	*đân*	*đêp*	*đo*	*đoe*	*đôt*
đam	*đâng*	*đêt*	*đoa*	*đoen*	*đơ*
đan	*đâp*	*đêu*	*đoac*	*đoeo*	*đơi*
đang	*đât*	*đi*	*đoach*	*đoet*	*đơm*
đanh	*đâu*	*đia*	*đoai*	*đoi*	*đơn*
đao	*đây*	*đich*	*đoam*	*đom*	*đợp*
đap	*đe*	*điêc*	*đoan*	*đon*	*đơt*
đat	*đec*	*điêm*	*đoang*	*đong*	*đu*
đau	*đem*	*điên*	*đoanh*	*đooc*	*đua*
đay	*đen*	*điêng*	*đoao*	*đoong*	*đuân*
đăc	*đeng*	*điêp*	*đoap*	*đop*	*đuâng*
đăk	*đeo*	*điêt*	*đoat*	*đot*	*đuât*
đăm	*đep*	*điêu*	*đoay*	*đô*	*đuây*
đăn	*đet*	*đim*	*đoăc*	*đôc*	*đuc*
đăng	*đê*	*đin*	*đoăm*	*đôi*	*đuê*
đăp	*đêch*	*đinh*	*đoăn*	*đôm*	*đuêch*

đuênh đuôi đut đuyp đứm đương
đui đuôm đuy đuyt đứn đươp
đuk đuôn đuya đuyu đứng đươt
đum đuông đuych đư đước đươu
đun đuôt đuyên đưa đươi đứt
đung đươ đuyêt đưc đươm đưu
đuôc đup đuynh đưi đươn đy

D.7 Initial g (123)

ga gâm goăm gôn guk guyu
gac gân goăn gông gum gư
gach gâng goăng gôp gun gưa
gai gâp goăt gôt gung gưc
gam gât goc gơ guôc gưi
gan gâu goe gơi guôi gưm
gang gây goen gơm guôm gưn
ganh go goeo gơn guôn gưng
gao goa goet gợp guông gươc
gap goac goi gợt guôt gươi
gat goach gom gu guơ gươm
gau goai gon gua gup gươn
gay goam gong guân gut gương
găc goan gooc guâng guy gươp
găk goang goong guât guya gươt
găm goanh gop guây guych gươu
găn goao got guc guyên gưt
găng goap gô guê guyêt gưu
găp goat gôc guêch guynh
găt goay gôi guênh guyp
gâc goăc gôm gui guyt

D.8 Initial gh (32)

ghe ghep ghênh ghich ghiêt ghit
ghec ghet ghêp ghiêc ghiêu ghiu
ghem ghê ghêt ghiêm ghim
ghen ghêch ghêu ghiên ghin
gheng ghêm ghi ghiêng ghinh
gheo ghên ghia ghiêp ghip

D.9 Initial gi (149)

gi giach gian giao giau giăk
gia giai giang giap giay giăm
giac giam gianh giat giăc giăn

giăng giăp giăt giâc giâm giân giâng giâp giât giâu giây gich gie giec giem gien gieng gieo giep giet giê giêc giêch giêm giên giêng giênh giêp giêt giêu gim gin ginh gio gioa gioac gioach gioai gioam gioan gioang gioanh gioao gioap gioat gioay gioăc gioăm gioăn gioăng gioăt gioc gioe gioen gioeo gioet gioi giom gion giong giooc gioong giop giot giô giôc giôi giôm giôn giông giôp giôt giơ giơi giơm giơn giơp giơt gip git giu giua giuân giuâng giuât giuây giuc giuê giuêch giuênh giui giuk gium giun giung giuôc giuôi giuôm giuôn giuông giuôt giuơ giup giut giuy giuya giuych giuyên giuyêt giuynh giuyp giuyt giuyu giư giưa giưc giưi giưm giưn giưng giươc giươi giươm giươn giương giươp giươt giươu giưt giưu giy

D.10 Initial h (156)

ha hac hach hai ham han hang hanh hao hap hat hau hay hăc hăk hăm hăn hăng hăp hăt hâc hâm hân hâng hâp hât hâu hây he hec hem hen heng heo hep het hê hêch hêm hên hênh hêp hêt hêu hi hia hich hiêc hiêm hiên hiêng hiêp hiêt hiêu him hin hinh hip hit hiu ho hoa hoac hoach hoai hoam hoan hoang hoanh hoao hoap hoat hoay hoăc hoăm hoăn hoăng hoăt hoc hoe hoen hoeo hoet hoi hom hon hong hooc hoong hop hot hô hôc hôi hôm hôn hông hôp hôt hơ hơi hơm hơn hợp hơt hu hua huân huâng huât huây huc huê huêch huênh hui huk hum hun hung huôc huôi huôm huôn huông huôt huơ hup hut huy huya huych

huyên	*huyt*	*hức*	*hưng*	*hươn*	*hươu*
huyêt	*huyu*	*hứi*	*hược*	*hương*	*hưt*
huynh	*hư*	*hứm*	*hươi*	*hượp*	*hưu*
huyp	*hứa*	*hứn*	*hươm*	*hượt*	*hy*

D.11 Initial k (33)

ke	*kep*	*kênh*	*kich*	*kiêt*	*kit*
kec	*ket*	*kêp*	*kiêc*	*kiêu*	*kiu*
kem	*kê*	*kêt*	*kiêm*	*kim*	*ky*
ken	*kêch*	*kêu*	*kiên*	*kin*	
keng	*kêm*	*ki*	*kiêng*	*kinh*	
keo	*kên*	*kia*	*kiêp*	*kip*	

D.12 Initial kh (156)

kha	*khâu*	*khiêt*	*khoc*	*khơt*	*khuya*
khac	*khây*	*khiêu*	*khoe*	*khu*	*khuych*
khach	*khe*	*khim*	*khoen*	*khua*	*khuyên*
khai	*khec*	*khin*	*khoeo*	*khuân*	*khuyêt*
kham	*khem*	*khinh*	*khoet*	*khuâng*	*khuynh*
khan	*khen*	*khip*	*khoi*	*khuât*	*khuyp*
khang	*kheng*	*khit*	*khom*	*khuây*	*khuyt*
khanh	*kheo*	*khiu*	*khon*	*khuc*	*khuyu*
khao	*khep*	*kho*	*khong*	*khuê*	*khư*
khap	*khet*	*khoa*	*khooc*	*khuêch*	*khưa*
khat	*khê*	*khoac*	*khoong*	*khuênh*	*khức*
khau	*khêch*	*khoach*	*khop*	*khui*	*khứi*
khay	*khêm*	*khoai*	*khot*	*khuk*	*khứm*
khăc	*khên*	*khoam*	*khô*	*khum*	*khứn*
khăk	*khênh*	*khoan*	*khôc*	*khun*	*khứng*
khăm	*khêp*	*khoang*	*khôi*	*khung*	*khước*
khăn	*khêt*	*khoanh*	*khôm*	*khuôc*	*khươi*
khăng	*khêu*	*khoao*	*khôn*	*khuôi*	*khươm*
khăp	*khi*	*khoap*	*không*	*khuôm*	*khươn*
khăt	*khia*	*khoat*	*khôp*	*khuôn*	*khương*
khâc	*khich*	*khoay*	*khôt*	*khuông*	*khượp*
khâm	*khiêc*	*khoăc*	*khơ*	*khuôt*	*khượt*
khân	*khiêm*	*khoăm*	*khơi*	*khuơ*	*khươu*
khâng	*khiên*	*khoăn*	*khơm*	*khup*	*khứt*
khâp	*khiêng*	*khoăng*	*khơn*	*khut*	*khưu*
khât	*khiêp*	*khoăt*	*khợp*	*khuy*	*khy*

D.13 Initial l (156)

la	*lac*	*lach*	*lai*	*lam*	*lan*

lang	*len*	*linh*	*loeo*	*lua*	*luych*
lanh	*leng*	*lip*	*loet*	*luân*	*luyên*
lao	*leo*	*lit*	*loi*	*luâng*	*luyêt*
lap	*lep*	*liu*	*lom*	*luât*	*luynh*
lat	*let*	*lo*	*lon*	*luây*	*luyp*
lau	*lê*	*loa*	*long*	*luc*	*luyt*
lay	*lêch*	*loac*	*looc*	*luê*	*luyu*
lăc	*lêm*	*loach*	*loong*	*luêch*	*lư*
lăk	*lên*	*loai*	*lop*	*luênh*	*lưa*
lăm	*lênh*	*loam*	*lot*	*lui*	*lưc*
lăn	*lêp*	*loan*	*lô*	*luk*	*lưi*
lăng	*lêt*	*loang*	*lôc*	*lum*	*lưm*
lăp	*lêu*	*loanh*	*lôi*	*lun*	*lưn*
lăt	*li*	*loao*	*lôm*	*lung*	*lưng*
lâc	*lia*	*loap*	*lôn*	*luôc*	*lươc*
lâm	*lich*	*loat*	*lông*	*luôi*	*lươi*
lân	*liêc*	*loay*	*lôp*	*luôm*	*lươm*
lâng	*liêm*	*loăc*	*lôt*	*luôn*	*lươn*
lâp	*liên*	*loăm*	*lơ*	*luông*	*lương*
lât	*liêng*	*loăn*	*lơi*	*luôt*	*lươp*
lâu	*liêp*	*loăng*	*lơm*	*lươ*	*lươt*
lây	*liêt*	*loăt*	*lơn*	*lup*	*lươu*
le	*liêu*	*loc*	*lợp*	*lut*	*lưt*
lec	*lim*	*loe*	*lơt*	*luy*	*lưu*
lem	*lin*	*loen*	*lu*	*luya*	*ly*

D.14 Initial m (156)

ma	*măt*	*mêm*	*mip*	*moăng*	*môn*
mac	*mâc*	*mên*	*mit*	*moăt*	*mông*
mach	*mâm*	*mênh*	*miu*	*moc*	*môp*
mai	*mân*	*mêp*	*mo*	*moe*	*môt*
mam	*mâng*	*mêt*	*moa*	*moen*	*mơ*
man	*mâp*	*mêu*	*moac*	*moeo*	*mơi*
mang	*mât*	*mi*	*moach*	*moet*	*mơm*
manh	*mâu*	*mia*	*moai*	*moi*	*mơn*
mao	*mây*	*mich*	*moam*	*mom*	*mơp*
map	*me*	*miêc*	*moan*	*mon*	*mơt*
mat	*mec*	*miêm*	*moang*	*mong*	*mu*
mau	*mem*	*miên*	*moanh*	*mooc*	*mua*
may	*men*	*miêng*	*moao*	*moong*	*muân*
măc	*meng*	*miêp*	*moap*	*mop*	*muâng*
măk	*meo*	*miêt*	*moat*	*mot*	*muât*
măm	*mep*	*miêu*	*moay*	*mô*	*muây*
măn	*met*	*mim*	*moăc*	*môc*	*muc*
măng	*mê*	*min*	*moăm*	*môi*	*muê*
măp	*mêch*	*minh*	*moăn*	*môm*	*muêch*

muênh	*muôi*	*mut*	*muyp*	*mứm*	*mương*
mui	*muôm*	*muy*	*muyt*	*mứn*	*mượp*
muk	*muôn*	*muya*	*muyu*	*mứng*	*mượt*
mum	*muông*	*muych*	*mư*	*mước*	*mươu*
mun	*muột*	*muyên*	*mứa*	*mười*	*mứt*
mung	*mươ*	*muyêt*	*mức*	*mươm*	*mứu*
muốc	*mup*	*muynh*	*mứi*	*mươn*	*my*

D.15 Initial n (156)

na	*nâu*	*niêt*	*noc*	*nơt*	*nuya*
nac	*nây*	*niêu*	*noe*	*nu*	*nuych*
nach	*ne*	*nim*	*noen*	*nua*	*nuyên*
nai	*nec*	*nin*	*noeo*	*nuân*	*nuyêt*
nam	*nem*	*ninh*	*noet*	*nuâng*	*nuynh*
nan	*nen*	*nip*	*noi*	*nuât*	*nuyp*
nang	*neng*	*nit*	*nom*	*nuây*	*nuyt*
nanh	*neo*	*niu*	*non*	*nuc*	*nuyu*
nao	*nep*	*no*	*nong*	*nuê*	*nư*
nap	*net*	*noa*	*nooc*	*nuêch*	*nứa*
nat	*nê*	*noac*	*noong*	*nuênh*	*nức*
nau	*nêch*	*noach*	*nop*	*nui*	*nứi*
nay	*nêm*	*noai*	*not*	*nuk*	*nứm*
năc	*nên*	*noam*	*nô*	*num*	*nứn*
năk	*nênh*	*noan*	*nôc*	*nun*	*nứng*
năm	*nêp*	*noang*	*nôi*	*nung*	*nước*
năn	*nêt*	*noanh*	*nôm*	*nuôc*	*nươi*
năng	*nêu*	*noao*	*nôn*	*nuôi*	*nươm*
năp	*ni*	*noap*	*nông*	*nuôm*	*nươn*
năt	*nia*	*noat*	*nôp*	*nuôn*	*nương*
nâc	*nich*	*noay*	*nôt*	*nuông*	*nượp*
nâm	*niêc*	*noăc*	*nơ*	*nuột*	*nượt*
nân	*niêm*	*noăm*	*nơi*	*nuơ*	*nươu*
nâng	*niên*	*noăn*	*nơm*	*nup*	*nứt*
nâp	*niêng*	*noăng*	*nơn*	*nut*	*nứu*
nât	*niêp*	*noăt*	*nợp*	*nuy*	*ny*

D.16 Initial ng (123)

nga	*ngao*	*ngăn*	*ngâp*	*ngoai*	*ngoay*
ngac	*ngap*	*ngăng*	*ngât*	*ngoam*	*ngoăc*
ngach	*ngat*	*ngăp*	*ngâu*	*ngoan*	*ngoăm*
ngai	*ngau*	*ngăt*	*ngây*	*ngoang*	*ngoăn*
ngam	*ngay*	*ngâc*	*ngo*	*ngoanh*	*ngoăng*
ngan	*ngăc*	*ngâm*	*ngoa*	*ngoao*	*ngoăt*
ngang	*ngăk*	*ngân*	*ngoac*	*ngoap*	*ngoc*
nganh	*ngăm*	*ngâng*	*ngoach*	*ngoat*	*ngoe*

ngoen	*ngôi*	*ngua*	*ngung*	*nguyên*	*ngươc*
ngoeo	*ngôm*	*nguân*	*nguôc*	*nguyêt*	*ngươi*
ngoet	*ngôn*	*nguâng*	*nguôi*	*nguynh*	*ngươm*
ngoi	*ngông*	*nguât*	*nguôm*	*nguyp*	*ngươn*
ngom	*ngôp*	*nguây*	*nguôn*	*nguyt*	*ngương*
ngon	*ngôt*	*nguc*	*nguông*	*nguyu*	*ngươp*
ngong	*ngơ*	*nguê*	*nguôt*	*ngư*	*ngươt*
ngooc	*ngơi*	*nguêch*	*nguơ*	*ngưa*	*ngươu*
ngoong	*ngơm*	*nguênh*	*ngup*	*ngưc*	*ngưt*
ngop	*ngơn*	*ngui*	*ngut*	*ngưi*	*ngưu*
ngot	*ngơp*	*nguk*	*nguy*	*ngưm*	
ngô	*ngơt*	*ngum*	*nguya*	*ngưn*	
ngôc	*ngu*	*ngun*	*nguych*	*ngưng*	

D.17 Initial ngh (32)

nghe	*nghep*	*nghênh*	*nghich*	*nghiêt*	*nghit*
nghec	*nghet*	*nghêp*	*nghiêc*	*nghiêu*	*nghiu*
nghem	*nghê*	*nghêt*	*nghiêm*	*nghim*	
nghen	*nghêch*	*nghêu*	*nghiên*	*nghin*	
ngheng	*nghêm*	*nghi*	*nghiêng*	*nghinh*	
ngheo	*nghên*	*nghia*	*nghiêp*	*nghip*	

D.18 Initial nh (156)

nha	*nhân*	*nhi*	*nhoan*	*nhoong*	*nhuây*
nhac	*nhâng*	*nhia*	*nhoang*	*nhop*	*nhuc*
nhach	*nhâp*	*nhich*	*nhoanh*	*nhot*	*nhuê*
nhai	*nhât*	*nhiêc*	*nhoao*	*nhô*	*nhuêch*
nham	*nhâu*	*nhiêm*	*nhoap*	*nhôc*	*nhuênh*
nhan	*nhây*	*nhiên*	*nhoat*	*nhôi*	*nhui*
nhang	*nhe*	*nhiêng*	*nhoay*	*nhôm*	*nhuk*
nhanh	*nhec*	*nhiêp*	*nhoăc*	*nhôn*	*nhum*
nhao	*nhem*	*nhiêt*	*nhoăm*	*nhông*	*nhun*
nhap	*nhen*	*nhiêu*	*nhoăn*	*nhôp*	*nhung*
nhat	*nheng*	*nhim*	*nhoăng*	*nhôt*	*nhuôc*
nhau	*nheo*	*nhin*	*nhoăt*	*nhơ*	*nhuôi*
nhay	*nhep*	*nhinh*	*nhoc*	*nhơi*	*nhuôm*
nhăc	*nhet*	*nhip*	*nhoe*	*nhơm*	*nhuôn*
nhăk	*nhê*	*nhit*	*nhoen*	*nhơn*	*nhuông*
nhăm	*nhêch*	*nhiu*	*nhoeo*	*nhơp*	*nhuôt*
nhăn	*nhêm*	*nho*	*nhoet*	*nhơt*	*nhuơ*
nhăng	*nhên*	*nhoa*	*nhoi*	*nhu*	*nhup*
nhăp	*nhênh*	*nhoac*	*nhom*	*nhua*	*nhut*
nhăt	*nhêp*	*nhoach*	*nhon*	*nhuân*	*nhuy*
nhâc	*nhêt*	*nhoai*	*nhong*	*nhuâng*	*nhuya*
nhâm	*nhêu*	*nhoam*	*nhooc*	*nhuât*	*nhuych*

nhuyên	*nhuyt*	*nhức*	*nhưng*	*nhươn*	*nhươu*
nhuyêt	*nhuyu*	*nhứi*	*nhươc*	*nhương*	*nhưt*
nhuynh	*như*	*nhứm*	*nhươi*	*nhươp*	*nhưu*
nhuyp	*nhưa*	*nhứn*	*nhươm*	*nhươt*	*nhy*

D.19 Initial p (156)

pa	*pâu*	*piêt*	*poc*	*pơt*	*puya*
pac	*pây*	*piêu*	*poe*	*pu*	*puych*
pach	*pe*	*pim*	*poen*	*pua*	*puyên*
pai	*pec*	*pin*	*poeo*	*puân*	*puyêt*
pam	*pem*	*pinh*	*poet*	*puâng*	*puynh*
pan	*pen*	*pip*	*poi*	*puât*	*puyp*
pang	*peng*	*pit*	*pom*	*puây*	*puyt*
panh	*peo*	*piu*	*pon*	*puc*	*puyu*
pao	*pep*	*po*	*pong*	*puê*	*pư*
pap	*pet*	*poa*	*pooc*	*puêch*	*pưa*
pat	*pê*	*poac*	*poong*	*puênh*	*pưc*
pau	*pêch*	*poach*	*pop*	*pui*	*pưi*
pay	*pêm*	*poai*	*pot*	*puk*	*pưm*
păc	*pên*	*poam*	*pô*	*pum*	*pưn*
păk	*pênh*	*poan*	*pôc*	*pun*	*pưng*
păm	*pêp*	*poang*	*pôi*	*pung*	*pươc*
păn	*pêt*	*poanh*	*pôm*	*puôc*	*pươi*
păng	*pêu*	*poao*	*pôn*	*puôi*	*pươm*
păp	*pi*	*poap*	*pông*	*puôm*	*pươn*
păt	*pia*	*poat*	*pôp*	*puôn*	*pương*
pâc	*pich*	*poay*	*pôt*	*puông*	*pươp*
pâm	*piêc*	*poăc*	*pơ*	*puôt*	*pươt*
pân	*piêm*	*poăm*	*pơi*	*puơ*	*pươu*
pâng	*piên*	*poăn*	*pơm*	*pup*	*pưt*
pâp	*piêng*	*poăng*	*pơn*	*put*	*pưu*
pât	*piêp*	*poăt*	*pơp*	*puy*	*py*

D.20 Initial ph (156)

pha	*phau*	*phân*	*pheo*	*phi*	*phin*
phac	*phay*	*phâng*	*phep*	*phia*	*phinh*
phach	*phăc*	*phâp*	*phet*	*phich*	*phip*
phai	*phăk*	*phât*	*phê*	*phiêc*	*phit*
pham	*phăm*	*phâu*	*phêch*	*phiêm*	*phiu*
phan	*phăn*	*phây*	*phêm*	*phiên*	*pho*
phang	*phăng*	*phe*	*phên*	*phiêng*	*phoa*
phanh	*phăp*	*phec*	*phênh*	*phiêp*	*phoac*
phao	*phăt*	*phem*	*phêp*	*phiêt*	*phoach*
phap	*phâc*	*phen*	*phêt*	*phiêu*	*phoai*
phat	*phâm*	*pheng*	*phêu*	*phim*	*phoam*

phoan	*phoeo*	*phông*	*phuc*	*phuơ*	*phưi*
phoang	*phoet*	*phôp*	*phuê*	*phup*	*phưm*
phoanh	*phoi*	*phôt*	*phuêch*	*phut*	*phưn*
phoao	*phom*	*phơ*	*phuênh*	*phuy*	*phưng*
phoap	*phon*	*phơi*	*phui*	*phuya*	*phươc*
phoat	*phong*	*phơm*	*phuk*	*phuych*	*phươi*
phoay	*phooc*	*phơn*	*phum*	*phuyên*	*phươm*
phoắc	*phoong*	*phợp*	*phun*	*phuyêt*	*phươn*
phoắm	*phop*	*phơt*	*phung*	*phuynh*	*phương*
phoắn	*phot*	*phu*	*phuôc*	*phuyp*	*phươp*
phoắng	*phô*	*phua*	*phuôi*	*phuyt*	*phươt*
phoắt	*phôc*	*phuân*	*phuôm*	*phuyu*	*phươu*
phoc	*phôi*	*phuâng*	*phuôn*	*phư*	*phưt*
phoe	*phôm*	*phuât*	*phuông*	*phưa*	*phưu*
phoen	*phôn*	*phuây*	*phuôt*	*phưc*	*phy*

D.21 Initial qu (66)

qua	*quau*	*quân*	*queo*	*qui*	*quơm*
quac	*quay*	*quâng*	*quep*	*quia*	*quơn*
quach	*quắc*	*quâp*	*quet*	*quich*	*quợp*
quai	*quắk*	*quât*	*quê*	*quim*	*quơt*
quam	*quắm*	*quâu*	*quêch*	*quin*	*quy*
quan	*quắn*	*quây*	*quêm*	*quinh*	*quych*
quang	*quắng*	*que*	*quên*	*quip*	*quyên*
quanh	*quắp*	*quec*	*quênh*	*quit*	*quyêt*
quao	*quắt*	*quem*	*quêp*	*quiu*	*quynh*
quap	*quâc*	*quen*	*quêt*	*quơ*	*quyp*
quat	*quâm*	*queng*	*quêu*	*quơi*	*quyt*

D.22 Initial r (156)

ra	*răm*	*rem*	*ria*	*ro*	*roăn*
rac	*răn*	*ren*	*rich*	*roa*	*roăng*
rach	*răng*	*reng*	*riêc*	*roac*	*roăt*
rai	*răp*	*reo*	*riêm*	*roach*	*roc*
ram	*răt*	*rep*	*riên*	*roai*	*roe*
ran	*râc*	*ret*	*riêng*	*roam*	*roen*
rang	*râm*	*rê*	*riêp*	*roan*	*roeo*
ranh	*rân*	*rêch*	*riêt*	*roang*	*roet*
rao	*râng*	*rêm*	*riêu*	*roanh*	*roi*
rap	*râp*	*rên*	*rim*	*roao*	*rom*
rat	*rât*	*rênh*	*rin*	*roap*	*ron*
rau	*râu*	*rêp*	*rinh*	*roat*	*rong*
ray	*rây*	*rêt*	*rip*	*roay*	*rooc*
răc	*re*	*rêu*	*rit*	*roắc*	*roong*
răk	*rec*	*ri*	*riu*	*roắm*	*rop*

rot	*rơm*	*ruê*	*ruôn*	*ruynh*	*rước*
rô	*rơn*	*ruêch*	*ruông*	*ruyp*	*rưới*
rôc	*rợp*	*ruênh*	*ruôt*	*ruyt*	*rươm*
rôi	*rợt*	*rui*	*ruơ*	*ruyu*	*rươn*
rôm	*ru*	*ruk*	*rup*	*rư*	*rương*
rôn	*rua*	*rum*	*rut*	*rưa*	*rượp*
rông	*ruân*	*run*	*ruy*	*rức*	*rượt*
rôp	*ruâng*	*rung*	*ruya*	*rửi*	*rươu*
rôt	*ruât*	*ruôc*	*ruych*	*rứm*	*rứt*
rơ	*ruây*	*ruôi*	*ruyên*	*rứn*	*rứu*
rơi	*ruc*	*ruôm*	*ruyêt*	*rứng*	*ry*

D.23 Initial s (156)

sa	*sâu*	*siêt*	*soc*	*sơt*	*suya*
sac	*sây*	*siêu*	*soe*	*su*	*suych*
sach	*se*	*sim*	*soen*	*sua*	*suyên*
sai	*sec*	*sin*	*soeo*	*suân*	*suyêt*
sam	*sem*	*sinh*	*soet*	*suâng*	*suynh*
san	*sen*	*sip*	*soi*	*suât*	*suyp*
sang	*seng*	*sit*	*som*	*suây*	*suyt*
sanh	*seo*	*siu*	*son*	*suc*	*suyu*
sao	*sep*	*so*	*song*	*suê*	*sư*
sap	*set*	*soa*	*sooc*	*suêch*	*sưa*
sat	*sê*	*soac*	*soong*	*suênh*	*sức*
sau	*sêch*	*soach*	*sop*	*sui*	*sửi*
say	*sêm*	*soai*	*sot*	*suk*	*sứm*
săc	*sên*	*soam*	*sô*	*sum*	*sứn*
săk	*sênh*	*soan*	*sôc*	*sun*	*sứng*
săm	*sêp*	*soang*	*sôi*	*sung*	*sươc*
săn	*sêt*	*soanh*	*sôm*	*suôc*	*sươi*
săng	*sêu*	*soao*	*sôn*	*suôi*	*sươm*
săp	*si*	*soap*	*sông*	*suôm*	*sươn*
săt	*sia*	*soat*	*sôp*	*suôn*	*sương*
sâc	*sich*	*soay*	*sôt*	*suông*	*sượp*
sâm	*siêc*	*soăc*	*sơ*	*suôt*	*sượt*
sân	*siêm*	*soăm*	*sơi*	*suơ*	*sươu*
sâng	*siên*	*soăn*	*sơm*	*sup*	*sứt*
sâp	*siêng*	*soăng*	*sơn*	*sut*	*sứu*
sât	*siêp*	*soăt*	*sợp*	*suy*	*sy*

D.24 Initial t (156)

ta	*tam*	*tao*	*tay*	*tăn*	*tâc*
tac	*tan*	*tap*	*tăc*	*tăng*	*tâm*
tach	*tang*	*tat*	*tăk*	*tăp*	*tân*
tai	*tanh*	*tau*	*tăm*	*tăt*	*tâng*

tập	*tich*	*toanh*	*tot*	*tuê*	*tuynh*
tât	*tiêc*	*toao*	*tô*	*tuêch*	*tuyp*
tâu	*tiêm*	*toap*	*tôc*	*tuênh*	*tuyt*
tây	*tiên*	*toat*	*tôi*	*tui*	*tuyu*
te	*tiêng*	*toay*	*tôm*	*tuk*	*tư*
tec	*tiêp*	*toăc*	*tôn*	*tum*	*tưa*
tem	*tiêt*	*toăm*	*tông*	*tun*	*tức*
ten	*tiêu*	*toăn*	*tôp*	*tung*	*tưi*
teng	*tim*	*toăng*	*tôt*	*tuôc*	*tưm*
teo	*tin*	*toăt*	*tơ*	*tuôi*	*tưn*
tep	*tinh*	*toc*	*tơi*	*tuôm*	*tưng*
tet	*tip*	*toe*	*tơm*	*tuôn*	*tươc*
tê	*tit*	*toen*	*tơn*	*tuông*	*tươi*
têch	*tiu*	*toeo*	*tơp*	*tuôt*	*tươm*
têm	*to*	*toet*	*tơt*	*tươ*	*tươn*
tên	*toa*	*toi*	*tu*	*tup*	*tương*
tênh	*toac*	*tom*	*tua*	*tut*	*tươp*
têp	*toach*	*ton*	*tuân*	*tuy*	*tươt*
têt	*toai*	*tong*	*tuâng*	*tuya*	*tươu*
têu	*toam*	*tooc*	*tuât*	*tuych*	*tưt*
ti	*toan*	*toong*	*tuây*	*tuyên*	*tưu*
tia	*toang*	*top*	*tuc*	*tuyêt*	*ty*

D.25 Initial th (156)

tha	*thân*	*thi*	*thoan*	*thoong*	*thuây*
thac	*thâng*	*thia*	*thoang*	*thop*	*thuc*
thach	*thâp*	*thich*	*thoanh*	*thot*	*thuê*
thai	*thât*	*thiêc*	*thoao*	*thô*	*thuêch*
tham	*thâu*	*thiêm*	*thoap*	*thôc*	*thuênh*
than	*thây*	*thiên*	*thoat*	*thôi*	*thui*
thang	*the*	*thiêng*	*thoay*	*thôm*	*thuk*
thanh	*thec*	*thiêp*	*thoăc*	*thôn*	*thum*
thao	*them*	*thiêt*	*thoăm*	*thông*	*thun*
thap	*then*	*thiêu*	*thoăn*	*thôp*	*thung*
that	*theng*	*thim*	*thoăng*	*thôt*	*thuôc*
thau	*theo*	*thin*	*thoăt*	*thơ*	*thuôi*
thay	*thep*	*thinh*	*thoc*	*thơi*	*thuôm*
thăc	*thet*	*thip*	*thoe*	*thơm*	*thuôn*
thăk	*thê*	*thit*	*thoen*	*thơn*	*thuông*
thăm	*thêch*	*thiu*	*thoeo*	*thơp*	*thuôt*
thăn	*thêm*	*tho*	*thoet*	*thơt*	*thuơ*
thăng	*thên*	*thoa*	*thoi*	*thu*	*thup*
thăp	*thênh*	*thoac*	*thom*	*thua*	*thut*
thăt	*thêp*	*thoach*	*thon*	*thuân*	*thuy*
thâc	*thêt*	*thoai*	*thong*	*thuâng*	*thuya*
thâm	*thêu*	*thoam*	*thooc*	*thuât*	*thuych*

thuyên	*thuyt*	*thức*	*thưng*	*thươn*	*thươu*
thuyêt	*thuyu*	*thưi*	*thươc*	*thương*	*thưt*
thuynh	*thư*	*thưm*	*thươi*	*thươp*	*thưu*
thuyp	*thưa*	*thưn*	*thươm*	*thươt*	*thy*

D.26 Initial tr (156)

tra	*trâu*	*triêt*	*troc*	*trơt*	*truya*
trac	*trây*	*triêu*	*troe*	*tru*	*truych*
trach	*tre*	*trim*	*troen*	*trua*	*truyên*
trai	*trec*	*trin*	*troeo*	*truân*	*truyêt*
tram	*trem*	*trinh*	*troet*	*truâng*	*truynh*
tran	*tren*	*trip*	*troi*	*truât*	*truyp*
trang	*treng*	*trit*	*trom*	*truây*	*truyt*
tranh	*treo*	*triu*	*tron*	*truc*	*truyu*
trao	*trep*	*tro*	*trong*	*truê*	*trư*
trap	*tret*	*troa*	*trooc*	*truêch*	*trưa*
trat	*trê*	*troac*	*troong*	*truênh*	*trưc*
trau	*trêch*	*troach*	*trop*	*trui*	*trưi*
tray	*trêm*	*troai*	*trot*	*truk*	*trưm*
trăc	*trên*	*troam*	*trô*	*trum*	*trưn*
trăk	*trênh*	*troan*	*trôc*	*trun*	*trưng*
trăm	*trêp*	*troang*	*trôi*	*trung*	*trươc*
trăn	*trêt*	*troanh*	*trôm*	*truôc*	*trươi*
trăng	*trêu*	*troao*	*trôn*	*truôi*	*trươm*
trăp	*tri*	*troap*	*trông*	*truôm*	*trươn*
trăt	*tria*	*troat*	*trôp*	*truôn*	*trương*
trâc	*trich*	*troay*	*trôt*	*truông*	*trươp*
trâm	*triêc*	*troăc*	*trơ*	*truôt*	*trươt*
trân	*triêm*	*troăm*	*trơi*	*truơ*	*trươu*
trâng	*triên*	*troăn*	*trơm*	*trup*	*trưt*
trâp	*triêng*	*troăng*	*trơn*	*trut*	*trưu*
trât	*triêp*	*troăt*	*trơp*	*truy*	*try*

D.27 Initial v (156)

va	*vau*	*vân*	*veo*	*vi*	*vin*
vac	*vay*	*vâng*	*vep*	*via*	*vinh*
vach	*văc*	*vâp*	*vet*	*vich*	*vip*
vai	*văk*	*vât*	*vê*	*viêc*	*vit*
vam	*văm*	*vâu*	*vêch*	*viêm*	*viu*
van	*văn*	*vây*	*vêm*	*viên*	*vo*
vang	*văng*	*ve*	*vên*	*viêng*	*voa*
vanh	*văp*	*vec*	*vênh*	*viêp*	*voac*
vao	*văt*	*vem*	*vêp*	*viêt*	*voach*
vap	*vâc*	*ven*	*vêt*	*viêu*	*voai*
vat	*vâm*	*veng*	*vêu*	*vim*	*voam*

voan	*voeo*	*vông*	*vuc*	*vuơ*	*vưi*
voang	*voet*	*vôp*	*vuê*	*vup*	*vưm*
voanh	*voi*	*vôt*	*vuêch*	*vut*	*vưn*
voao	*vom*	*vơ*	*vuênh*	*vuy*	*vưng*
voap	*von*	*vơi*	*vui*	*vuya*	*vươc*
voat	*vong*	*vơm*	*vuk*	*vuych*	*vươi*
voay	*vooc*	*vơn*	*vum*	*vuyên*	*vươm*
voăc	*voong*	*vơp*	*vun*	*vuyêt*	*vươn*
voăm	*vop*	*vơt*	*vung*	*vuynh*	*vương*
voăn	*vot*	*vu*	*vuôc*	*vuyp*	*vươp*
voăng	*vô*	*vua*	*vuôi*	*vuyt*	*vươt*
voăt	*vôc*	*vuân*	*vuôm*	*vuyu*	*vươu*
voc	*vôi*	*vuâng*	*vuôn*	*vư*	*vưt*
voe	*vôm*	*vuât*	*vuông*	*vưa*	*vưu*
voen	*vôn*	*vuây*	*vuôt*	*vưc*	*vy*

D.28 Initial x (156)

xa	*xâu*	*xiêt*	*xoc*	*xơt*	*xuya*
xac	*xây*	*xiêu*	*xoe*	*xu*	*xuych*
xach	*xe*	*xim*	*xoen*	*xua*	*xuyên*
xai	*xec*	*xin*	*xoeo*	*xuân*	*xuyêt*
xam	*xem*	*xinh*	*xoet*	*xuâng*	*xuynh*
xan	*xen*	*xip*	*xoi*	*xuât*	*xuyp*
xang	*xeng*	*xit*	*xom*	*xuây*	*xuyt*
xanh	*xeo*	*xiu*	*xon*	*xuc*	*xuyu*
xao	*xep*	*xo*	*xong*	*xuê*	*xư*
xap	*xet*	*xoa*	*xooc*	*xuêch*	*xưa*
xat	*xê*	*xoac*	*xoong*	*xuênh*	*xưc*
xau	*xêch*	*xoach*	*xop*	*xui*	*xưi*
xay	*xêm*	*xoai*	*xot*	*xuk*	*xưm*
xăc	*xên*	*xoam*	*xô*	*xum*	*xưn*
xăk	*xênh*	*xoan*	*xôc*	*xun*	*xưng*
xăm	*xêp*	*xoang*	*xôi*	*xung*	*xươc*
xăn	*xêt*	*xoanh*	*xôm*	*xuôc*	*xươi*
xăng	*xêu*	*xoao*	*xôn*	*xuôi*	*xươm*
xăp	*xi*	*xoap*	*xông*	*xuôm*	*xươn*
xăt	*xia*	*xoat*	*xôp*	*xuôn*	*xương*
xâc	*xich*	*xoay*	*xôt*	*xuông*	*xươp*
xâm	*xiêc*	*xoăc*	*xơ*	*xuôt*	*xươt*
xân	*xiêm*	*xoăm*	*xơi*	*xuơ*	*xươu*
xâng	*xiên*	*xoăn*	*xơm*	*xup*	*xưt*
xâp	*xiêng*	*xoăng*	*xơn*	*xut*	*xưu*
xât	*xiêp*	*xoăt*	*xơp*	*xuy*	*xy*

D.29 Initial kr (156)

kra	*krâu*	*kriêt*	*kroc*	*krơt*	*kruya*
krac	*krây*	*kriêu*	*kroe*	*kru*	*kruych*
krach	*kre*	*krim*	*kroen*	*krua*	*kruyên*
krai	*krec*	*krin*	*kroeo*	*kruân*	*kruyêt*
kram	*krem*	*krinh*	*kroet*	*kruâng*	*kruynh*
kran	*kren*	*krip*	*kroi*	*kruât*	*kruyp*
krang	*kreng*	*krit*	*krom*	*kruây*	*kruyt*
kranh	*kreo*	*kriu*	*kron*	*kruc*	*kruyu*
krao	*krep*	*kro*	*krong*	*kruê*	*krư*
krap	*kret*	*kroa*	*krooc*	*kruêch*	*krưa*
krat	*krê*	*kroac*	*kroong*	*kruênh*	*krưc*
krau	*krêch*	*kroach*	*krop*	*krui*	*krưi*
kray	*krêm*	*kroai*	*krot*	*kruk*	*krưm*
krăc	*krên*	*kroam*	*krô*	*krum*	*krưn*
krăk	*krênh*	*kroan*	*krôc*	*krun*	*krưng*
krăm	*krêp*	*kroang*	*krôi*	*krung*	*krươc*
krăn	*krêt*	*kroanh*	*krôm*	*kruôc*	*krươi*
krăng	*krêu*	*kroao*	*krôn*	*kruôi*	*krươm*
krăp	*kri*	*kroap*	*krông*	*kruôm*	*krươn*
krăt	*kria*	*kroat*	*krôp*	*kruôn*	*krương*
krâc	*krich*	*kroay*	*krôt*	*kruông*	*krươp*
krâm	*kriêc*	*kroăc*	*krơ*	*kruôt*	*krươt*
krân	*kriêm*	*kroăm*	*krơi*	*kruơ*	*krươu*
krâng	*kriên*	*kroăn*	*krơm*	*krup*	*krưt*
krâp	*kriêng*	*kroăng*	*krơn*	*krut*	*krưu*
krât	*kriêp*	*kroăt*	*krơp*	*kruy*	*kry*

APPENDIX

E

Regression Data

This appendix reproduces two of the data files that the engine tests read, from `tests/data/` at commit `675038e`, and the per-key traces quoted in the book.

E.1 English words that become Vietnamese

The regression corpus contains 1,171 English and developer word entries (1,138 distinct words). With Telex and spelling checks on, 74 entries, 73 distinct words (*list* appears twice), spell a Vietnamese syllable and are converted; the test requires exactly these results (`english_known.txt`). The other 1,097 entries must stay as typed.

Table E.1 The 73 English words whose Telex keys spell a Vietnamese syllable.

Keys	Result	Keys	Result	Keys	Result
of	*ò*	as	*á*	this	*thí*
his	*hí*	her	*hẻ*	or	*ỏ*
there	*thể*	if	*ì*	see	*sê*
now	*nơ*	its	*ít*	how	*hơ*
these	*thế*	most	*mót*	us	*ú*
is	*í*	was	*ứa*	been	*bên*
has	*há*	did	*đi*	does	*doé*
says	*sáy*	goes	*goé*	air	*ải*
arm	*ảm*	bar	*bả*	best	*bét*
born	*bỏn*	box	*bõ*	car	*cả*
chair	*chải*	cost	*cót*	data	*dât*
deep	*dêp*	door	*dổ*	down	*đơn*
gas	*gá*	hair	*hải*	here	*hể*
keep	*kêp*	last	*lát*	list	*lít*
lose	*loé*	low	*lơ*	major	*mảo*
meet	*mêt*	more	*moẻ*	must	*mút*
nor	*nỏ*	own	*ơn*	past	*pát*
per	*pẻ*	piece	*piêc*	poor	*pổ*
rest	*rét*	room	*rôm*	seem	*sêm*
sense	*sến*	six	*sĩ*	soon	*sôn*
tax	*tã*	term	*tẻm*	test	*tét*
those	*thoé*	thus	*thú*	too	*tô*
town	*tơn*	tree	*trê*	turn	*tùn*
war	*ửa*	wrong	*ưởng*	rust	*rút*
var	*vả*				

E.2 Vietnamese sentences

The 30 sentences of `vietnamese_telex.txt` are typed with the tone key at the end of each syllable, as most people type. Each line

shows the keys and the expected text; the test requires every sentence to come out exactly.

```
Xin chaof cacs banj, hoom nay trowif ddepj quas.
```
Xin chào các bạn, hôm nay trời đẹp quá.

```
Tooi yeeu tieengs Vieetj vaf muoons gox ddungs ngay laanf ddaauf.
```
Tôi yêu tiếng Việt và muốn gõ đúng ngay lần đầu.

```
Chungs ta cungf nhau xaay duwngj mootj booj gox thaatj nhanh.
```
Chúng ta cùng nhau xây dựng một bộ gõ thật nhanh.

```
Nguowif Vieetj Nam raats hieeus khachs vaf thaan thieenj.
```
Người Việt Nam rất hiếu khách và thân thiện.

```
Haf Nooij laf thur ddoo cuar nuowcs Coongj hoaf Xax hooij Chur nghiax Vieetj Nam.
```
Hà Nội là thủ đô của nước Cộng hoà Xã hội Chủ nghĩa Việt Nam.

```
Thanhf phoos Hoof Chis Minh ddoong dducs vaf nawng ddoongj.
```
Thành phố Hồ Chí Minh đông đúc và năng động.

```
Ddawks Lawks nooir tieengs vowis caf phee vaf voi.
```
Đắk Lắk nổi tiếng với cà phê và voi.

```
Muaf thu nawm aays, las vangf rowi khawps phoos.
```
Mùa thu năm ấy, lá vàng rơi khắp phố.

```
Camr own banj ddax danhf thowif gian ddocj baif vieets nayf.
```
Cảm ơn bạn đã dành thời gian đọc bài viết này.

```
Chucs muwngf nawm mowis, an khang thinhj vuowngj.
```
Chúc mừng năm mới, an khang thịnh vượng.

```
Hocj, hocj nuwax, hocj maix.
```
Học, học nữa, học mãi.

```
Cos coong maif sawts cos ngayf neen kim.
```
Có công mài sắt có ngày nên kim.

```
Uoongs nuowcs nhows nguoonf.
```
Uống nước nhớ nguồn.

```
Ddi mootj ngayf ddangf hocj mootj sangf khoon.
```
Đi một ngày đàng học một sàng khôn.

```
Baauf owi thuowng laays bis cungf, tuy rawngf khacs gioongs nhuwng chung mootj gianf.
```
Bầu ơi thương lấy bí cùng, tuy rằng khác giống nhưng chung một giàn.

```
Quyeenr sachs nayf raats hay, tooi ddax ddocj nos ba laanf.
```
Quyển sách này rất hay, tôi đã đọc nó ba lần.

```
Khuya rooif, banj neen ddi ngur sowms ddeer giuwx ginf suwcs khoer.
```
Khuya rồi, bạn nên đi ngủ sớm để giữ gìn sức khoẻ.

`Chieecs xe ddapj cux vaanx chayj toots sau hai muowi nawm.`
Chiếc xe đạp cũ vẫn chạy tốt sau hai mươi năm.

`Huyeenf thoaij veef Thanhs Giongs dduowcj truyeenf qua nhieeuf thees heej.`
Huyền thoại về Thánh Gióng được truyền qua nhiều thế hệ.

`Nhuwngx nguowif thowj lamf vieecj chawm chir tuwf sangs towis toois.`
Những người thợ làm việc chăm chỉ từ sáng tới tối.

`Giaos vieen huowngs daanx hocj sinh giair baif toans khos.`
Giáo viên hướng dẫn học sinh giải bài toán khó.

`Phowr bof vaf buns char laf nhuwngx mons awn dduowcj yeeu thichs.`
Phở bò và bún chả là những món ăn được yêu thích.

`Truowngf ddaij hocj toor chuwcs leex toots nghieepj vaof thangs bayr.`
Trường đại học tổ chức lễ tốt nghiệp vào tháng bảy.

`Thuowr nhor, tooi thuowngf theo baf ra ddoongf xem luas.`
Thuở nhỏ, tôi thường theo bà ra đồng xem lúa.

`Ngoaif trowif ddang muwa rar richs suoots car buooir chieeuf.`
Ngoài trời đang mưa rả rích suốt cả buổi chiều.

`Khuyeens khichs moij nguowif tieets kieemj ddieenj vaf nuowcs.`
Khuyến khích mọi người tiết kiệm điện và nước.

`Quar xoaif chins vangf uowm, thowm phuwcs car gian beeps.`
Quả xoài chín vàng ươm, thơm phức cả gian bếp.

`Nghieeng minhf truowcs nhuwngx hy sinh cuar thees heej ddi truowcs.`
Nghiêng mình trước những hy sinh của thế hệ đi trước.

`Gia ddinhf laf nowi binhf yeen nhaats treen ddowif.`
Gia đình là nơi bình yên nhất trên đời.

`Tuyeetj vowif, chungs ta ddax hoanf thanhf duwj ans ddungs hanj.`
Tuyệt vời, chúng ta đã hoàn thành dự án đúng hạn.

E.3 Per-key traces

The traces below were printed by `eval/trace.cpp` (Appendix F). Each row is one key: whether the engine handled it, how many characters the edit deletes, what it inserts, and the text that results. ␣ is Space and ⌫ is Backspace; the traditional tone style is used only for `hoanf⌫`.

tieengs

Key	Handled	Delete	Insert	Screen
t	no	0		*t*
i	no	0		*ti*
e	no	0		*tie*
e	yes	1	*ê*	*tiê*
n	no	0		*tiên*
g	no	0		*tiêng*
s	yes	3	*ếng*	*tiếng*
␣	no	0		*tiếng* ␣

tiengse

Key	Handled	Delete	Insert	Screen
t	no	0		*t*
i	no	0		*ti*
e	no	0		*tie*
n	no	0		*tien*
g	no	0		*tieng*
s	yes	3	*éng*	*tiéng*
e	yes	3	*ếng*	*tiếng*
␣	no	0		*tiếng* ␣

dduowcj

Key	Handled	Delete	Insert	Screen
d	no	0		*d*
d	yes	1	*đ*	*đ*
u	no	0		*đu*
o	no	0		*đuo*
w	yes	2	*ươ*	*đươ*
c	no	0		*đươc*
j	yes	2	*ợc*	*được*
␣	no	0		*được* ␣

dduocjw

Key	Handled	Delete	Insert	Screen
d	no	0		*d*
d	yes	1	*đ*	*đ*
u	no	0		*đu*
o	no	0		*đuo*
c	no	0		*đuoc*
j	yes	2	*ọc*	*đuọc*
w	yes	3	*ược*	*được*
␣	no	0		*được* ␣

windows

Key	Handled	Delete	Insert	Screen
w	yes	0	*ư*	*ư*
i	no	0		*ưi*
n	yes	2	*win*	*win*
d	no	0		*wind*
o	no	0		*windo*
w	no	0		*window*
s	no	0		*windows*
␣	no	0		*windows* ␣

windows (spelling checks off)

Key	Handled	Delete	Insert	Screen
w	yes	0	*ư*	*ư*
i	no	0		*ưi*
n	no	0		*ưin*
d	no	0		*ưind*
o	no	0		*ưindo*
w	no	0		*ưindow*
s	yes	6	*ứindow*	*ứindow*
␣	no	0		*ứindow* ␣

haxy

Key	Handled	Delete	Insert	Screen
h	no	0		*h*
a	no	0		*ha*
x	yes	1	*ã*	*hã*
y	no	0		*hãy*
␣	no	0		*hãy* ␣

thuowr

Key	Handled	Delete	Insert	Screen
t	no	0		*t*
h	no	0		*th*
u	no	0		*thu*
o	no	0		*thuo*
w	yes	2	*ươ*	*thươ*
r	yes	1	*ở*	*thưở*
␣	yes	2	*uở*	*thuở* ␣

pass

Key	Handled	Delete	Insert	Screen
p	no	0		*p*
a	no	0		*pa*
s	yes	1	*á*	*pá*
s	yes	1	*as*	*pas*
␣	yes	0	*s*	*pass* ␣

tesst

Key	Handled	Delete	Insert	Screen
t	no	0		*t*
e	no	0		*te*
s	yes	1	*é*	*té*
s	yes	1	*es*	*tes*

Key	Handled	Delete	Insert	Screen
t	no	0		*test*
␣	no	0		*test* ␣

hoanf ⌫ (traditional style)

Key	Handled	Delete	Insert	Screen
h	no	0		*h*
o	no	0		*ho*
a	no	0		*hoa*
n	no	0		*hoan*
f	yes	2	*àn*	*hoàn*
⌫	yes	3	*òa*	*hòa*
␣	no	0		*hòa* ␣

xin ␣ ⌫ f

Key	Handled	Delete	Insert	Screen
x	no	0		*x*
i	no	0		*xi*
n	no	0		*xin*
␣	no	0		*xin* ␣
⌫	no	0		*xin*
f	yes	2	*ìn*	*xìn*
␣	no	0		*xìn* ␣

tieengs ↻

Key	Handled	Delete	Insert	Screen
t	no	0		*t*
i	no	0		*ti*
e	no	0		*tie*
e	yes	1	*ê*	*tiê*
n	no	0		*tiên*
g	no	0		*tiêng*

Key	Handled	Delete	Insert	Screen
s	yes	3	*ếng*	*tiếng*
⎋	yes	3	*eengs*	*tieengs*
␣	no	0		*tieengs* ␣

d9u7o7ng2 (VNI)

Key	Handled	Delete	Insert	Screen
d	no	0		*d*
9	yes	1	*đ*	*đ*
u	no	0		*đu*
7	yes	1	*ư*	*đư*
o	no	0		*đưo*
7	yes	1	*ơ*	*đươ*
n	no	0		*đươn*
g	no	0		*đương*
2	yes	3	*ờng*	*đường*
␣	no	0		*đường* ␣

use

Key	Handled	Delete	Insert	Screen
u	no	0		*u*
s	yes	1	*ú*	*ú*
e	no	0		*úe*
␣	yes	2	*use*	*use* ␣

nguoiwf

Key	Handled	Delete	Insert	Screen
n	no	0		*n*
g	no	0		*ng*
u	no	0		*ngu*
o	no	0		*nguo*
i	no	0		*nguoi*

Key	Handled	Delete	Insert	Screen
`w`	yes	3	*ươi*	*ngươi*
`f`	yes	2	*ời*	*người*
`␣`	no	0		*người ␣*

APPENDIX

F

Measurement Programs

Every number in this book that was not taken from the Dokey repository or the published working paper was produced by one of the programs in this appendix. Their sources are in the book's `eval/` directory, together with their outputs (`trace.txt`, `syllables.txt`).

F.1 Environment

Item	Value
Source	`https://github.com/dhtoan/Dokey`, commit `675038e` (3 November 2026)
Machine	Intel Core i5-4590, 4 cores, 3.3 GHz; Windows 11 Pro
Compiler	Clang 21.1.0, from the `ziglang` 0.16 Python package (`python -m ziglang c++`)
Flags	`-std=c++17 -O2`
Date	6 November 2026

F.2 Building

Each program is compiled together with the engine sources from the repository root, for example:

```
python -m ziglang c++ -std=c++17 -O2 -Iinclude bench.cpp \
    src/engine.cpp src/vietnamese.cpp src/hangul.cpp src/c_api.cpp -o
bench
./bench tests/data
```

`inventory.cpp` includes the internal header `vietnamese.hpp`, so it is compiled with `-Isrc` as well. The programs were reformatted for print after the measurements; the trace and inventory outputs of the reformatted programs were checked to produce the same output as the originals, and the `bench2.cpp` timings in Chapter 14 come from the reformatted program.

F.3 bench.cpp: time per key on the regression corpus

Published with the working paper. Used for Table 14.1 and the first row of Table 14.3. The edit statistics are identical in every run.

```
// Per-key cost of dokey_key over the repo's own test corpus (Telex
sentences + English words).
#include <dokey/dokey.h>
#include <chrono>
#include <cstdio>
#include <fstream>
#include <string>
#include <vector>
#include <algorithm>
int main(int argc, char** argv) {
    std::string dir = argv[1];
    std::string text;
    std::ifstream vi(dir + "/vietnamese_telex.txt");
    for (std::string l; std::getline(vi, l);) { if (l.empty() || l[0] ==
'#') continue; auto b = l.find(" | "); if (b ≠ std::string::npos) text
+= l.substr(0, b) + " "; }
    std::ifstream en(dir + "/english.txt");
    for (std::string l; std::getline(en, l);) { if (l.empty() || l[0] ==
'#') continue; text += l + " "; }
    dokey_options opt = { DOKEY_TELEX, DOKEY_UNICODE, 1, 1, 0 };
    const int reps = 200;
    std::vector<double> runs;
    long handled = 0, keys = 0, bs = 0;
    for (int r = 0; r < 7; r++) {
        dokey_engine* e = dokey_create(&opt);
        dokey_edit ed;
        auto t0 = std::chrono::steady_clock::now();
        for (int k = 0; k < reps; k++)
            for (unsigned char c : text) { int h = dokey_key(e, c, &ed);
if (r == 0 && k == 0) { keys++; if (h) { handled++; bs += ed.backspaces;
} } }
        auto t1 = std::chrono::steady_clock::now();
        runs.push_back(std::chrono::duration<double, std::nano>(t1 -
t0).count() / (double(reps) * text.size()));
        dokey_destroy(e);
    }
```

```
    std::sort(runs.begin(), runs.end());
    std::printf("keys per pass %zu, passes %d x 7 runs\n", text.size(),
reps);
    std::printf("ns per key: min %.0f median %.0f max %.0f\n",
runs.front(), runs[3], runs.back());
    std::printf("handled (edit sent) %.1f%% of keys, mean backspaces per
edit %.2f\n", 100.0 * handled / keys, handled ? double(bs) / handled :
0.0);
}
```

F.4 bench2.cpp: time per key by input, options and word length

Written for this book. Used for Tables 14.2 to 14.4 and Figure 14.1.

```
// Book measurements: per-key cost of dokey_key by input, options and
word length.
// Same timing method as bench.cpp: 7 runs, each with a fresh engine;
median reported.
#include <dokey/dokey.h>

#include <algorithm>
#include <chrono>
#include <cstdio>
#include <fstream>
#include <string>
#include <vector>

struct Stats {
    long keys = 0, handled = 0, backspaces = 0;
};

// Median time per key over 7 runs of `reps` passes; edit statistics
from the first pass.
static double run(const std::string& text, dokey_options opt, int reps,
Stats* stats) {
    std::vector<double> runs;
    for (int r = 0; r < 7; r++) {
        dokey_engine* e = dokey_create(&opt);
        dokey_edit edit;
        auto t0 = std::chrono::steady_clock::now();
        for (int i = 0; i < reps; i++) {
            for (unsigned char c : text) {
                int h = dokey_key(e, c, &edit);
                if (stats && r == 0 && i == 0) {
                    stats->keys++;
                    if (h) {
                        stats->handled++;
                        stats->backspaces += edit.backspaces;
                    }
```

```
                }
            }
        }
        auto t1 = std::chrono::steady_clock::now();
        runs.push_back(std::chrono::duration<double, std::nano>(t1 -
t0).count() / (double(reps) * text.size()));
        dokey_destroy(e);
    }
    std::sort(runs.begin(), runs.end());
    return runs[3];
}

int main(int argc, char** argv) {
    if (argc < 2) return 2;
    const std::string dir = argv[1];
    std::string vi, en;
    std::ifstream vf(dir + "/vietnamese_telex.txt");
    for (std::string l; std::getline(vf, l);) {
        if (l.empty() || l[0] == '#') continue;
        auto bar = l.find(" | ");
        if (bar != std::string::npos) vi += l.substr(0, bar) + " ";
    }
    std::ifstream ef(dir + "/english.txt");
    for (std::string l; std::getline(ef, l);) {
        if (l.empty() || l[0] == '#') continue;
        en += l + " ";
    }

    const dokey_options base = {DOKEY_TELEX, DOKEY_UNICODE, 1, 1, 0};
    Stats s;
    double t = run(vi, base, 2000, &s);
    std::printf("Vietnamese sentences only: %ld keys, median %.0f
ns/key, edits %.1f%%, bs/edit %.2f\n",
                s.keys, t, 100.0 * s.handled / s.keys,
double(s.backspaces) / s.handled);

    s = Stats();
    t = run(en, base, 200, &s);
    std::printf("English words only: %ld keys, median %.0f ns/key, edits
%.1f%%, bs/edit %.2f\n",
                s.keys, t, 100.0 * s.handled / s.keys, s.handled ?
double(s.backspaces) / s.handled : 0.0);

    dokey_options noSpell = base;
    noSpell.check_spelling = 0;
    s = Stats();
    t = run(vi + en, noSpell, 200, &s);
    std::printf("All, spelling off: median %.0f ns/key, edits %.1f%%\n",
t, 100.0 * s.handled / s.keys);

    dokey_options combining = base;
    combining.encoding = DOKEY_UNICODE_COMBINING;
    t = run(vi + en, combining, 200, nullptr);
```

```
    std::printf("All, combining: median %.0f ns/key\n", t);

    // Word length sweep: words built by repeating the letters of
"nghieeng".
    for (int n : {2, 4, 8, 16, 24, 32, 40}) {
        const char* pattern = "nghieeng";
        std::string word;
        for (int i = 0; i < n; i++) word += pattern[i % 8];
        word += " ";
        std::string text;
        for (int i = 0; i < 200; i++) text += word;
        t = run(text, base, 50, nullptr);
        std::printf("word of %2d keys: median %.0f ns/key\n", n, t);
    }
    return 0;
}
```

F.5 inventory.cpp: syllable inventory facts

Written for this book. Counts the generated entries, distinct syllables, prefixes and lengths reported in Chapters 2 and 5, and writes the list printed in Appendix D.

```
// Book measurements: facts about the syllable inventory, read from
Dokey Core's own tables.
// Usage: inventory <output file for the syllable list>
#include "dokey/engine.hpp"
#include "vietnamese.hpp"  // internal header in src/ (compile with -
Isrc): allSyllables()

#include <chrono>
#include <cstdio>
#include <fstream>
#include <map>
#include <set>

using namespace dokey;

static bool endsWith(const std::u16string& w, const char16_t* tail) {
    std::u16string t(tail);
    return w.size() ⩾ t.size() && w.compare(w.size() - t.size(),
t.size(), t) == 0;
}

static bool checked(const std::u16string& w) {
    return endsWith(w, u"c") || endsWith(w, u"ch") || endsWith(w, u"p")
|| endsWith(w, u"t") || endsWith(w, u"k");
}

int main(int argc, char** argv) {
```

```
    if (argc < 2) return 2;
    auto t0 = std::chrono::steady_clock::now();
    const auto& all = allSyllables();  // the first call builds the
dictionary
    auto t1 = std::chrono::steady_clock::now();
    std::set<std::u16string> unique(all.begin(), all.end());
    std::printf("vector entries %zu unique %zu build_ms %.2f\n",
all.size(), unique.size(),
                std::chrono::duration<double, std::milli>(t1 -
t0).count());

    std::set<std::u16string> prefixes;
    for (const auto& s : unique)
        for (size_t n = 1; n <= s.size(); n++)
prefixes.insert(s.substr(0, n));
    std::printf("distinct marked prefixes %zu\n", prefixes.size());

    size_t stops = 0, toned = 0;
    for (const auto& s : unique) {
        if (checked(s)) {
            stops++;
            toned += 3;  // no tone, acute, dot below
        } else {
            toned += 6;
        }
    }
    std::printf("checked syllables %zu, others %zu, toned forms %zu\n",
stops, unique.size() - stops, toned);

    std::map<size_t, size_t> byLength;
    for (const auto& s : unique) byLength[s.size()]++;
    for (const auto& kv : byLength) std::printf("len %zu: %zu\n",
kv.first, kv.second);

    std::map<std::u16string, int> count;
    for (const auto& s : all) count[s]++;
    int duplicates = 0;
    for (const auto& kv : count) {
        if (kv.second > 1) {
            std::printf("dup %s x%d\n", toUtf8(kv.first).c_str(),
kv.second);
            duplicates++;
        }
    }
    std::printf("dups %d\n", duplicates);

    std::ofstream out(argv[1], std::ios::binary);
    for (const auto& s : unique) out << toUtf8(s) << "\n";
    return 0;
}
```

F.6 trace.cpp: per-key traces

Written for this book. Prints the edit returned for every key of the example inputs quoted in Chapters 3, 4 and 7 and in Appendix E.

```
// Book measurements: the edit Dokey Core returns for every key of some
example inputs.
// "\b" is Backspace (through backspaceEdit) and "\x1b" is Escape
(undo), as in the tests.
#include "dokey/engine.hpp"

#include <algorithm>
#include <cstdio>
#include <string>

using namespace dokey;

static void trace(const char* label, const std::string& keys, const
Options& options) {
    Engine engine(options);
    std::u16string screen;
    std::printf("## %s\n", label);
    for (char16_t c : fromUtf8(keys)) {
        Edit edit;
        std::string name;
        if (c == u'\b') {
            edit = engine.backspaceEdit();
            name = "BS";
            if (!edit.handled && !screen.empty()) screen.pop_back();
        } else if (c == 0x1B) {
            edit = engine.undo();
            name = "ESC";
        } else {
            edit = engine.key(c);
            name = c == u' ' ? "SP" : std::string(1, static_cast<char>
(c));
            if (!edit.handled) screen.push_back(c);
        }
        if (edit.handled) {
            screen.erase(screen.size() - std::min<size_t>
(edit.backspaces, screen.size()));
            screen += edit.insert;
        }
        std::printf("%-4s | %-5s | %d | %-6s | %s\n", name.c_str(),
edit.handled ? "yes" : "no",
                    edit.handled ? edit.backspaces : 0, edit.handled ?
toUtf8(edit.insert).c_str() : "",
                    toUtf8(screen).c_str());
    }
}
```

```
int main() {
    Options telex;
    Options traditional = telex;
    traditional.modernToneStyle = false;
    Options vni = telex;
    vni.method = InputMethod::VNI;
    Options noSpell = telex;
    noSpell.checkSpelling = false;

    trace("tieengs", "tieengs ", telex);
    trace("tiengse", "tiengse ", telex);
    trace("dduowcj", "dduowcj ", telex);
    trace("dduocjw", "dduocjw ", telex);
    trace("windows", "windows ", telex);
    trace("windows nospell", "windows ", noSpell);
    trace("haxy", "haxy ", telex);
    trace("thuowr", "thuowr ", telex);
    trace("pass", "pass ", telex);
    trace("tesst", "tesst ", telex);
    trace("hoanf BS old", "hoanf\b ", traditional);
    trace("xin SP BS f", "xin \bf ", telex);
    trace("tieengs ESC", "tieengs\x1b ", telex);
    trace("vni d9u7o7ng2", "d9u7o7ng2 ", vni);
    trace("aaa", "aaa ", telex);
    trace("use", "use ", telex);
    trace("quas", "quas ", telex);
    trace("gifn", "gifn ", telex);
    trace("nguoiwf", "nguoiwf ", telex);
    trace("ddoox", "ddoox ", telex);
    trace("toans", "toans ", telex);
    return 0;
}
```

APPENDIX

G

The Korean Prototype

Version 0.5 of Dokey added a prototype composer for Korean, typed in the standard two-set layout (*2-beolsik*, KS X 5002). It is outside the main subject of this book, but it is part of the same library and the same C API, and it tests the claim that the pure-rebuild design is not specific to Vietnamese. This appendix summarises it from the project's design notes (`docs/hangul-design.md`) and its source (`src/hangul.cpp`, `include/dokey/hangul.hpp`). On both platforms it is labelled beta.

G.1 Motivation

Common Korean input methods on macOS compose text as *marked text*: the syllable being formed stays uncommitted in the text field until the user moves on. When the user presses Enter, the application receives the commit and the Enter key together. The design notes record that Chromium, Electron applications and terminal-based programs often handle the Enter before the commit, which loses or duplicates the last syllable. Dokey's approach never leaves anything uncommitted: the text on screen is always real text, corrected by edits. If Korean worked the same way, there would be nothing to commit when Enter arrives. The design notes are careful to call this an inference to be confirmed by testing, and list what has been verified.

G.2 The composer

`hangul::compose(keys)` is a pure function from Latin letter keys, read by their position on a US keyboard, to Hangul text.

- **Keys.** `q w e r t y u i o p` give ㅂ ㅈ ㄷ ㄱ ㅅ ㅛ ㅕ ㅑ ㅐ ㅔ; `a s d f g h j k l` give ㅁ ㄴ ㅇ ㄹ ㅎ ㅗ ㅓ ㅏ ㅣ; `z x c v b n m` give ㅋ ㅌ ㅊ ㅍ ㅠ ㅜ ㅡ. Shift matters only for `Q W E R T O P`, which give ㅃ ㅉ ㄸ ㄲ ㅆ ㅒ ㅖ.
- **Syllables.** An initial consonant, a vowel and an optional final are composed into one precomposed character, U+AC00 + (initial × 21 + vowel) × 28 + final.
- **Compound vowels and finals.** ㅗ + ㅏ ㅐ ㅣ, ㅜ + ㅓ ㅔ ㅣ and ㅡ + ㅣ form compound vowels; eleven pairs form double finals.
- **Re-syllabification.** A vowel after a final takes that final as its initial: 갑 + ㅏ → 가바. From a double final only the second consonant moves: 값 + ㅣ → 갑시.
- **Stand-alone jamo.** Letters that cannot combine are shown as compatibility jamo (U+3131 to U+3163).
- **Word breaks** pass through and are never handled, so Enter is never part of an edit.
- **Backspace** drops the last key and recomposes: 한 ⌫ → 하.
- **Long words** start over after 256 keys.

The engine around the composer is the same as for Vietnamese: it keeps the raw keys, recomposes after every key, and diffs the result against what it last showed. The difference is the edit rate. A Latin key never produces itself in Korean, so every letter key is rewritten. In the test program's 20,000 random words, edits deleted 0.33 and inserted 1.04 characters per key on average.

One behaviour differs from common Korean input methods, and the notes flag it for review by a native speaker: 가바 ⌫ gives 갑 in Dokey, because the last key is dropped and the rest recomposed, whereas input methods that edit the syllable being composed give 가ㅂ. Matching them would require state outside the key sequence.

G.3 Platform integration

On macOS, Korean reads letter keys by their key code, so the result does not depend on the selected keyboard layout, and Caps Lock is ignored; the user keeps the ABC input source. In the VS Code family, edits use Backspace instead of selection, because the integrated terminal reads Shift+Left Arrow as an escape sequence. In browsers and Electron applications, Korean text events carry the Space key code (Chapter 9). On Windows, letters are read from the virtual-key code, and the 한/영 key toggles Korean while Dokey types it.

G.4 Tests

`tests/hangul_tests.cpp` performs 53,204 checks: 86 hand-written cases with common words; all 11,172 precomposed syllables, each typed alone, followed by a syllable that keeps its final, and followed by a vowel that moves it; 20,000 random words checked after every key against `compose()` of the keys typed; and C API checks that switch to Korean and back. The design notes also record a mutation test: removing the ㄹ + ㄱ double final made 801 checks fail. Typing tests in VS Code on macOS are summarised in Chapter 13. The translations of the Korean user interface have not been reviewed by Korean speakers.

Glossary

Breve. The diacritic of *ă*. Typed `aw` in Telex and `8` in VNI.

Canonical keys. The key sequence the engine writes for a word on screen: each letter followed by its diacritic key, the tone key last. Used to resume a word after Backspace.

Checked syllable. A syllable ending in *c*, *ch*, *p* or *t* (and *k* in Central Highlands names). It can carry only the acute or dot-below tone.

Circumflex. The diacritic of *â*, *ê* and *ô*. Typed by doubling the vowel in Telex and `6` in VNI.

Combining encoding. Output in which the tone is a separate combining character (U+0300 to U+0323) after the vowel.

Commit. In Dokey, the step that compares the new word with the word on screen and produces an edit.

Compatible prefix. A string that becomes a syllable prefix when diacritics are added to its plain letters.

Diacritic. In this book, a mark that changes a letter (circumflex, breve, horn, stroke), as opposed to a tone.

Edit. The engine's output for one key: a number of characters to delete and a text to insert, or *not handled*.

Event tap. A macOS mechanism (`CGEventTap`) for observing and changing input events as they pass through the window server.

Gate. Dokey's use of syllable validity to decide whether a mark may be applied and whether a word is shown as Vietnamese.

Guard character. A character typed before deletions to replace any selected inline completion; U+00B7 in Dokey 0.5.4.

Horn. The diacritic of *ơ* and *ư*. Typed `w` in Telex and `7` in VNI.

Initial. The consonant or consonant cluster that begins a syllable; may be empty.

Inline completion. A suggestion that a text field types after the caret and selects, as in browser address bars.

Input method. Software that turns key sequences into text that the keyboard cannot type directly.

Literal mode. The state after undo, in which the rest of the word is typed exactly as keyed.

Low-level keyboard hook. A Windows hook (`WH_KEYBOARD_LL`) called for every keyboard event before it reaches any application.

Marked text. Uncommitted text that an input method composes inside an application's text field, typical of East Asian input methods.

NFC. Unicode Normalization Form C, which prefers precomposed characters.

Plausible. In Dokey, a word whose tone is allowed and whose letters are a prefix, or a compatible prefix, of some syllable.

Precomposed. A single code point for a letter with its diacritics and tone, such as U+1EBF for *ế*.

Pure function. A function whose result depends only on its arguments and which has no side effects.

Quick Telex. An optional extension in which a doubled initial consonant types a two-letter initial (`cc` → *ch*).

Rhyme. The part of a syllable after the initial: optional medial, nucleus and optional final.

Select-and-replace. Delivery that selects the old letters with Shift+Left Arrow and types the new text over the selection.

Stroke. The diacritic of *đ*. Typed `dd` in Telex and `9` in VNI, on the first letter only.

Synthetic event. A keyboard event created by software rather than by the keyboard.

Telex. The Vietnamese typing convention that uses letter keys for diacritics and tones.

Tone. One of six pitch contours of a Vietnamese syllable; five are written as a mark on one vowel.

VNI. The Vietnamese typing convention that uses digit keys for diacritics and tones.

Vowel group. The run of vowel letters that forms a syllable's nucleus, excluding the *u* of *qu* and the *i* of *gi* when another vowel follows.

Word break. A character that ends the current word: space, punctuation, Enter, Tab, and digits in Telex.

References

1. Do Huy Toan. *Dokey: A Vietnamese Input Method That Rebuilds the Whole Word on Every Keystroke* (Version 2). Working paper. Zenodo, 2026. https://doi.org/10.5281/zenodo.23888629
2. Do Huy Toan. Dokey source repository, specification (`SPEC.md`), test record (`docs/TESTING.md`), design notes (`docs/hangul-design.md`) and release notes, versions 0.1.0 to 0.5.4. https://github.com/dhtoan/Dokey
3. Apple Inc. Quartz Event Services: `CGEvent.tapCreate`, `CGEventTapPostEvent`, `CGEventSource`. Apple Developer Documentation. https://developer.apple.com/documentation/coregraphics/quartz_event_services
4. Apple Inc. Accessibility API (`AXUIElement`). Apple Developer Documentation. https://developer.apple.com/documentation/applicationservices/axuielement
5. Microsoft. `LowLevelKeyboardProc` callback function (`WH_KEYBOARD_LL`). Win32 documentation. https://learn.microsoft.com/windows/win32/winmsg/lowlevelkeyboardproc
6. Microsoft. `SendInput` function and `KEYBDINPUT` structure (`KEYEVENTF_UNICODE`). Win32 documentation. https://learn.microsoft.com/windows/win32/api/winuser/nf-winuser-sendinput
7. Microsoft. `ToUnicodeEx` function. Win32 documentation. https://learn.microsoft.com/windows/win32/api/winuser/nf-winuser-tounicodeex
8. Microsoft. `SetWinEventHook` function. Win32 documentation. https://learn.microsoft.com/windows/win32/api/winuser/nf-winuser-setwineventhook
9. The Unicode Consortium. *The Unicode Standard*, Latin Extended Additional (U+1E00–U+1EFF) and Combining Diacritical Marks (U+0300–U+036F). https://www.unicode.org/charts/
10. The Unicode Consortium. *Unicode Normalization Forms*, Unicode Standard Annex #15. https://unicode.org/reports/tr15/
11. Korean Agency for Technology and Standards. KS X 5002, keyboard layout for Korean (two-set). Referenced for the key assignment of the Korean prototype in Appendix G.

Index

Page numbers refer to the print edition. Code identifiers are set in monospace.

A

Accessibility permission, 77, 79, 134, 142
acute tone, 3, 10, 11, 13, 15, 19, 20, 22, 25–27, 37, 38, 46, …
address bar, 5, 72, 81, 92, 106, 111, 118, 120
`AppKind`, 80
Apple Silicon, 6, 77, 110, 119, 133, 142
`applyCircumflex`, 22, 24
`applyHornBreve`, 22, 24
`applyStroke`, 22, 25
`applyTone`, 22, 23
auto-repeat, 38, 71, 79, 151

B

Backspace, 4, 5, 7, 26, 31, 32, 38, 51, 55, 56, 59–62, 65, …
`backspaceEdit`, 60, 79, 90, 98, 101, 108
`bench.cpp`, 124–127
breve, 3, 4, 9–11, 19, 20, 22, 24, 28, 44, 148
`BuildInputs`, 92, 106, 111
bundle identifier, 80, 81, 86, 110

C

canonical keys, 26, 59, 60, 65, 108, 128
C API, 7, 8, 39, 59, 63, 64, 77, 85, 87, 95, 98, 100, …
Central Highlands, 13, 16, 109, 157, 158
`CGEventTap`, 7, 70, 73, 77–79, 82, 84, 85, 99, 120
checked syllable, 13–15, 44, 48, 108, 148
Chromium, 5, 70, 71, 74, 81, 84, 90, 93, 94, 119, 151, 195
circumflex, 3, 4, 9–11, 19, 20, 22, 24–26, 28, 44, 107, 148
code signing, 121, 134, 136, 137, 142
combining encoding, 11, 56, 57, 83, 107, 116, 120, 125, 155
commit (engine), 35–37, 61, 62
`completeUo`, 15, 34
`composeLetter`, 34, 55, 57
continuous integration, 87, 107, 110, 117, 119, 133, 135, 137, 141

D

`decomposeLetter`, 55
delivery profile, 75, 79, 81, 82, 128
diacritic, 3, 9–11, 16, 17, 21, 23–25, 42–44, 48, 51–57, 64, 107, 149
dictionary (syllable), 14–16, 42, 43, 47, 48, 54, 55, 74, 82, 97, 99, 100, …
`dokey_backspace`, 98
dokey (command-line tool), 7, 101, 131, 132
`dokey_create`, 97, 100
`dokey_edit`, 95–98, 100
`dokey_key`, 4, 69, 80, 90, 97, 100, 109, 123
`DOKEY_MAX_EDIT`, 39, 64, 96, 98
`dokey_options`, 96, 97, 100, 101
`dokey_reset`, 70, 99
`dokey_set_language`, 99, 101
`dokey_set_macros`, 63, 99, 101, 109
`dokey_undo`, 80, 90, 98, 101, 109
double letters (English), 38, 46–49, 63, 107, 116, 150
`dwExtraInfo`, 89, 91, 111

E

edit (engine output), 7, 35, 65, 69, 75
edit rate, 123, 126, 129, 196
Electron, 7, 70–72, 74, 80–82, 84, 93, 118–120, 142, 151, 195, 197
`encodeKeys`, 26, 60
end of word, 15, 34, 35, 37, 38, 45, 48, 49, 85, 110, 125, 150
English words, 5, 6, 27, 38, 41, 45, 46, 49, 63, 85, 106, 107, …

Escape (undo), 62, 63, 80, 90, 98, 109, 120, 151
excluded applications, 8, 48, 85
`expandMacro`, 64

F

`freeTextKey`, 83, 84, 86, 140

G

gate (spelling), 4, 7, 8, 33, 37, 41, 42, 44, 45, 47, 48, 64, ...
Google Docs, 74, 81, 82, 93, 120, 151
grave tone, 10, 11, 14, 19, 20, 22, 44, 55, 56, 84, 148, 154, ...
guard character, 7, 72–75, 82, 83, 86, 92, 111, 120, 140, 151

H

Hangul, 7, 80, 83, 84, 93, 99, 110, 116, 118, 120, 131, 141, ...
hook tone, 10, 56, 90, 148, 155
horn, 3, 4, 9–11, 15, 19, 20, 22, 24, 28, 44, 46, 53, ...

I

incremental input method, 6, 31, 51, 123
initial (consonant), 12–17, 27, 42, 54, 63, 119, 140, 157, 158, 196
inline completion, 72, 74, 75, 81, 83, 92, 94, 118, 120
Inno Setup, 131, 135, 137
`isCompatibleSyllablePrefix`, 43, 44, 128
`isSyllablePrefix`, 43, 44

K

kMaxWordKeys (32 keys), 34, 39, 64, 127–129

L

literal mode, 33, 62, 63, 65, 98, 109
low-level keyboard hook, 8, 87, 88, 90, 94

M

macOS 10.13, 77, 119, 133, 142
marked text, 77, 78, 195
Mark (enum), 11
modern tone style, 17, 53, 55, 61, 96, 124, 149

N

NFC, 11, 56, 154

P

pass-through, 4, 33, 34, 39, 62, 109, 110, 116, 140
pending (Windows queue), 89–91, 93, 94
plausible, 24, 27, 33, 34, 37, 38, 43–45, 48, 128, 148
precomposed, 11, 55–57, 83, 96, 124, 153, 154, 196, 197
ProseMirror, 5, 71, 118
pure function, 6, 57, 83, 92, 105, 139, 196

Q

Quick Telex, 27, 32, 34, 46, 63, 65, 96, 109, 116, 150

R

render (engine), 15, 33–35, 37, 44–46, 48, 52, 60, 85, 111, 112, 127, 128, ...
reopening a word, 32, 38, 59, 61, 62, 65, 98, 109, 116
reset, 38–40, 60, 65, 70, 78–80, 89, 90, 94, 95, 99, 151
rhyme, 12–17, 37, 42, 140, 157, 158
round trip (test), 26, 39, 54, 106, 108, 116, 140

S

secure input, 79, 84, 120
select-and-replace, 71–74, 81, 83, 118, 120, 126, 140, 142, 151, 197
self-test, 85, 106, 110–112, 133, 135
self-update, 118, 131, 136, 141
`SendInput`, 8, 70, 74, 88–91, 93, 94, 111, 151
SHA-256, 118, 133, 135–137
slow-machine profile, 74, 82, 119, 142
Spotlight, 81, 83, 86, 120
`stopFinal`, 13
stroke (đ), 4, 9–11, 19, 20, 25, 28, 44, 55, 148
synthetic event, 4–6, 8, 36, 39, 70, 71, 73, 74, 77, 110, 123, 126, ...

T

Telex, 3, 5–7, 9, 17, 19–28, 32–34, 37, 39, 41, 46, 47, 63–65, ...

text shortcuts (macros), 8, 32, 37, 48, 55, 57, 63, 97–99, 109, 150
tilde tone, 10, 11, 14, 19, 20, 44, 55, 56, 148, 154, 155
tone, 3–7, 9–17, 19–26, 28, 31, 33–35, 37–39, 42–46, 48, 51–57, 59–61, 64, ...
`tonePosition`, 26, 34, 51, 52, 149
`ToUnicodeEx`, 90
traditional tone style, 17, 51, 53, 55, 61, 98, 107, 110, 116, 180, 184
typed (keys as typed), 7, 32–37, 41, 44, 45, 47–49, 62–64, 92, 94, 96, 98, 109, ...

U

UIPI, 91, 94
undo by repetition, 20, 46, 149
Unicode, 11, 35, 51, 54–56, 75, 83, 86, 92, 96, 108, 124, 153
universal binary, 6, 77, 110, 133
ươ, 14–16, 19, 20, 24, 26, 33, 35–38, 42, 44–46, 48, 49, 52, ...

V

VNI, 4, 7, 11, 17, 19, 20, 22–28, 33, 34, 85, 101, 107, ...
vowel group, 13, 16, 23–25, 28, 37, 52–54, 128, 148, 149

W

word break, 20, 33, 37, 40, 45, 63, 64, 94, 98, 150, 196

Z

ziglang, 115

About the Author

Do Huy Toan is an American entrepreneur and independent researcher based in Delaware. He is the founder and chief executive of an e-commerce and digital media company, and he runs an online fashion store. His technical work centres on small, carefully tested software built by very small teams, often by one person working with modern AI coding tools.

He designed and built Dokey, the Vietnamese input method for macOS and Windows described in this book, and is the author of its working paper, *Dokey: A Vietnamese Input Method That Rebuilds the Whole Word on Every Keystroke* (Zenodo, 2026, DOI 10.5281/zenodo.23888629).

His name in Vietnamese is Đỗ Huy Toàn.

- Website: https://dhtoan.github.io/
- ORCID: https://orcid.org/0009-0007-5803-4405
- GitHub: https://github.com/dhtoan